

شهادة Claude Certified Architect — Foundations

دليل الدراسة (بناءً على دليل الاختبار الرسمي)

المقدمة

أن المتخصص قادر على اتخاذ قرارات موازنة سليمة **Claude Certified Architect — Foundations** تؤكد شهادة Claude Code و Claude Agent SDK يقيّم الاختبار المعرفة الأساسية بـ Claude. عند تنفيذ حلول واقعية مبنية على Claude. وهي التقنيات المحورية لبناء تطبيقات إنتاجية باستخدام، (MCP) Model Context Protocol و Claude API و

تعتمد أسئلة الاختبار على سيناريوهات صناعية واقعية: بناء أنظمة وكيلة لخدمة العملاء، وتصميم خطوط بحث متعددة وإنشاء أدوات إنتاجية للمطورين، واستخراج بيانات منظمة من مستندات غير، CI/CD في Claude Code الوكلاء، ودمج منظمة.

المرشح المستهدف

ينبغي أن تكون لديك خبرة عملية. Claude المرشح المثالي هو معماري حلول يصمم ويطلق تطبيقات إنتاجية باستخدام: لا تقل عن 6 أشهر في

- تنسيق عدة وكلاء، والتفويض إلى وكلاء فرعيين، وتكامل الأدوات، وخطافات دورة الحياة — Claude Agent SDK
- ومهارات الوكيل، ووضع التخطيط، MCP، وخواص، CLAUDE.md ملفات — Claude Code
- الأدوات والموارد لتكامل الأنظمة الخلفية — Model Context Protocol (MCP)
- وقوالب استخراج البيانات، few-shot وأمثلة، JSON هندسة المطالبات — مخططات
- نوافذ السياق — العمل مع المستندات الطويلة، وتمرير السياق بين عدة وكلاء
- مراجعة الكود الآلية، وتوليد الاختبارات — CI/CD خطوط
- التصعيد والموثوقية — معالجة الأخطاء، وإدخال الإنسان في الحلقة

صيغة الاختبار

البند	القيمة
نوع السؤال	اختبار من متعدد (إجابة صحيحة واحدة من 4)
نظام الدرجات	من 100 إلى 1000، ودرجة النجاح 720
عقوبة التخمين	لا توجد (أجب عن كل الأسئلة!)
السيناريوهات	من أصل 8 سيناريوهات محتملة (يتم اختيارها عشوائيًا) 4

محتوى الاختبار: 5 مجالات

الوزن	المجال
27%	بنية الوكلاء والتنسيق 1.
18%	MCP تصميم الأدوات وتكامل 2.
20%	وسير العمل Claude Code إعداد 3.
20%	هندسة المطالبات والمخرجات المنظمة 4.
15%	إدارة السياق والموثوقية 5.

سيناريوهات الاختبار

السيناريو 1: وكيل خدمة العملاء

يستخدم Claude Agent SDK تبني وكيلًا للتعامل مع عمليات الإرجاع، ونزاعات الفوترة، ومشكلات الحساب باستخدام أدوات MCP الوكيل (`get_customer` , `lookup_order` , `process_refund` , `escalate_to_human`). الهدف هو تحقيق أكثر من 80% من حل المشكلات من أول تواصل، مع التصعيد المناسب.

Claude Code السيناريو 2: توليد الكود باستخدام

لتسريع التطوير: توليد الكود، وإعادة الهيكلة، والتصحيح، والتوثيق. تحتاج إلى دمج مع أوامر Claude Code تستخدم `CLAUDE.md` مخصصة وإعدادات `slash` وفهم متى تستخدم وضع التخطيط.

السيناريو 3: نظام بحث متعدد الوكلاء

يفوض وكيل منسق المهام إلى وكلاء فرعيين متخصصين: بحث الويب، وتحليل المستندات، والتركيب، وتوليد التقارير. يجب أن ينتج النظام تقارير كاملة مع الاستشهادات.

السيناريو 4: أدوات إنتاجية المطورين

يساعد الوكيل المهندسين على استكشاف قواعد كود غير مألوفة، وتوليد كود نمطي، وأتمتة المهام الروتينية. يُستخدم MCP وخوادم (Read, Write, Bash, Grep, Glob) الأدوات المدمجة.

للتكامل المستمر Claude Code: السيناريو 5

لمراجعات الكود الآلية، وتوليد الاختبارات، وتقديم ملاحظات على طلبات السحب. CI/CD في خط Claude Code دمج. يجب تصميم المطالبات لتقليل الإيجابيات الكاذبة.

السيناريو 6: استخراج البيانات المنظمة

ويحافظ JSON، يستخرج النظام معلومات من مستندات غير منظمة، ويتحقق من المخرجات باستخدام مخططات على دقة عالية. يجب أن يتعامل بشكل صحيح مع الحالات الحدية.

السيناريو 7: أنماط معمارية الذكاء الاصطناعي الحواري

تصمم أنظمة حوار متعددة الأدوار تشمل إدارة نافذة السياق، واستمرار التعليمات عبر الأدوار، واستراتيجيات الذاكرة، وتصميم أدوات للتنفيذ الآمن، والتعامل مع مدخلات المستخدم الغامضة أو المتعارضة.

السيناريو 8: أدوات الذكاء الاصطناعي الوكيل (المحتوى ناقص — ساعدونا على إكماله!)

تم الإبلاغ عن هذا السيناريو من بعض المتقدمين للاختبار، لكنه غير مغطى بعد في هذا الدليل. إذا واجهت أسئلة من هذا حتى نضيف تغطية كاملة. ستساعد مساهمتك كل [GitHub Issues](#) السيناريو في الاختبار الحقيقي، يرجى مشاركتها في من يستعد للاختبار.

التوثيق الرسمي

المورد	الرابط
Claude API — Messages	https://platform.claude.com/docs/en/api/messages
Claude API — Tool Use	https://platform.claude.com/docs/en/build-with-claude/tool-use
Claude API — Message Batches	https://platform.claude.com/docs/en/build-with-claude/message-batches
Claude Agent SDK — Overview	https://platform.claude.com/docs/en/agent-sdk/overview
Claude Agent SDK — Hooks	https://platform.claude.com/docs/en/agent-sdk/hooks
Claude Agent SDK — Subagents	https://platform.claude.com/docs/en/agent-sdk/subagents
Claude Agent SDK — Sessions	https://platform.claude.com/docs/en/agent-sdk/sessions
Model Context Protocol (MCP)	https://modelcontextprotocol.io/
MCP — Tools	https://modelcontextprotocol.io/docs/concepts/tools
MCP — Resources	https://modelcontextprotocol.io/docs/concepts/resources
MCP — Servers	https://modelcontextprotocol.io/docs/concepts/servers
Claude Code — Documentation	https://code.claude.com/docs/en/overview
Claude Code — CLAUDE.md and Memory	https://code.claude.com/docs/en/memory
Claude Code — Skills (incl. slash commands)	https://code.claude.com/docs/en/skills
Claude Code — Hooks	https://code.claude.com/docs/en/hooks
Claude Code — Sub-agents	https://code.claude.com/docs/en/sub-agents
Claude Code — MCP Integration	https://code.claude.com/docs/en/mcp
Claude Code — GitHub Actions CI/CD	https://code.claude.com/docs/en/github-actions

Claude Code — GitLab CI/CD	https://code.claude.com/docs/en/gitlab-ci-cd
Claude Code — Headless (non-interactive mode)	https://code.claude.com/docs/en/headless
Prompt Engineering Guide	https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview
Extended Thinking	https://platform.claude.com/docs/en/build-with-claude/extended-thinking
Anthropic Cookbook (code examples)	https://github.com/anthropics/anthropic-cookbook

الجزء الأول: الأسس النظرية

يغطي هذا الجزء كل النظرية التي تحتاجها لاجتياز الاختبار بنجاح. تم تنظيم المادة حسب التقنيات والمفاهيم بدلاً من مجالات الاختبار، لأن ذلك يساعدك على بناء فهم أعمق لكل موضوع.

أساسيات التفاعل مع النموذج — Claude API: الفصل 1

التوثيق: [Messages API](#) | [Prompt Engineering](#)

1.1 API بنية طلب

ما يلي Claude Messages API نموذج طلب-استجابة. يتضمن كل طلب إلى Claude API يتبع:

```
{
  "model": "claude-sonnet-4-6",
  "max_tokens": 1024,
  "system": "You are a helpful assistant.",
  "messages": [
    {"role": "user", "content": "Hi!"},
    {"role": "assistant", "content": "Hello!"},
    {"role": "user", "content": "How are you?"},
  ],
  "tools": [...],
  "tool_choice": {"type": "auto"}
}
```

الحقول الأساسية:

- `model` — اختيار النموذج (`claude-opus-4-6` , `claude-sonnet-4-6` , `claude-haiku-4-5`)
- `max_tokens` — الحد الأقصى لعدد الرموز في الاستجابة
- `system` — مطالبة النظام (تحدد سلوك النموذج)

- `messages` — سجل المحادثة (يجب إرسال السجل الكامل للحفاظ على الاتساق)
- `tools` — تعريفات الأدوات المتاحة
- `tool_choice` — استراتيجية اختيار الأداة

1.2 أدوار الرسائل

ثلاثة أدوار `messages` تستخدم مصفوفة:

- `user` — رسائل المستخدم
- `assistant` — ردود النموذج (تُدرج عند إرسال السجل)
- `tool_result` (لا يتم تعيين الدور صراحة؛ تظهر كتلة محتوى) نتائج استدعاءات الأدوات — `tool`

يجب إرسال سجل المحادثة الكامل. لا يحتفظ النموذج بالحالة بين الطلبات، فكل استدعاء API مهم جدًا: في كل طلب مستقل.

1.3 في الاستجابة `stop_reason` حقل

وهو يوضح سبب توقف النموذج عن التوليد، `stop_reason` الحقل Claude API تتضمن استجابة:

القيمة	الوصف	الإجراء
"end_turn"	أنهى النموذج استجابته	اعرض النتيجة للمستخدم
"tool_use"	يريد النموذج استدعاء أداة	نفذ الأداة وأعد النتيجة
"max_tokens"	تم الوصول إلى حد الرموز	الاستجابة متبورة؛ قد تحتاج إلى زيادة الحد
"stop_sequence"	تمت مصادفة تسلسل إيقاف	عالجه حسب منطق تطبيقك

لأنهما تتحكمان في حلقة الوكيل "end_turn" و "tool_use" في الأنظمة الوكيلية، أهم قيمتين هما

1.4 مطالبة النظام

مطالبة النظام هي تعليمات خاصة تحدد السياق والقواعد السلوكية. وهي:

- `system`؛ تُمرر منفصلة في الحقل `messages` ليست جزءًا من مصفوفة
- لها أولوية على رسائل المستخدم
- تُحمّل مرة واحدة وتُطبق طوال المحادثة
- تُستخدم لتعريف الدور والقيود وصيغة الإخراج

مهم للاختبار: صياغة مطالبة النظام قد تنشئ ارتباطات غير مقصودة بالأدوات. مثلًا، تعليمة مثل "تحقق دائمًا من `get_customer` العميل" قد تجعل النموذج يفرط في استخدام

1.5 نافذة السياق

نافذة السياق هي إجمالي النص (بالرموز) الذي يستطيع النموذج معالجته دفعة واحدة. وتشمل:

- مطالبة النظام

- سجل الرسائل الكامل
- تعريفات الأدوات
- نتائج الأدوات

مشكلات نافذة السياق الأساسية:

1. تأثير ضياع المعلومات في الوسط: تعالج النماذج المعلومات في بداية المدخلات ونهايتها بموثوقية أكبر، وقد تفوت تفاصيل في الوسط. المعالجة: ضع المعلومات المهمة قرب البداية أو النهاية
2. تراكم نتائج الأدوات: يضيف كل استدعاء أداة مخرجات إلى السياق. إذا كانت الأداة ترجع أكثر من 40 حرفًا ولا يهم منها إلا 5، فمعظم السياق يُهدر
3. التلخيص التدريجي: عند ضغط السجل، غالبًا ما تُفقد القيم الرقمية والنسب والتواريخ وتتحول إلى تعبيرات مبهمه. "مثل" "تقريبًا" و"نحو" و"بعض".

tool_use الفصل 2: الأدوات و

Tool Use: التوثيق

2.1 ما هو tool_use

باستدعاء وظائف خارجية. النموذج لا يشغل الكود مباشرة، بل يولد طلب استدعاء Claude آلية تسمح لـ tool_use بأداة منظمًا؛ ثم ينفذ كودك ذلك الطلب ويرجع النتيجة

2.2 تعريف الأداة

JSON تُعرّف كل أداة باستخدام مخطط:

```
{
  "name": "get_customer",
  "description": "Finds a customer by email or ID. Returns the customer profile, including name, email, order history, and account status. Use this tool BEFORE lookup_order to verify the customer's identity. Accepts an email (format: user@domain.com) or a numeric customer_id.",
  "input_schema": {
    "type": "object",
    "properties": {
      "email": {"type": "string", "description": "Customer email"},
      "customer_id": {"type": "integer", "description": "Numeric customer ID"}
    },
    "required": []
  }
}
```

نقاط مهمة جدًا في وصف الأداة:

الوصف هو آلية الاختيار الأساسية. يختار نموذج اللغة الأدوات بناءً على أوصافها. الأوصاف المختصرة مثل "يجلب معلومات العميل" تؤدي إلى أخطاء عند تداخل الأدوات.

2. ضمّن في الوصف:

- ما الذي تفعله الأداة وما الذي ترجمه
 - صيغ الإدخال وقيم أمثلة
 - الحالات الحدية والقيود
 - متى تستخدم هذه الأداة مقارنة بالبدائل المشابهة
- يمكن أن `analyze_document` و `analyze_content` تجنب الأوصاف المتطابقة أو المتداخلة بين الأدوات. إذا كان أوصافًا شبه متطابقة، فسوف يخلط النموذج بينهما.
- ذات MCP على أدوات (Read, Grep) قد يفضل الوكلاء الأدوات المدمجة: **MCP الأدوات المدمجة مقابل أدوات** وابرز المزايا المحددة أو البيانات الفريدة أو السياق الذي لا MCP، الوظائف المشابهة. لمنع ذلك، قوّ أوصاف أدوات تستطيع الأدوات المدمجة توفيره.

2.3 معامل `tool_choice`

في كيفية اختيار النموذج للأدوات `tool_choice` يتحكم

القيمة	السلوك	متى تستخدمه
<code>{"type": "auto"}</code>	يقرر النموذج هل يستدعي أداة أم يجب نصيًا	الافتراضي في معظم الحالات
<code>{"type": "any"}</code>	يجب على النموذج استدعاء أداة ما	عندما تحتاج إلى إخراج منظم مضمون
<code>{"type": "tool", "name": "extract_metadata"}</code>	يجب على النموذج استدعاء أداة محددة	عندما تحتاج إلى خطوة أولى/ترتيب تنفيذ مضمون

سيناريوهات مهمة:

- أدوات استخراج متعددة → يختار النموذج أفضل أداة، لكنك تحصل على إخراج منظم + `tool_choice: "any"` على أي حال
- قبل الإثراء `extract_metadata` (مثل) الاختيار الإجباري → عندما يجب ضمان إجراء أول محدد

2.4 للمخرجات المنظمة JSON مخططات

فهو Claude. هو أوثق طريقة للحصول على إخراج منظم من JSON مع مخططات `tool_use` استخدام

- صحيًا نحويًا (لا أقواس ناقصة ولا فواصل زائدة) JSON يضمن
- يفرض البنية المطلوبة (الحقول المطلوبة موجودة)
- لا يضمن الصحة الدلالية (قد تكون القيم خاطئة)

تصميم المخطط — مبادئ أساسية:

```
{
  "type": "object",
  "properties": {
    "category": {
      "type": "string",
      "enum": ["bug", "feature", "docs", "unclear", "other"]
    },
    "category_detail": {
      "type": ["string", "null"],
      "description": "Details if category = 'other' or 'unclear'"
    },
    "severity": {
      "type": "string",
      "enum": ["critical", "high", "medium", "low"]
    },
    "confidence": {
      "type": "number",
      "minimum": 0,
      "maximum": 1
    },
    "optional_field": {
      "type": ["string", "null"],
      "description": "Null if the information was not found in the source"
    }
  },
  "required": ["category", "severity"]
}
```

قواعد تصميم المخطط:

- الحقول المطلوبة مقابل الاختيارية: اجعل الحقل مطلوبًا فقط إذا كانت المعلومة متاحة دائمًا. الحقول المطلوبة تدفع النموذج إلى اختلاق قيم عندما تكون البيانات مفقودة.
- للمعلومات التي قد تكون غائبة. يستطيع `"type": ["string", "null"]` استخدام `null` الحقول القابلة للقيمة بدلاً من الهلوسة `null` النموذج إرجاع.
- مع حقل تفاصيل لتجنب فقدان البيانات خارج الفئات المحددة مسبقًا `"other"` أضف `"other"` مع Enums.
- الصادقة أفضل من فئة `"unclear"` للحالات التي لا يستطيع فيها النموذج اختيار فئة بثقة؛ `"unclear"` Enum خاطئة.

الأخطاء النحوية مقابل الدلالية 2.5

نوع الخطأ	مثال	التخفيف
نحوي	غير صالح، نوع حقل خاطئ JSON	(يزيله) JSON مع مخطط <code>tool_use</code>
دلالي	الإجماليات لا تتطابق، قيمة في حقل خاطئ، هلوسة	فحوصات تحقق، إعادة المحاولة مع تغذية راجعة، التصحيح الذاتي

بناء الأنظمة الوكيلية — Claude Agent SDK: الفصل 3

التوثيق: [Agent SDK](#) | [Hooks](#) | [Subagents](#) | [Sessions](#)

3.1 ما هي حلقة الوكيل

حلقة الوكيل هي النمط الأساسي لتنفيذ المهام ذاتيًا. النموذج لا يجيب فقط، بل ينفذ سلسلة من الإجراءات:

1. Send a request to Claude with tools
2. Receive a response
3. Check stop_reason:
 - "tool_use" -> execute the tool, append the result to history, go back to step 1
 - "end_turn" -> the task is complete, show the result to the user
4. Repeat until completion

الأداة التالية بناءً على السياق ونتائج الأدوات السابقة. وهذا يختلف عن أشجار Claude هذا نهج يقوده النموذج: يقرر القرار المبرمجة مسبقًا التي يكون فيها تسلسل الإجراءات ثابتًا.

أنماط مضادة (تجنبها):

- ("Task completed") تحليل نص المساعد لاكتشاف الاكتمال
- كإشارة إيقاف أساسية (مثل `max_iterations=5`) استخدام حد تكرار اعتباطي
- التحقق مما إذا كان المساعد أنتج نصًا كإشارة اكتمال

النهج الصحيح: إشارة الاكتمال الموثوقة الوحيدة هي `stop_reason == "end_turn"`.

3.2 إعداد AgentDefinition

Claude Agent SDK هو كائن إعداد الوكيل في `AgentDefinition`:

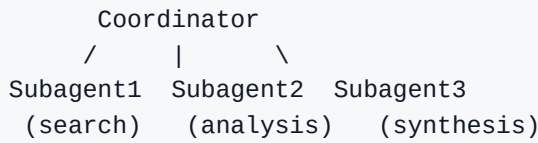
```
agent = AgentDefinition(  
    name="customer_support",  
    description="Handles customer requests for returns and order issues",  
    system_prompt="You are a customer support agent...",  
    allowed_tools=["get_customer", "lookup_order", "process_refund",  
    "escalate_to_human"],  
    # For a coordinator:  
    # allowed_tools=["Task", "get_customer", ...]  
)
```

المعاملات الأساسية:

- `name` / `description` — تعريف الوكيل ووصفه
- `system_prompt` — مطالبة النظام مع التعليمات
- `allowed_tools` — قائمة الأدوات المسموحة (مبدأ أقل صلاحية)

نموذج المحور والأذرع: المنسق والوكلاء الفرعيون 3.3

عادة تُبنى البنية متعددة الوكلاء كطوبولوجيا محور وأذرع:



مسؤوليات المنسق:

- تفكيك المهمة إلى مهام فرعية
- تحديد الوكلاء الفرعيين المطلوبين (اختيار ديناميكي)
- تفويض العمل للوكلاء الفرعيين
- تجميع النتائج والتحقق منها
- معالجة الأخطاء وإعادة المحاولة
- إيصال النتائج إلى المستخدم

مبدأ حرج: الوكلاء الفرعيون لديهم سياق معزول

- لا يرث الوكلاء الفرعيون تلقائيًا سجل محادثة المنسق
- يجب تمرير كل السياق المطلوب صراحةً في مطالبة الوكيل الفرعي
- لا يتشارك الوكلاء الفرعيون الذاكرة بين الاستدعاءات
- تمر كل الاتصالات عبر المنسق (للملاحظة والتحكم في الأخطاء)

لإنشاء الوكلاء الفرعيين Task أداة 3.4

Task: يتم إنشاء الوكلاء الفرعيين عبر أداة

```
# The coordinator's allowedTools must include "Task"
coordinator_agent = AgentDefinition(
    allowed_tools=["Task", "get_customer"]
)
```

تمرير السياق الصريح إلزامي:

```
# Bad: the subagent has no context
Task: "Analyze the document"

# Good: full context in the prompt
Task: "Analyze the following document.
Document: [full document text]
Prior search results: [web search results]
Output format requirements: [schema]"
```

في استجابة واحدة، فتعمل الوكلاء الفرعيون بالتوازي **Task** **الإنشاء المتوازي:** يستطيع المنسق استدعاء عدة

```
# One coordinator response contains:
Task 1: "Search for articles about X"
Task 2: "Analyze document Y"
Task 3: "Search for articles about Z"
# All three run concurrently
```

3.5 Agent SDK الخطافات في

تسمح الخطافات بالاعتراض والتحويل عند نقاط محددة في دورة حياة الوكيل.

يعترض نتيجة الأداة قبل تزويد النموذج بها `PostToolUse`:

```
# Example: normalize date formats from different MCP tools
@hook("PostToolUse")
def normalize_dates(tool_result):
    # Convert Unix timestamp -> ISO 8601
    # Convert "Mar 5, 2025" -> "2025-03-05"
    return normalized_result
```

خطاف اعتراض الاستدعاء الصادر يمنع الإجراءات التي تنتهك السياسة:

```
# Example: block refunds above $500
@hook("PreToolUse")
def enforce_refund_limit(tool_call):
    if tool_call.name == "process_refund" and tool_call.args.amount > 500:
        return redirect_to_escalation(tool_call)
```

الفرق الأساسي: الخطافات مقابل تعليمات المطالبة

الخاصية	الخطافات	تعليمات المطالبة
الضمان	حتمي (100%)	احتمالي (<90%، وليس 100%)
متى تُستخدم	قواعد الأعمال الحرجة، العمليات المالية، الامتثال	التفضيلات العامة، التوصيات، التنسيق
مثال	منع المبالغ المستردة فوق 500 دولار	"Try to solve before escalating"

القاعدة: عندما يؤدي الفشل إلى عواقب مالية أو قانونية أو تتعلق بالسلامة، استخدم الخطافات لا المطالبات.

4 الفصل: Model Context Protocol (MCP)

التوثيق: [MCP](#) | [Tools](#) | [Resources](#) | [Servers](#)

4.1 ما هو MCP

Model Context Protocol (MCP) هو بروتوكول مفتوح لربط الأنظمة الخارجية بـ Claude. ثلاثة أنواع MCP يعرّف:

أساسية من الموارد:

1. **Tools** — (تنفيذ أوامر، API استدعاءات، CRUD عمليات) وظائف يستطيع الوكيل استدعاءها لتنفيذ إجراءات
2. **Resources** — بيانات يستطيع الوكيل قراءتها كسياق (توثيق، مخططات قواعد بيانات، فهارس محتوى)
3. **Prompts** — قوالب مطالبات معدة مسبقًا للمهام الشائعة

4.2 خوادم MCP

MCP وتوفر أدوات/موارد. عند الاتصال بخادم MCP هو عملية تنفذ بروتوكول MCP خادم:

- يتم اكتشاف كل الأدوات تلقائيًا
- تكون أدوات كل الخوادم المتصلة متاحة في الوقت نفسه
- تحدد أوصاف الأدوات كيفية استخدام النموذج لها

4.3 MCP إعداد خوادم

للاستخدام الفريق — (`.mcp.json`) إعداد المشروع:

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "${GITHUB_TOKEN}"
      }
    },
    "jira": {
      "command": "npx",
      "args": ["-y", "mcp-server-jira"],
      "env": {
        "JIRA_TOKEN": "${JIRA_TOKEN}"
      }
    }
  }
}
```

نقاط أساسية:

- في جذر المشروع ويُدَار ضمن نظام التحكم بالإصدارات `.mcp.json` يُخزن
- للأسرار؛ لا تُحفظ الرموز نفسها في المستودع (`${GITHUB_TOKEN}`) تُستخدم متغيرات البيئة
- يكون متاحًا لكل المساهمين في المشروع

للخوادم الشخصية/التجريبية — (`~/.claude.json`) إعداد المستخدم:

- يُخزن في مجلد المنزل للمستخدم

- لا يُشارك عبر نظام التحكم بالإصدارات
- مناسب للتجارب الشخصية والاختبار

اختيار الخوادم

- المجتمعية الحالية MCP فضّل خوادم (Jira, GitHub, Slack) للتكاملات القياسية
- ابنِ خوادمك الخاصة فقط لسير العمل الفريدة الخاصة بالفريق

4.4 رؤية `isError` في MCP

في الاستجابة. هذا يشير للوكيل إلى أن الاستدعاء فشل `isError: true` خطأ، تستخدم MCP عندما تواجه أداة

خطأ منظم (جيد):

```
{
  "isError": true,
  "content": {
    "errorCategory": "transient",
    "isRetryable": true,
    "message": "The service is temporarily unavailable. Timeout while calling the orders API.",
    "attempted_query": "order_id=12345",
    "partial_results": null
  }
}
```

خطأ عام (نمط مضاد):

```
{
  "isError": true,
  "content": "Operation failed"
}
```

الخطأ العام لا يعطي الوكيل أي معلومات لاتخاذ القرار: هل يعيد المحاولة، أم يغير الاستعلام، أم يصعد؟

4.5 موارد MCP

الموارد هي بيانات يستطيع الوكيل طلبها للحصول على سياق دون تنفيذ إجراءات:

- فهارس المحتوى (مثل قائمة بكل مهام المشروع أو تنقل هرمي)
- مخططات قواعد البيانات (لفهم بنية البيانات)
- (أدلة داخلية، API مراجع) التوثيق
- ملخصات القضايا/المهام

ميزة الموارد: لا يحتاج الوكيل إلى استدعاءات أدوات استكشافية لفهم البيانات الموجودة. يقدّم المورد "خريطة" فورية

الإعداد وسير العمل — Claude Code: الفصل 5

التوثيق: [Claude Code](#) | [Memory / CLAUDE.md](#) | [Skills](#) | [MCP](#) | [Hooks](#) | [Sub-agents](#) | [GitHub Actions](#) | [Headless](#)

5.1 هرمية CLAUDE.md

توجد هرمية من ثلاثة مستويات. Claude Code هو ملف أو ملفات التعليمات الخاصة بـ CLAUDE.md:

1. User-level: `~/.claude/CLAUDE.md`
 - Applies only to that user
 - NOT shared via VCS
 - Personal preferences and working style
2. Project-level: `.claude/CLAUDE.md` or a root `CLAUDE.md`
 - Applies to all project contributors
 - Managed via VCS
 - Coding standards, testing standards, architectural decisions
3. Directory-level: `CLAUDE.md` in subdirectories
 - Applies when working with files in that directory
 - Conventions specific to that part of the codebase

مستوى `~/.claude/CLAUDE.md` خطأ شائع: لا يحصل عضو فريق جديد على تعليمات المشروع لأنها وُضعت في `.claude/CLAUDE.md` بدلاً من (المستخدم (مستوى المشروع).

5.2 صيغة @path (استيراد الملفات)

مما يجعل الإعداد معيارياً، `@path` إلى ملفات خارجية باستخدام `CLAUDE.md` يمكن أن يشير:

Project CLAUDE.md

Coding standards are described in `@./standards/coding-style.md`
Test requirements are in `@./standards/testing-requirements.md`
Project overview is in `@README.md` and dependencies are in `@package.json`

قواعد `@path`:

- استخدم `@` مباشرة قبل مسار الملف (دون مسافة)
- المسارات النسبية والمطلقة مدعومة
- تُحل المسارات النسبية بالنسبة للملف الذي يحتوي على الاستيراد
- أقصى عمق لتداخل الاستيراد هو 5

يساعد ذلك على تجنب التكرار و يتيح لكل حزمة تضمين المعايير ذات الصلة فقط.

5.3 مجلد `.claude/rules/`

ضخم، ويستخدم لتنظيم القواعد حسب الموضوع `CLAUDE.md` بديل عن ملف `.claude/rules/`.

```
.claude/rules/
testing.md          -- testing conventions
api-conventions.md  -- API conventions
deployment.md       -- deployment rules
react-patterns.md   -- React patterns
```

للتحميل الشرطي `paths` مع **YAML frontmatter**: الميزة الأساسية

```
---
paths: ["src/api/**/*"]
---
```

For API files, use `async/await` with explicit error handling.
Each endpoint must return a standard response wrapper.

```
---
paths: ["**/*.test.tsx", "**/*.test.ts"]
---
```

Tests must use `describe/it` blocks.
Use data factories instead of hardcoding.
Do not mock the database—use a test database.

كيف يعمل:

- ملفًا يطابق نمط `Claude Code` تُحمّل القاعدة فقط عندما يحرر `paths`
- هذا يوفر السياق والرموز لأن القواعد غير ذات الصلة لا تُحمّل
- تطبيق الاصطلاحات حسب نوع الملف بغض النظر عن الموقع (مثالي للاختبارات المنتشرة في `Glob` تتيح أنماط قاعدة الكود)

على مستوى المجلد `CLAUDE.md` مقابل `paths` مع `.claude/rules/` متى تستخدم

- عندما تنطبق الاصطلاحات على ملفات منتشرة عبر مجلدات كثيرة — `paths` مع `.claude/rules/` (الاختبارات، `migrations`)
- على مستوى المجلد — عندما تكون الاصطلاحات مرتبطة بمجلد محدد ولا تحتاج في غيره `CLAUDE.md`

5.4 المخصصة والمهارات `Slash` أوامر

(`.claude/commands/`) تم توحيد الأوامر المخصصة، `Claude Code` ملاحظة: في الإصدار الحالي من يشير دليل الاختبار إلى `/name` كلا التنسيقين ينشئ أوامر. (`.claude/skills/`) مع المهارات `.claude/commands/`، وهذا التنسيق لا يزال مدعومًا.

: `/name` هي قوالب مطالبات قابلة لإعادة الاستخدام وتُستدعى عبر `Slash` أوامر

تنسيق `.claude/commands/` (قديم لكنه مدعوم):

```
.claude/commands/
review.md          -- /review -- standard code review
test-gen.md        -- /test-gen -- test generation
```

تنسيق `.claude/skills/` (الحالي):

```
.claude/skills/
review/SKILL.md    -- /review -- with frontmatter configuration
test-gen/SKILL.md
```

أوامر المشروع (`.claude/commands/` أو `.claude/skills/`):

- وتتاح للجميع عند استنساخ الـ VCS تُخزن في
- تضمن سير عمل متسقًا عبر الفريق

أوامر المستخدم (`~/claude/commands/` أو `~/claude/skills/`):

- VCS أوامر شخصية لا تُشارك عبر
- لسير العمل الفردي

5.5 المهارات — `.claude/skills/`

frontmatter المهارات أوامر متقدمة تُعد عبر SKILL.md:

```
---
context: fork
allowed-tools: ["Read", "Grep", "Glob"]
argument-hint: "Path to the directory to analyze"
---

Analyze the code structure in the specified directory.
Output a report on dependencies and architectural patterns.
```

frontmatter معاملات:

المعامل	الوصف
<code>context: fork</code>	يشغّل المهارة في وكيل فرعي معزول. لا يلوّث الإخراج المطول الجلسة الرئيسية
<code>allowed-tools</code>	يقيّد الأدوات المتاحة (أمان؛ مثلًا لا تستطيع المهارة حذف ملفات إذا لم يُسمح لها)
<code>argument-hint</code>	تلميح يطلب معاملاً عند الاستدعاء دون معاملات

CLAUDE.md متى تستخدم مهارة مقابل:

- **المهارة** — استدعاء عند الحاجة لمهمة محددة (مراجعة، تحليل، توليد)
- **CLAUDE.md** — معايير واصطلاحات عامة تُحمّل دائمًا

المهارات الشخصية (`~/claude/skills/`):

- أنشئ نسجًا شخصية بأسماء مختلفة حتى لا تؤثر على زملائك

5.6 وضع التخطيط مقابل التنفيذ المباشر

وضع التخطيط:

- يحقق النموذج ويخطط فقط؛ لا يجري تغييرات
- لاستكشاف قاعدة الكود Glob وGrep وRead يستخدم
- ينتج خطة تنفيذ يوافق عليها المستخدم
- استكشاف آمن دون آثار جانبية

متى تستخدم وضع التخطيط:

- تغييرات كبيرة (عشرات الملفات)
- (كيف نحدد الحدود؟: microservices) وجود عدة مقاربات محتملة
- قرارات معمارية (أي إطار؟ أي بنية؟)
- قاعدة كود غير مألوفة (يجب الفهم قبل التغيير)
- ترحيلات مكتبات تؤثر على أكثر من 45 ملفًا

متى تستخدم التنفيذ المباشر:

- واضح stack trace إصلاحات في ملف واحد مع
- إضافة تحقق واحد
- تغييرات مفهومة وواضحة وغير ملتبسة

نهج مدمج:

1. وضع التخطيط للتحقيق والتصميم
2. يوافق المستخدم على الخطة
3. التنفيذ المباشر لتطبيق الخطة المعتمدة

وكيل فرعي متخصص لاستكشاف قاعدة الكود — Explore subagent

- يعزل الإخراج المطول عن السياق الرئيسي
- يرجع ملخصًا فقط
- يمنع استنزاف نافذة السياق في المهام متعددة المراحل

5.7 الأمر /compact

أمر مدمج لضغط السياق /compact:

- يلخص السجل السابق لتحرير نافذة السياق
- يُستخدم في جلسات التحقيق الطويلة عندما يمتلئ السياق بإخراج أدوات مطول
- الخطر: قد تُفقد القيم الرقمية والتواريخ والتفاصيل الدقيقة أثناء التلخيص

5.8 الأمر /memory

أمر مدمج لإدارة الذاكرة بين الجلسات /memory:

- للتحريّر، مما يسمح بحفظ الملاحظات والتفضيلات والسياق `CLAUDE.md` يفتح ملف
- تستمر المعلومات بين الجلسات وتُحمّل تلقائيًا عند البدء
- مفيد لتخزين اصطلاحات المشروع، وتفضيلات المستخدم، والأوامر المتكررة، وسياق العمل الحالي
- بديل عن إعادة شرح التعليمات نفسها في كل جلسة

5.9 Claude Code CLI لـ CI/CD واجهة

العلم `--print` (أو `-p`):

```
claude -p "Analyze this pull request for security issues"
```

- ثم يخرج `stdout` وضع غير تفاعلي: يعالج المطالبة، يطبع إلى
- لا ينتظر إدخال المستخدم
- في `CI/CD` خطوط Claude الطريقة الصحيحة الوحيدة لتشغيل

CI: إخراج منظم لـ

```
claude -p "Review this PR" --output-format json --json-schema  
'{"type":"object",...}'
```

- `--output-format json` — الإخراج بصيغة JSON
- `--json-schema` — التحقق من الإخراج مقابل مخطط
- داخلية تلقائيًا PR يمكن تحليل النتيجة لنشر تعليقات

عزل سياق الجلسة: الجلسة نفسها التي ولدت الكود غالبًا تكون أقل فعالية في مراجعته، لأن النموذج يحتفظ بسياق تفكيره ويكون أقل ميلًا لتحدي قراراته. استخدم مئيلاً مستقلاً للمراجعة

جديدة، ضمّن نتائج المراجعة السابقة في السياق واطلب **commits منع التعليقات المكررة**: عند إعادة المراجعة بعد الإبلاغ فقط عن المشكلات الجديدة أو غير المحلولة Claude من

5.10 fork_session وإدارة الجلسات

`--resume <session-name>` يستأنف جلسة مسماة:

```
claude --resume investigation-auth-bug
```

- يواصل محادثة سابقة مع سياق محفوظ
- مفيد للتحقيقات الطويلة عبر عدة جلسات
- الخطر: إذا تغيرت الملفات منذ الجلسة السابقة، قد تكون نتائج الأدوات قديمة

`fork_session` ينشئ فرعًا مستقلاً من سياق مشترك:

```
Codebase investigation
  |
  fork_session
  /      \
Approach A:  Approach B:
Redux       Context API
```

- يرث الفرعان السياق حتى نقطة التفرع
- بعد ذلك يتباعدان بشكل مستقل
- مفيد لمقارنة المقاربات أو اختبار الاستراتيجيات

حتى تبدأ جلسة جديدة بدلاً من الاستئناف

- نتائج الأدوات قديمة (تغيرت الملفات)
- مر وقت طويل وتدهور السياق
- بدلاً من الاستئناف بسياق أدوات قديم "Here is a short summary of what we found: ..." الأفضل أن تبدأ بـ

الفصل 6: هندسة المطالبات — تقنيات متقدمة

التوثيق: [Prompt Engineering](#) | [Anthropic Cookbook](#)

6.1 Few-shot Prompting بالمثلة قليلة

هو تضمين 2-4 أمثلة إدخال/إخراج في المطالبة لتوضيح السلوك المتوقع Few-shot prompting.

أكثر فعالية من الوصف النصي few-shot لماذا تكون أمثلة:

- تعليمية مبهمة مثل "كن أكثر دقة" يمكن تفسيرها بطرق كثيرة
- المثال يوضح بلا لبس الصيغة ومنطق القرار المتوقعين
- يعمم النموذج النمط على حالات جديدة (لا يكرر الأمثلة فقط)

ومتي تستخدمها few-shot أنواع أمثلة:

1. أمثلة للسيناريوهات الغامضة:

```
Request: "My order is broken"
Action: Call get_customer -> lookup_order -> check status.
Rationale: "broken" may mean a damaged item; you need order details.
```

```
Request: "Get me a manager"
Action: Immediately call escalate_to_human.
Rationale: The customer explicitly requests a human. Do not attempt to solve autonomously.
```

2. أمثلة لتنسيق الإخراج:

Finding example:

```
{
  "location": "src/auth/login.ts:42",
  "issue": "SQL injection in the username parameter",
  "severity": "critical",
  "suggested_fix": "Use a parameterized query"
}
```

3. أمثلة للتمييز بين الكود المقبول والمشكلة:

```
// Acceptable (do not flag):
const items = data.filter(x => x.active);

// Problem (flag):
const items = data.filter(x => x.active == true); // Use strict equality ===
```

4. أمثلة للاستخراج من صيغ مستندات مختلفة:

Document with inline citations:

```
"As shown in the study (Smith, 2023), the rate is 42%."
-> {"value": "42%", "source": "Smith, 2023", "type": "inline_citation"}
```

Document with bibliography references:

```
"The rate is 42%. [1]"
-> {"value": "42%", "source": "reference_1", "type": "bibliography"}
```

5. أمثلة للقياسات غير الرسمية:

```
Text: "about two handfuls of rice"
-> {"amount": "~100g", "original_text": "two handfuls", "precision": "approximate"}
```

```
Text: "a pinch of salt"
-> {"amount": "~1g", "original_text": "a pinch", "precision": "approximate"}
```

فعالة خصوصًا لاستخراج وحدات القياس غير الرسمية وغير القياسية التي يصعب تغطيتها بتعليمات few-shot تكون قاعدية فقط.

صارمة للمخرجات المنظمة، أضف قواعد تطبيع JSON قواعد تطبيع الصيغة في المطالبات: عند استخدام مخططات في المطالبة:

Normalization:

- Dates: always ISO 8601 (YYYY-MM-DD); "yesterday" -> compute an absolute date
- Currency: numeric amount + currency code; "five bucks" -> {"amount": 5, "currency": "USD"}
- Percentages: decimal fraction; "half" -> 0.5

صحيحًا نحويًا لكن القيم غير متسقة JSON هذا يمنع الأخطاء الدلالية حيث يكون

6.2 معايير صريحة مقابل تعليمات مبهمه

سيئ (مبهم):

Check code comments for accuracy.
Be conservative—report only high-confidence findings.

جيد (معايير صريحة):

Flag a comment as problematic ONLY if:

1. The comment describes behavior that CONTRADICTS the actual code behavior
2. The comment references a non-existent function or variable
3. A TODO/FIXME comment refers to a bug that has already been fixed in code

Do NOT flag:

- Comments that are merely stylistically outdated
- Comments with minor wording inaccuracies
- Missing comments (that is a separate category)

عرّف معايير الشدة مع أمثلة:

CRITICAL: Runtime failure for users
Example: NullPointerException while processing a payment

HIGH: Security vulnerability
Example: SQL injection, XSS, missing authorization checks

MEDIUM: Logic bug without immediate impact
Example: Wrong sorting, off-by-one error

LOW: Code quality
Example: Duplication, suboptimal algorithm for small data

6.3 Prompt Chaining سلاسل المطالبات

تكسر سلاسل المطالبات المهمة المعقدة إلى خطوات مركزة متتابعة:

Step 1: Analyze auth.ts (local issues only)
-> Output: list of issues in auth.ts

Step 2: Analyze database.ts (local issues only)
-> Output: list of issues in database.ts

Step 3: Integration pass (cross-file dependencies)
-> Output: issues at module boundaries

لماذا هذا مهم:

- يتجنب تشتت الانتباه؛ عندما يستقبل النموذج ملفات كثيرة دفعة واحدة، قد يفوت أخطاء في بعضها ويقدم تعليقات سطحية على أخرى
- يضمن جودة تحليل متنسقة لكل ملف

- يسمح بتحليل منفصل للتفاعلات بين الملفات

مقابل التفكير الديناميكي prompt chaining متى تستخدم

- **Prompt chaining** — مهام متوقعة ومتكررة (مراجعة كود، ترحيلات ملفات)
- **التفكير الديناميكي** — تحقيقات مفتوحة تتضح مهامها الفرعية أثناء التنفيذ

6.4 "نمط" المقابلة

أسئلة توضيحية Claude قبل تنفيذ حل، يطرح

```
Claude: "Before implementing caching for the API, a few questions:
1. Which cache invalidation strategy do you prefer—TTL or event-based?
2. Is stale data acceptable when the cache is unavailable?
3. Should caching be per-user or global?
4. What is the expected data volume to cache?"
```

متى يكون مفيدًا:

- (الرعاية الصحية، الأنظمة القانونية، fintech) مجال غير مألوف
- مهام ذات تبعات غير واضحة (استراتيجيات التخزين المؤقت، أنماط الفشل)
- عدة مقاربات قابلة للتطبيق يعتمد اختيار الأفضل منها على السياق

6.5 التحقق وإعادة المحاولة مع التغذية الراجعة

عندما تفشل البيانات المستخرجة في التحقق:

```
Step 1: Extract data from the document
Step 2: Validate (Pydantic, JSON Schema, business rules)
Step 3: If there's an error—retry with context:
- The original document
- The previous (incorrect) extraction
- The specific error: "Field 'total' = 150, but sum(line_items) = 145. Re-check values."
```

متى تكون إعادة المحاولة فعالة:

- أخطاء الصيغة (تاريخ بصيغة خاطئة)
- أخطاء بنيوية (حقل في مكان خاطئ)
- عدم اتساق حسابي (يستطيع النموذج إعادة الفحص)

متى لا تساعد إعادة المحاولة:

- المعلومة غير موجودة في المستند المصدر
- السياق المطلوب خارجي (البيانات في مستند آخر غير مقدم)

للتحقق من البيانات بناءً على المخططات. لأغراض الاختبار، النقاط Python مكتبة Pydantic: كأداة تحقق Pydantic الأساسية هي:

- Claude من JSON تُفحص في الكود بعد استقبال enum **التحقق البنيوي**: الأنواع، الإلزام، وقيود

- `start_date < end_date`) مجموع العناصر يساوي الإجمالي!) **التحقق الدلالي**: متحققات مخصصة تفرض منطق الأعمال
- مع سياق الخطأ Claude ابن رسالة خطأ وأعد مطالبة ، Pydantic **حلقات تحقق** -إعادة محاولة: عند فشل تحقق
- فتكون مصدر حقيقة واحدًا ، `tool_use` ل JSON Schema توليد Pydantic تستطيع نماذج **JSON Schema توليد**

6.6 التصحيح الذاتي

نمط لاكتشاف التناقضات الداخلية

```
{
  "stated_total": "$150.00",
  "calculated_total": "$145.00",
  "conflict_detected": true,
  "line_items": [
    {"name": "Widget A", "price": 75.00},
    {"name": "Widget B", "price": 70.00}
  ]
}
```

بالتعامل مع `conflict_detected` يستخرج النموذج القيمة المصّرّح بها والقيمة المحسوبة؛ إذا اختلفتا، يسمح بالتعارض.

7 الفصل: Message Batches API

التوثيق: [Message Batches](#)

7.1 نظرة عامة

إرسال دفعات من الطلبات للمعالجة غير المتزامنة Message Batches API تتيح

الخاصية	القيمة
التوفير	مقارنة بالاستدعاءات المتزامنة 50%
نافذة المعالجة	(للكمون SLA لا يوجد ضمان) حتى 24 ساعة
استدعاء أدوات متعدد الأدوار	غير مدعوم (طلب واحد = استجابة واحدة)
الربط	لربط الطلب بالاستجابة <code>custom_id</code> حقل

7.2 Synchronous API مقابل Batch API متى تستخدم

المهمة	API	السبب
--------	-----	-------

المطور ينتظر؛ 24 ساعة غير مقبولة	Synchronous	قبل الدمج PR فحص
%النتيجة مطلوبة صباحًا؛ توفير 50	Batch	تقرير ديون تقنية ليلي
%غير عاجل؛ توفير 50	Batch	تدقيق أمني أسبوعي
استجابة فورية مطلوبة	Synchronous	مراجعة كود تفاعلية
معالجة ضخمة؛ التوفير كبير	Batch	معالجة 10,000 مستند

7.3 استخدام custom_id

```
{
  "custom_id": "doc-invoice-2024-001",
  "params": {
    "model": "claude-sonnet-4-6",
    "max_tokens": 1024,
    "messages": [{"role": "user", "content": "Extract data from: ..."}]
  }
}
```

بـ custom_id يسمح

- ربط النتيجة بالمستند الأصلي
- عند الفشل، إعادة إرسال المستندات الفاشلة فقط
- تجنب إعادة معالجة المستندات الناجحة

7.4 التعامل مع الإخفاقات في الدفعات

1. أرسل دفعة من 100 مستند
2. نجح 95 وفشل 5 (تجاوز حد السياق)
3. حدد الإخفاقات عبر custom_id
4. عدّل الاستراتيجية (مثل تقسيم المستندات الطويلة إلى أجزاء)
5. أعد إرسال المستندات الخمسة الفاشلة فقط

7.5 SLA التخطيط حسب

قد يستغرق حتى 24 ساعة Batch API إذا كنت تحتاج النتيجة خلال 30 ساعة وكان

- نافذة الإرسال: 30 - 24 = 6 ساعات
- يجب إرسال الدفعات قبل الموعد النهائي بـ 24 ساعة على الأقل
- للإرسال المتكرر، قسّمها إلى نوافذ من 4 ساعات

الفصل 8: استراتيجيات تفكيك المهام

8.1 خطوط ثابتة (Prompt Chaining)

تُعرّف كل خطوة مسبقًا:

```
Document -> Metadata extraction -> Data extraction -> Validation -> Enrichment -> Final output
```

متى تستخدمها:

- بنية المهمة متوقعة (المراجعات تتبع القالب نفسه دائمًا)
- كل الخطوات معروفة مسبقًا
- تحتاج إلى الاستقرار وإمكانية إعادة الإنتاج

8.2 التفكيك الديناميكي التكيفي

تُولد المهام الفرعية بناءً على النتائج الوسيطة:

- "Add tests for a legacy codebase"
- > First: map the structure (Glob, Grep)
- > Found: 3 modules with no tests, 2 with partial coverage
- > Prioritize: start with the payments module (high risk)
- > During work: discovered a dependency on an external API
- > Adapt: add a mock for the external API before writing tests

متى تستخدمه:

- المهام الاستقصائية المفتوحة
- عندما لا يكون النطاق الكامل معروفًا مسبقًا
- عندما تعتمد كل خطوة على نتائج الخطوة السابقة

8.3 مراجعة كود متعددة المرور

لطلبات السحب التي تحتوي على أكثر من 10 ملفات:

```
Pass 1 (per-file): Analyze auth.ts -> list local issues
Pass 1 (per-file): Analyze database.ts -> list local issues
Pass 1 (per-file): Analyze routes.ts -> list local issues
...
Pass 2 (integration): Analyze relationships between files
-> Cross-file issues: inconsistent types, circular dependencies
```

لماذا المرور الواحد على 14 ملفًا سيئ:

- تشنت الانتباه: تحليل عميق لبعض الملفات وسطحي لآخرى
- تعليقات غير متسقة: يُرفع نمط في ملف ويُقبل في آخر

- أخطاء فائتة: تُتجاوز أخطاء واضحة بسبب الحمل المعرفي

الفصل 9: التصعيد وإدخال الإنسان في الحلقة

9.1 متى تصعد إلى إنسان

محفزات التصعيد (قواعد واضحة)

الحالة	الإجراء
“get me a manager” يطلب العميل صراحة	صعد فورًا؛ لا تحاول الحل
السياسة لا تغطي الطلب	صعد (مثل مطابقة سعر منافس عندما تكون السياسة صامتة)
لا يستطيع الوكيل إحراز تقدم	صعد بعد عدد معقول من المحاولات
عملية مالية فوق حد معين	صعد (يفضل فرض ذلك بخطاف لا بمطالبة)
وجود عدة مطابقات عند البحث عن عميل	اطلب معرفات إضافية؛ لا تخمن

ما لا يُعد محفزًا موثوقًا:

طريقة غير موثوقة	لماذا تفشل
تحليل المشاعر	مزاج العميل لا يرتبط بتعقيد القضية
تقييم النموذج لثقتة بنفسه (1-10)	قد يكون النموذج واثقًا وهو مخطئ؛ المعايير ضعيفة
مصنف تلقائي	تعقيد زائد؛ قد يحتاج بيانات تدريب غير متاحة

9.2 أنماط التصعيد

تصعيد فوري:

```
Customer: "I want to speak to a manager"
Agent: [immediately calls escalate_to_human]
NOT: "I can help with your issue, let me..."
```

تصعيد بعد محاولة حل:

```
Customer: "My refrigerator broke two days after purchase"
Agent: [checks the order, offers a warranty replacement]
If the customer is not satisfied -> escalate
```

تصعيد دقيق (اعتراف → حل → تصعيد عند التكرار)

Customer: "This is outrageous, I'm very unhappy with the quality!"
Agent: [acknowledges frustration] "I understand your frustration."
[offers resolution] "I can offer a replacement or a refund."
Customer: "No, I want to talk to someone!"
Agent: [customer insists again -> immediate escalation]

المبدأ الأساسي: اعترف بالمشاعر أولاً، ثم اقترح حلاً ملموساً، ولا تصعد إلا إذا كرر العميل رغبته في إنسان. لا تصعد عند أول تعبير عن عدم الرضا، فهذا ليس مثل طلب المدير.

التصعيد بسبب فجوة في السياسة:

Customer: "Competitor X has this item 30% cheaper—give me a discount"
Policy: covers price adjustments only on your own site
Agent: [escalates – policy does not cover competitor price matching]

9.3 بروتوكولات تسليم منظمة

عند التصعيد، ينبغي أن يمرر الوكيل ملخصاً منظماً إلى الإنسان:

```
{
  "customer_id": "CUST-12345",
  "customer_name": "Ivan Petrov",
  "issue_summary": "Refund request for a damaged item",
  "order_id": "ORD-67890",
  "root_cause": "Item arrived damaged; photos attached",
  "actions_taken": [
    "Verified customer via get_customer",
    "Confirmed order via lookup_order",
    "Offered a standard replacement – customer insists on a refund"
  ],
  "refund_amount": "$89.99",
  "recommended_action": "Approve a full refund",
  "escalation_reason": "Customer requested to speak with a manager"
}
```

لا يرى المشغل البشري النص الكامل للمحادثة، بل هذا الملخص فقط. لذلك يجب أن يكون كاملاً ومكتفياً بذاته.

9.4 معايير الثقة والرقابة البشرية

لأنظمة استخراج البيانات:

- درجات ثقة على مستوى الحقول: يخرج النموذج درجة ثقة لكل حقل مستخرج
- المعايرة: استخدم مجموعات تحقق موسومة لضبط العتبات
- التوجيه:
 - ثقة عالية + دقة مستقرة -> معالجة آلية
 - ثقة منخفضة أو مصادر غامضة -> مراجعة بشرية

أخذ عينات عشوائية طبقية:

- حتى للاستخراجات عالية الثقة، دقق عينة بانتظام
- قد تخفي دقة إجمالية 97% أخطاء بنسبة 40% لنوع مستند معين
- حلل الدقة حسب نوع المستند والحقل، لا الإجمالي فقط

الفصل 10: معالجة الأخطاء في الأنظمة متعددة الوكلاء

10.1 فئات الأخطاء

إجراء الوكيل	قابلة للإعادة؟	أمثلة	الفئة
أسّي backoff إعادة المحاولة مع	نعم	فشل شبكة، 503، Timeout	عابر
عدّل الطلب وأعد المحاولة	لا (أصلح الإدخال)	صيغة إدخال غير صالحة، حقل مطلوب ناقص	تحقق
اشرح للمستخدم واقتح بديلاً	لا	انتهاك سياسة، تجاوز حد	أعمال
صعّد	لا	رفض الوصول	إذن

10.2 أنماط مضادة في معالجة الأخطاء

النمط المضاد	المشكلة	النهج الصحيح
أرجع نوع الخطأ، الاستعلام، النتائج الجزئية، البدائل	لا يستطيع المنسق تقرير كيفية التعافي	حالة عامة "search unavailable"
"ميّز بين" لا نتائج" و"فشل البحث	يظن المنسق أنه لا توجد مطابقات بينما كان هناك فشل	الكبت الصامت (نتيجة فارغة = نجاح)
تابع بالنتائج الجزئية وعلّق الفجوات	تخسر كل النتائج الجزئية	إيقاف سير العمل كله بسبب فشل واحد
تعافٍ محلي (1-2 محاولة)، ثم مرّر للمنسق	كمون وموارد مهدرة	إعادة محاولات لا نهائية داخل وكيل فرعي

خطأ وكيل فرعي منظم 10.3

```
{
  "status": "partial_failure",
  "failure_type": "timeout",
  "attempted_query": "AI impact on music industry 2024",
  "partial_results": [
    {"title": "AI Music Generation Report", "url": "...", "relevance": 0.8}
  ],
  "alternative_approaches": [
    "Try a narrower query: 'AI music composition tools'",
    "Use an alternative data source"
  ],
  "coverage_impact": "Not covered: AI impact on music production"
}
```

يوفر هذا للمنسق المعلومات اللازمة لاتخاذ القرار:

- هل يعيد المحاولة باستعلام معدل؟
- هل يستخدم النتائج الجزئية؟
- هل يفوض لوكيل فرعي مختلف؟
- هل يتابع دون هذا القسم مع توضيح الفجوة؟

تعليقات التغطية في التركيب النهائي 10.4

```
## Report: AI Impact on Creative Industries

### Visual Art (FULL COVERAGE)
[research results]

### Music (PARTIAL COVERAGE – search agent timeout)
[partial results]
⚠ Note: coverage for this section is limited due to a timeout in the search agent.

### Literature (FULL COVERAGE)
[research results]
```

الفصل 11: إدارة السياق في أنظمة الإنتاج

استخراج الحقائق في كتلة منفصلة 11.1

بدلاً من الاعتماد على سجل المحادثة (الذي يتدهور أثناء التلخيص)، استخرج الحقائق الأساسية في كتلة منظمة

```
=== CASE FACTS (updated whenever a new fact appears) ===
Customer ID: CUST-12345
Order ID: ORD-67890
Order Date: 2025-01-15
Order Amount: $89.99
Issue: Damaged item on delivery
Customer Request: Full refund
Status: Pending manager approval
===
```

أدرج هذه الكتلة في كل مطالبة بغض النظر عن كيفية تلخيص السجل.

11.2 تقديم نتائج الأدوات

يرجع أكثر من 40 حقلاً ولا تحتاج إلا 5 للمهمة الحالية `lookup_order` إذا كان

```
# PostToolUse hook: keep only relevant fields
@hook("PostToolUse", tool="lookup_order")
def trim_order_fields(result):
    return {
        "order_id": result["order_id"],
        "status": result["status"],
        "total": result["total"],
        "items": result["items"],
        "return_eligible": result["return_eligible"]
    }
```

هذا يحافظ على السياق ويقلل الضجيج.

11.3 إدخال واعٍ بالموقع

ضع المعلومات الحرجة مع مراعاة تأثير الضياع في الوسط:

```
[KEY FINDINGS – at the top]
Found 3 critical vulnerabilities...

[DETAILED RESULTS – middle]
=== File auth.ts ===
...
=== File database.ts ===
...

[ACTION ITEMS – at the end]
Priority: fix auth.ts vulnerabilities before merge.
```

11.4 ملفات Scratchpad

scratchpad: في التحقيقات الطويلة، يستطيع الوكيل كتابة النتائج الأساسية إلى ملف

```
# investigation-scratchpad.md
## Key findings
- PaymentProcessor in src/payments/processor.ts inherits from BaseProcessor
- refund() is called from 3 places: OrderController, AdminPanel, CronJob
- External PaymentGateway API has a rate limit of 100 req/min
- Migration #47 added refund_reason (NOT NULL) – 2024-12-01
```

بدلاً من إعادة الاستكشاف scratchpad عندما يتدهور السياق (أو في جلسة جديدة)، يستطيع الوكيل الرجوع إلى

11.5 التفويض للوكلاء الفرعيين لحماية السياق

```
Main agent: "Investigate dependencies of the payments module"
-> Subagent (Explore): reads 15 files, traces imports
-> Returns: "Payments depends on AuthService, OrderModel, and the external
PaymentGateway API"
```

Main agent: keeps one line in context instead of 15 files

طبقة سياق منفصلة: في الأنظمة متعددة الوكلاء، يعمل كل وكيل فرعي ضمن ميزانية سياق محدودة، ولا يستقبل إلا المعلومات المطلوبة لمهمته. يعمل المنسق كطبقة سياق منفصلة: يجمع مخرجات الوكلاء الفرعيين، ويخزن الحالة العامة، ويوزع السياق. يمنع هذا "تسرب السياق"، حيث يستهلك وكيل واحد النافذة بمعلومات غير ذات صلة للآخرين.

ميزانيات سياق مقيدة للوكلاء الفرعيين:

- أرسل الحد الأدنى من السياق: مهمة محددة + البيانات اللازمة
- اطلب من الوكيل الفرعي إرجاع نتائج منظمة لا تفريغات بيانات خام
- لتقييد مجموعة أدوات الوكيل الفرعي؛ أدوات أقل تعني مشتتات أقل وتكلفة سياق أقل `allowedTools` استخدم

11.6 استمرار الحالة المنظمة (للتعافي من الانهيار)

يصدّر كل وكيل حالته إلى موقع معروف:

```
// agent-state/web-search-agent.json
{
  "status": "completed",
  "queries_executed": ["AI music 2024", "AI music composition"],
  "results_count": 12,
  "key_findings": [...],
  "coverage": ["music composition", "music production"],
  "gaps": ["music distribution", "music licensing"]
}
```

عند الاستئناف manifest يحمل المنسق:

```
// agent-state/manifest.json
{
  "web-search": "completed",
  "doc-analysis": "in_progress",
  "synthesis": "not_started"
}
```

الفصل 12: الحفاظ على المصدرية

12.1 مشكلة فقدان الإسناد

”عند تلخيص نتائج من مصادر متعددة، قد تضع صلة “الادعاء ← المصدر

Bad: "The AI music market is estimated at \$3.2B." (No source, no year.)

Good:

```
{
  "claim": "The AI music market is estimated at $3.2B.",
  "source_url": "https://example.com/report",
  "source_name": "Global AI Music Report 2024",
  "publication_date": "2024-06-15",
  "confidence": 0.9
}
```

12.2 التعامل مع البيانات المتعارضة

عندما يقدم مصدران قيمًا مختلفة


```
{
  "claim": "Share of AI-generated music on streaming platforms",
  "values": [
    {
      "value": "12%",
      "source": "Spotify Annual Report 2024",
      "date": "2024-03",
      "methodology": "Automated classification"
    },
    {
      "value": "8%",
      "source": "Music Industry Association Survey",
      "date": "2024-07",
      "methodology": "Survey of 500 labels"
    }
  ],
  "conflict_detected": true,
  "possible_explanation": "Difference in methodology and time period"
}
```

لا تختار قيمة عشوائيًا. احتفظ بالقيمتين مع الإسناد ودع المنسق يقرر.

12.3 تضمين التواريخ للتفسير الصحيح

بدون التواريخ، قد تُفسر الفروق الزمنية كتعارضات:

Bad: "Source A says 10%, source B says 15%. Contradiction."

Good: "Source A (2023) says 10%, source B (2024) says 15%. Likely +5% growth over a year."

12.4 العرض حسب نوع المحتوى

لا تجبر كل شيء على صيغة واحدة:

- البيانات المالية -> جداول
- الأخبار والتحليل -> نشر
- النتائج التقنية -> قوائم منظمة
- السلاسل الزمنية -> ترتيب زمني

الدمجة Claude Code الفصل 13: أدوات

13.1 مرجع اختيار الأدوات

المهمة	الأداة	مثال
--------	--------	------

العثور على الملفات بالاسم/النمط	Glob	<code>**/*.test.tsx</code> , <code>src/components/**/*.ts</code>
البحث داخل الملفات	Grep	اسم دالة، رسالة خطأ، import
قراءة ملف كامل	Read	تحميل ملف للتحليل
كتابة ملف جديد	Write	إنشاء ملف من الصفر
تحرير ملف موجود بدقة	Edit	استبدال مقطع محدد عبر تطابق نص فريد
shell تشغيل أمر	Bash	git, npm، تشغيل الاختبارات، build

استراتيجية التحقيق التدريجي 13.2

لا تقرأ كل الملفات دفعة واحدة. ابنِ الفهم تدريجيًا

1. Grep: find entry points (function definition, export)
 2. Read: read the found files
 3. Grep: find usages (import, calls)
 4. Read: read consumer files
 5. Repeat until you have a complete picture

Edit بدلاً من Read + Write الرجوع إلى 13.3

بسبب تطابق نص غير فريد Edit عندما يفشل:

1. Read — حمّل محتوى الملف كاملاً
2. عدّل المحتوى برمجياً
3. Write — اكتب النسخة المحدثة

الجزء الثاني: ملاحظات مجالات الاختبار

المجال 1: بنية الوكلاء والتنسيق (27%)

تصميم حلقات وكيلة لتنفيذ المهام ذاتيًا 1.1

المعرفة الأساسية:

- نفذ ("end_turn" مقابل "tool_use") (stop_reason افحص Claude، دورة حياة حلقة الوكيل: أرسل طلب الأدوات، أعد النتائج للتكرار التالي
- تُضاف نتائج الأدوات إلى سجل المحادثة حتى يستطيع النموذج تقرير الإجراء التالي
- مقابل أشجار القرار المبرمجة مسبقًا (يختار الأداة التالية Claude) اتخاذ القرار بقيادة النموذج

المهارات الأساسية:

- `stop_reason = "tool_use"` وتوقف عند `"end_turn"`
- إضافة نتائج الأدوات إلى السياق بين التكرارات
- تجنب الأنماط المضادة: تحليل نص المساعد لاكتشاف الاكتمال، واستخدام حد تكرار اعتباطي كآلية توقف أساسية

تنسيق الأنظمة متعددة الوكلاء (منسق-وكيل فرعي) 1.2

المعرفة الأساسية:

- بنية المحور والأذرع: يملك المنسق كل اتصالات الوكلاء، ومعالجة الأخطاء، والتوجيه
- تعمل الوكلاء الفرعيون بسياق معزول؛ لا يرثون سجل المنسق تلقائيًا
- مسؤوليات المنسق: تفكيك المهمة، التفويض، تجميع النتائج، الاختيار الديناميكي للوكلاء الفرعيين
- خطر التفكيك الضيق جدًا من المنسق

المهارات الأساسية:

- تقسيم تغطية البحث بين الوكلاء الفرعيين لتقليل التكرار
- تنفيذ حلقات تحسين تكرارية (يقيم المنسق التركيب ويعيد توجيه المهام)
- توجيه كل الاتصالات عبر المنسق للملاحظة

إعداد استدعاءات الوكلاء الفرعيين، وتمرير السياق، والإنشاء 1.3

المعرفة الأساسية:

- `"Task"` للمنسق `allowedTools` تنشئ الوكلاء الفرعيين؛ يجب أن تتضمن `Task` أداة
- يجب إدراج سياق الوكيل الفرعي صراحة في المطالبة؛ لا يرث الوكلاء الفرعيون سياق الأب
- الأوصاف، مطالبات النظام، قيود الأدوات: `AgentDefinition` إعداد
- لاستكشاف البدائل `fork_session` إدارة الجلسات عبر

المهارات الأساسية:

- تضمين المخرجات الكاملة من الوكلاء السابقين في مطالبة الوكيل الفرعي
- استخدام صيغ منظمة لفصل البيانات عن البيانات الوصفية عند تمرير السياق
- في دور منسق واحد `Task` إنشاء وكلاء فرعيين متوازيين عبر عدة استدعاءات
- كتابة مطالبات المنسق من حيث الأهداف ومعايير الجودة لا التعليمات خطوة بخطوة

متعددة الخطوات مع أنماط الإنفاذ والتسليم workflows تنفيذ 1.4

المعرفة الأساسية:

- الفرق بين الإنفاذ البرمجي (خطافات، شروط مسبقة) وإرشاد المطالبة لترتيب سير العمل
- عندما تحتاج إلى ضمانات حتمية (مثل التحقق من الهوية قبل العمليات المالية)، لا تكفي المطالبات وحدها
- بروتوكولات تسليم منظمة أثناء التصعيد (معرف العميل، السبب، الإجراء الموصى به)

المهارات الأساسية:

- مثل منع) شروط مسبقة برمجية تمنع الاستدعاءات اللاحقة حتى تكتمل الخطوات السابقة (`process_refund` مثل منع)
(معرفةً محققًا `get_customer` حتى يرجع
- تفكيك طلبات العملاء متعددة الجوانب إلى عناصر منفصلة
- إنتاج ملخصات منظمة عند التصعيد إلى إنسان

لاعتراض استدعاءات الأدوات وتطبيع البيانات Agent SDK خطافات 1.5

المعرفة الأساسية:

- لاعتراض نتائج الأدوات قبل استهلاك النموذج لها (`PostToolUse` مثل) أنماط الخطافات
- (فوق حد معين `refunds` مثل منع) خطافات تعترض الاستدعاءات الصادرة لفرض قواعد الامتثال
- تقدم الخطافات ضمانات حتمية مقابل تعليمات المطالبات التي تقدم امتثالاً احتماليًا

المهارات الأساسية:

- (رموز حالة رقمية، ISO 8601، Unix طوايع) لتطبيع صيغ البيانات `PostToolUse` خطافات
- خطافات اعتراض لمنع الإجراءات المخالفة للسياسة مع إعادة التوجيه إلى التصعيد
- اختيار الخطافات بدل المطالبات عندما تتطلب قواعد الأعمال امتثالاً مضمونًا

استراتيجيات تفكيك المهام لسير العمل المعقد 1.6

المعرفة الأساسية:

- مقابل تفكيك ديناميكي تكيفي بناءً على النتائج الوسيطة (prompt chaining) خطوط ثابتة
- خطوات متتابعة (حلل كل ملف منفصلاً، ثم نفذ تمرير تكامل): Prompt chaining
- خطط تحقيق تكيفية تولد مهام فرعية بناءً على الاكتشافات

المهارات الأساسية:

- للمراجعات متعددة الجوانب المتوقعة؛ واستخدم التفكيك الديناميكي للتحقيقات prompt chaining استخدم المفتوحة
- قسّم مراجعات الكود الكبيرة إلى تحليل لكل ملف ثم تمرير تكامل منفصل بين الملفات
- فكك المهام المفتوحة: ارسم البنية أولاً، ثم ابن خطة ذات أولوية

حالة الجلسة، والاستئناف، والتفرع 1.7

المعرفة الأساسية:

- لمتابعة الجلسات المسماة `--resume <session-name>`
- لإنشاء فروع تحقيق مستقلة من سياق مشترك `fork_session`
- أهمية إبلاغ الوكيل بتغييرات الملفات عند استئناف الجلسات
- قد تكون الجلسة الجديدة مع ملخص منظم أكثر موثوقية من الاستئناف بنتائج قديمة

المهارات الأساسية:

- لمتابعة جلسات التحقيق المسماة `--resume` استخدم
- لمقارنة المقاربات بالتوازي `fork_session` استخدم
- اختر بين الاستئناف (السياق ما يزال حديثًا) والبدء من جديد (النتائج قديمة)

MCP (18%) المجال 2: تصميم الأدوات وتكامل

تصميم واجهات الأدوات بأوصاف واضحة 2.1

المعرفة الأساسية:

- لاختيار الأدوات؛ الأوصاف المختصرة تؤدي إلى اختيار غير LLM أوصاف الأدوات هي الآلية الأساسية التي يستخدمها موثوق
- أهمية تضمين صيغ الإدخال، أمثلة الاستعلامات، الحالات الحدية، وحدود التطبيق
- الأوصاف الغامضة أو المتداخلة تسبب التوجيه الخاطئ
- صياغة مطالبة النظام قد تنشئ ارتباطات غير مقصودة بالأدوات

المهارات الأساسية:

- كتابة أوصاف تميز كل أداة بوضوح عن البدائل المشابهة
- إعادة تسمية الأدوات لإزالة التداخل الوظيفي (مثل `analyze_content` -> `extract_web_results`)
- تقسيم الأدوات العامة إلى أدوات متخصصة بعقود إدخال/إخراج واضحة

MCP تنفيذ استجابات أخطاء منظمة لأدوات 2.2

المعرفة الأساسية:

- MCP في استجابات أدوات `isError` راية
- أخطاء التحقق (إدخال سيئ)، أخطاء الأعمال (انتهاكات سياسة)، وأخطاء (timeouts) الفرق بين الأخطاء العابرة الوصول/الأدوات
- تمنع قرارات التعافي الصحيحة ("Operation failed") الأخطاء العامة
- الفرق بين الأخطاء القابلة وغير القابلة لإعادة المحاولة

المهارات الأساسية:

- `isRetryable` و `errorCategory` (transient/validation/permission) إرجاع بيانات وصفية منظمة مثل رسالة مفهومة للإنسان
- لانتهاكات قواعد الأعمال مع شروحات واضحة للمستخدم `retryable: false` استخدام
- تنفيذ التعافي المحلي داخل الوكلاء الفرعيين للأخطاء العابرة؛ وتميرير فقط ما لا يستطيعون حله
- التمييز بين فشل الوصول والنتائج الفارغة الصالحة

2.3 tool_choice توزيع الأدوات بين الوكلاء وإعداد

المعرفة الأساسية:

- الأدوات الكثيرة جدًا لكل وكيل (مثل 18 بدل 4-5) تقلل موثوقية اختيار الأداة
- الوكلاء الذين يملكون أدوات خارج تخصصهم يميلون إلى إساءة استخدامها
- نطاق وصول الأدوات: الأدوات ذات الصلة بالدور فقط، مع مجموعة محدودة من أدوات عابرة للأدوار
- ({"type": "tool", "name": "..."}) والاختيار الإجباري "any" و "auto" : tool_choice

المهارات الأساسية:

- تقييد أدوات كل وكيل فرعي بما يناسب دوره
- (fetch_url -> load_document) مثل استبدال الأدوات العامة ببدايل مقيدة
- لضمان استدعاء أداة بدل إجابة نصية tool_choice: "any" استخدام
- إجبار أداة محددة لضمان ترتيب التنفيذ

2.4 وسير عمل الوكلاء في Claude Code MCP دمج خوادم

المعرفة الأساسية:

- للتجارب (~/.claude.json) للفرق مقابل المستخدم (.mcp.json) المشروع: MCP نطاق خادم
- لإدارة الأسرار (\${GITHUB_TOKEN}) مثل (.mcp.json) استبدال متغيرات البيئة في
- المتصلة عند الاتصال وتتاح في الوقت نفسه MCP تُكتشف أدوات كل خوادم
- كـ "فهارس محتوى" (ملخصات مهام، مخططات قواعد بيانات) لتقليل استدعاءات الأدوات الاستكشافية MCP موارد

المهارات الأساسية:

- للمشروع مع رموز عبر متغيرات البيئة (.mcp.json) مشتركة في MCP إعداد خوادم
- (~/.claude.json) إبقاء الخوادم الشخصية/التجريبية في
- المجتمعية على الخوادم المخصصة للتكاملات القياسية MCP تفضيل خوادم

2.5 اختيار وتطبيق الأدوات المدمجة (Read, Write, Edit, Bash, Grep, Glob)

المعرفة الأساسية:

- (imports, أسماء الدوال، رسائل الخطأ) البحث داخل محتوى الملفات: Grep
- العثور على الملفات حسب أنماط الاسم/الامتداد: Glob
- تغييرات دقيقة عبر تطابق نص فريد: Edit عمليات على الملف كاملاً: Read/Write
- Read + Write بسبب تطابقات غير فريدة، ارجع إلى Edit إذا فشل

المهارات الأساسية:

- لاكتشاف الملفات بأنماط Glob للبحث في المحتوى و Grep استخدم
- لتتبع التدفقات Read لنقاط الدخول، ثم Grep: ابن الفهم تدريجيًا
- wrapper تتبع استخدام الدالة عبر وحدات

وسير العمل (20%) Claude Code المجال 3: إعداد

3.1 بالهرمية والنطاق والتنظيم المعياري CLAUDE.md إعداد

المعرفة الأساسية:

- أو `.claude/CLAUDE.md` المشروع، `~/claude/CLAUDE.md` المستخدم (CLAUDE.md هرمية (في المجلدات الفرعية CLAUDE.md) ومستوى المجلد، (في الجذر CLAUDE.md)
- VCS إعدادات مستوى المستخدم تنطبق على مستخدم واحد فقط ولا تُشارك عبر
- CLAUDE.md لجعل `@./standards/coding-style.md` (مثل) للإشارة إلى ملفات خارجية `@path` صيغة معيارًا
- ضخم CLAUDE.md لملفات قواعد مركزة حسب الموضوع بدل `.claude/rules/` مجلد

المهارات الأساسية:

- تشخيص مشكلات الهرمية (عضو فريق جديد لا يرى التعليمات لأنها على مستوى المستخدم بدل المشروع)
- لكل حزمة CLAUDE.md لتضمن المعايير انتقائيًا في `@./standards/testing.md` (مثل) `@path` استخدام
- متعددة `.claude/rules/` كبير إلى ملفات CLAUDE.md تقسيم (testing.md, api-conventions.md, deployment.md)

3.2 والمهارات المخصصة Slash إنشاء وإعداد أوامر

المعرفة الأساسية:

- مقابل أوامر المستخدم في (VCS تُشارك عبر) `.claude/commands/` أوامر المشروع في `~/claude/commands/`
- `SKILL.md` في frontmatter مع `.claude/skills/` المهارات في `context: fork`, `allowed-tools`, `argument-hint`
- يشغل المهارة في سياق وكيل فرعي معزول حتى لا يلوث الجلسة الرئيسية `context: fork`
- بأسماء مختلفة `~/claude/skills/` يمكن وضع نسخ مهارات شخصية في

المهارات الأساسية:

- حتى يحصل عليها الفريق كله `.claude/commands/` الخاصة بالمشروع في slash تخزين أوامر
- لعزل المهارات ذات الإخراج المطول `context: fork` استخدام
- لتقييد الأدوات التي تستطيع المهارة استخدامها `allowed-tools` استخدام
- لمطالبة المطورين بالمعاملات المطلوبة `argument-hint` استخدام

3.3 استخدام قواعد خاصة بالمسارات للتحميل الشرطي للاصطلاحات

المعرفة الأساسية:

- glob لتفعيل القواعد حسب أنماط `paths` مع frontmatter YAML تضمين `.claude/rules/` تستطيع ملفات

- لا تُحمّل قواعد المسارات إلا عند تحرير ملفات مطابقة، مما يوفر السياق والرموز
- على مستوى المجلد عندما تنطبق الاصطلاحات عبر CLAUDE.md أفضل من glob قد تكون القواعد المبنية على عدة مجلدات (مثل الاختبارات)

المهارات الأساسية:

- للتحميل فقط عند العمل على ملفات `paths: ["terraform/**/*"]` مع `.claude/rules/` إنشاء ملفات مطابقة
- لتطبيق الاصطلاحات حسب نوع الملف بغض النظر عن الموقع (`**/*.test.tsx`) استخدام أنماط glob
- مستوى المجلد عندما تمتد الاصطلاحات عبر قاعدة الكود CLAUDE.md تفصيل القواعد الخاصة بالمسار على

3.4 تحديد متى تستخدم وضع التخطيط مقابل التنفيذ المباشر

المعرفة الأساسية:

- وضع التخطيط: للمهام المعقدة ذات التغييرات الكبيرة والمقاربات المتعددة والقرارات المعمارية
- التنفيذ المباشر: للتغييرات البسيطة المفهومة جيدًا (مثل إضافة تحقق واحد)
- يتيح وضع التخطيط استكشافًا آمنًا لقاعدة الكود قبل إجراء تغييرات
- إخراج الاكتشاف المطول Explore subagent يعزل

المهارات الأساسية:

- (تمس +45 ملقًا microservices, migrations) استخدم وضع التخطيط للمهام ذات التبعات المعمارية
- واضح وملف واحد stack trace استخدم التنفيذ المباشر لإصلاحات ذات
- لمنع استنزاف نافذة السياق في المهام متعددة المراحل Explore subagent استخدم
- اجمع بين النهجين: خطط للاكتشاف، ثم نفذ للتطبيق

3.5 التحسين التكراري للتطوير التدريجي

المعرفة الأساسية:

- أمثلة الإدخال/الإخراج الملموسة هي أكثر الطرق فعالية لتوصيل التوقعات
- التكرار الموجه بالاختبارات: اكتب الاختبارات أولاً، ثم كرر بناءً على الإخفاقات
- أسئلة لكشف اعتبارات تصميم غير واضحة Claude نمط "المقابلة": يطرح
- متى تقدم كل المشكلات في رسالة واحدة (متراصة) مقابل متابعة (مستقلة)

المهارات الأساسية:

- تقديم 2-3 أمثلة إدخال/إخراج ملموسة لتوضيح متطلبات التحويل
- بناء مجموعات اختبار بالسلوك المتوقع، الحالات الحدية، ومتطلبات الأداء قبل التنفيذ
- (أنماط الفشل، cache إبطال) استخدام نمط المقابلة لكشف جوانب التصميم
- تقديم حالات اختبار ملموسة مع مدخلات عينة ومخرجات متوقعة للحالات الحدية

3.6 CI/CD في خطوط Claude Code دمج

:المعرفة الأساسية:

- للوضع غير التفاعلي في خطوط الأتمتة (`--print` أو `-p`) العلم
- CI للمخرجات المنظمة في `--json-schema` و `--output-format json`
- CI عند تشغيله من Claude Code سياق المشروع (معايير الاختبار، معايير المراجعة) لـ CLAUDE.md يوفر
- عزل سياق الجلسة: الجلسة نفسها التي ولدت الكود أقل فعالية في مراجعته من مثيل مستقل

:المهارات الأساسية:

- لتجنب التعليق بانتظار إدخال تفاعلي `-p` باستخدام CI في Claude Code تشغيل
- (داخلية PR مثل تعليقات) للنتائج المنظمة `--json-schema` + `--output-format json` استخدام
- جديدة (الإبلاغ فقط عن الجديد/غير المُصلح) commits تضمين نتائج المراجعة السابقة عند إعادة التشغيل بعد
- لتحسين جودة توليد الاختبارات CLAUDE.md المتاحة في fixturesتوثيق معايير الاختبار وال
- تضمين ملفات الاختبار الحالية في السياق عند توليد اختبارات جديدة لتجنب التكرار والحفاظ على الأسلوب

المجال 4: هندسة المطالبات والمخرجات المنظمة (20%)

تصميم مطالبات بمعايير صريحة لتحسين الدقة 4.1

:المعرفة الأساسية:

- المعايير الصريحة أكثر فعالية من التعليمات المبهمة (مثل "ارفع التعليقات فقط عندما تناقض الكود" بدل "افحص دقة التعليقات")
- الإرشاد العام مثل "كن أكثر تحفظاً" يعمل أسوأ من معايير فئوية ملموسة
- أثر الإيجابيات الكاذبة على ثقة المطور: ارتفاع الإيجابيات الكاذبة في فئات معينة يقوض الثقة حتى في الفئات الدقيقة

:المهارات الأساسية:

- تعريف معايير المراجعة: ما يجب الإبلاغ عنه (أخطاء، أمن) وما يجب تجاهله (أسلوب بسيط)
- تعطيل الفئات ذات الإيجابيات الكاذبة العالية مؤقتاً
- تعريف معايير شدة صريحة مع أمثلة كود لكل مستوى

لتحسين اتساق المخرجات Few-shot استخدام 4.2

:المعرفة الأساسية:

- هي أكثر الطرق فعالية لإنتاج إخراج منسق وقابل للتنفيذ باستمرار few-shot أمثلة
- توضيح التعامل مع الحالات الغامضة (اختيار الأداة، فجوات تغطية الاختبارات) few-shot تستطيع
- النموذج على التعميم إلى أنماط جديدة بدل تكرار الافتراضات few-shot تساعد
- الهلوسة في مهام الاستخراج few-shot يمكن أن تقلل

المهارات الأساسية:

- تقديم 2-4 أمثلة موجهة للسيناريوهات الغامضة مع مبررات
- توضيح صيغة الإخراج (الموقع، المشكلة، الشدة، الإصحاح المقترح) few-shot تضمين أمثلة
- تقديم أمثلة تميز أنماط الكود المقبولة عن المشكلات الحقيقية
- تقديم أمثلة للاستخراج الصحيح من مستندات ذات بنى مختلفة

JSON ومخططات tool_use فرض المخرجات المنظمة باستخدام 4.3

المعرفة الأساسية:

- النحوية JSON هو أوثق طريقة لضمان إخراج مطابق للمخطط وإزالة أخطاء JSON Schemas مع tool_use
- يجب أن يستدعي أداة؛ والاختيار الإجباري "any" يستطيع النموذج إرجاع نص؛ ومع tool_choice: "auto" مع يختار أداة محددة
- الصارمة أخطاء الصيغة لكنها لا تمنع الأخطاء الدلالية (الإجماليات لا تتطابق، قيم غير JSON تنزيل مخططات صحيحة)
- تمنع الاختلاق عندما تكون البيانات مفقودة nullable/الحقول الاختيارية

المهارات الأساسية:

- فقط عندما تكون المعلومات موجودة دائماً required تصميم مخططات مع
- لتجنب التصنيف الخاطئ أو فقدان البيانات other / unclear مع enum استخدام
- لضمان إخراج منظم tool_choice: "any" استخدام
- إضافة تحقق دلالي بعد الإخراج عند الحاجة

تنفيذ التحقق وإعادة المحاولة وحلقات التغذية الراجعة لجودة الاستخراج 4.4

المعرفة الأساسية:

- التحقق البنيوي (الأنواع والحقول) لا يكفي للتحقق الدلالي (الإجماليات، العلاقات الزمنية)
- أو قواعد الأعمال التحقق من الإخراج JSON Schema أو Pydantic يمكن لـ
- إعادة المحاولة مع رسالة خطأ محددة أكثر فعالية من إعادة المحاولة العامة
- لا تساعد إعادة المحاولة إذا كانت المعلومة غير موجودة في المصدر

المهارات الأساسية:

- extract -> validate -> retry with feedback تنفيذ حلقة
- تضمين الاستخراج السابق والخطأ المحدد في إعادة المحاولة
- (sum(line_items) == total, start < end) بناء فحوصات حسابية ومنطقية
- بدل إجبار النموذج على التخمين null/unclear استخدام

تصميم استراتيجيات معالجة دفعات فعالة 4.5

المعرفة الأساسية:

- توفر 50% لكنها قد تستغرق حتى 24 ساعة Message Batches API

- لا يدعم استدعاء الأدوات متعدد الأدوار Batch API
- يربط الطلبات بالنتائج ويسهل إعادة إرسال الفاشل فقط `custom_id`
- مناسب للمهام غير العاجلة ذات الحجم الكبير

المهارات الأساسية:

- للمعالجة الليلية/الأسبوعية أو المستندات الضخمة غير العاجلة Batch اختيار
- للفحوصات قبل الدمج أو الاستخدام التفاعلي synchronous اختيار
- لتتبع الإخفاقات وإعادة معالجة الفاشل فقط `custom_id` استخدام
- SLA التخطيط لنوافذ الإرسال حسب

تصميم بنى مراجعة متعددة المثلثات ومتعددة المرور 4.6

المعرفة الأساسية:

- مراجعة الكود في الجلسة نفسها التي ولدت الكود أقل موثوقية
- المرور الواحد على ملفات كثيرة يعاني من تشتت الانتباه
- تحليل كل ملف ثم تمرير تكامل بين الملفات يزيد جودة المراجعة
- يمنع التعليقات المكررة prior review context

المهارات الأساسية:

- استخدام مثيل مستقل لمراجعة الكود المولد
- تكاملي pass لكل ملف و pass كبير إلى PR تقسيم مراجعة
- تضمين نتائج المراجعة السابقة عند إعادة المراجعة
- وتحدد الفئات بوضوح false positives تصميم مطالبات تقلل

المجال 5: إدارة السياق والموثوقية (15%)

إدارة سياق المحادثة للحفاظ على المعلومات الحرجة 5.1

المعرفة الأساسية:

- مستقل ويتطلب إرسال التاريخ الكامل API كل طلب
- التلخيص قد يفقد أرقامًا وتواريخ وتفاصيل دقيقة
- يجعل المعلومات في منتصف السياق أقل موثوقية lost-in-the-middle تأثير
- كتل الحقائق المنظمة تساعد على الحفاظ على الحالة الحرجة

المهارات الأساسية:

- إلى كتلة حالة منفصلة وإدراجها دائمًا في المطالبة facts استخراج
- وضع المعلومات الحرجة في بداية أو نهاية السياق
- تقليل نتائج الأدوات للاحتفاظ بالحقول ذات الصلة فقط
- للنتائج المهمة في التحقيقات الطويلة scratchpad استخدام

تصميم أنماط تصعيد فعالة وحل الغموض 5.2

:المعرفة الأساسية

- طلب الإنسان الصريح يؤدي إلى تصعيد فوري
- تعبير العميل عن الإحباط ليس سببًا كافيًا للتصعيد بحد ذاته
- عندما توجد عدة مطابقات، يجب طلب معرفات إضافية بدل التخمين
- التصعيد يجب أن يمرر ملخصًا منظمًا مكثفًا بذاته

:المهارات الأساسية

- التمييز بين طلب المدير الصريح والتذمر العام
- الاعتراف بالمشاعر ثم تقديم حل ملموس قبل التصعيد عند الحاجة
- التصعيد عند فجوات السياسة أو العمليات المالية فوق الحد
- كامل handoff structured summary إنشاء

تنفيذ استراتيجيات انتشار الأخطاء في الأنظمة متعددة الوكلاء 5.3

:المعرفة الأساسية

- يجب أن تتضمن الأخطاء معلومات مثل النوع، القابلية لإعادة المحاولة، الاستعلام، والنتائج الجزئية
- الفشل الجزئي لا يجب أن يوقف سير العمل كله
- "يجب التمييز بين "لا نتائج" و"فشل"
- ينبغي أن يتعافى الوكيل الفرعي محليًا من الأخطاء العابرة ثم يمرر ما يفشل

:المهارات الأساسية

- تصميم استجابات أخطاء منظمة للوكلاء الفرعيين
- الاستمرار بالنتائج الجزئية مع تعليقات تغطية
- للأخطاء العابرة وعدد محدود من المحاولات backoff استخدام
- للمنسق alternative approaches تمرير

إدارة السياق بكفاءة عند التحقيق في قواعد كود كبيرة 5.4

:المعرفة الأساسية

- قراءة كل الملفات دفعة واحدة تهدر السياق
- Read تساعد على بناء خريطة أولية قبل Grep/Glob
- Explore subagent يعزل الإخراج المطول
- CLAUDE.md والصلة ذات التعليمات غير ذات الصلة

:المهارات الأساسية

- لاكتشاف الملفات Glob لنقاط الدخول و Grep استخدام
- قراءة الملفات تدريجيًا حسب الحاجة
- تفويض الاستكشاف الواسع لوكيل فرعي يعيد ملخصًا

- لتقليل السياق المحمل path-scoped rules استخدام

برقابة بشرية ومعايرة ثقة workflows تصميم 5.5

:المعرفة الأساسية

- الثقة الذاتية للنموذج ليست موثوقة وحدها
- يجب معايرة العتبات باستخدام بيانات موسومة
- يمكن أن تخفي الدقة الإجمالية أخطاء كبيرة في فئات فرعية
- العشوائي الطبعي يكشف مشكلات التغطية sampling

:المهارات الأساسية

- مع عتبات مراجعة بشرية field-level confidence استخدام
- إجراء تدقيق عشوائي حتى على النتائج عالية الثقة
- تحليل الدقة حسب نوع المستند والحقل
- توجيه الحالات الغامضة أو منخفضة الثقة إلى الإنسان

الحفاظ على المصدرية والتعامل مع عدم اليقين في التركيب متعدد المصادر 5.6

:المعرفة الأساسية

- يجب الحفاظ على رابط الادعاء بالمصدر أثناء التلخيص
- البيانات المتعارضة يجب أن تُعرض مع المصدر والمنهجية والتاريخ
- التواريخ تساعد على تمييز النمو من التعارض
- تختلف صيغة العرض المناسبة حسب نوع المحتوى

:المهارات الأساسية

- confidence و date و source_name و source_url مع claim تخزين كل
- عرض القيم المتعارضة بدل اختيار واحدة بلا أساس
- شرح الاختلافات المحتملة في المنهجية أو الفترة الزمنية
- استخدام الجداول للبيانات، والنشر للتحليل، والقوائم للنتائج التقنية

أمثلة على أسئلة الاختبار مع الشرح

السؤال 1 (السيناريو: وكيل خدمة العملاء)

الموقف: يقول عميل: "أريد التحدث إلى المدير الآن." ما الإجراء الصحيح؟

ما الإجراء الصحيح؟

- محاولة حل المشكلة أولاً A)

- إجراء تحليل مشاعر B)
- فورًا. [صحيح] escalate_to_human استدعاء C)
- طلب تقييم رضا العميل D)

طلب الإنسان صراحة هو محفز تصعيد فوري. لا ينبغي للوكيل محاولة الحل الذاتي عندما يطلب العميل مديراً C: لماذا أو إنساناً بشكل واضح.

السؤال 2 (السيناريو: وكيل خدمة العملاء)

قبل التحقق من هوية العميل. ما أفضل حل؟ process_refund الموقف: تريد ضمان عدم تنفيذ

ما أفضل حل؟

- system prompt أضف تعليمة في A)
- أو شرطاً برمجياً يمنع الاستدعاء. [صحيح] hook استخدم B)
- اطلب من النموذج تقييم ثقته C).
- زد max_tokens D).

العمليات المالية تحتاج ضماناً حتمياً. المطالبات احتمالية، أما الخطافات والشروط البرمجية فتمنع الاستدعاء B: لماذا فعلياً.

السؤال 3 (السيناريو: وكيل خدمة العملاء)

ما المشكلة؟ "Operation failed" مع النص isError: true ترجع MCP الموقف: أداة

ما المشكلة؟

- A) JSON الخطأ ليس.
- B) لا يعطي الوكيل معلومات للتعافي. [صحيح]
- C) false هو isError يجب أن يكون.
- D) يجب حذف الأداة.

الخطأ العام لا يوضح هل هو عابر، أو متعلق بالتحقق، أو بالإذن، أو بسياسة الأعمال. لذلك لا يستطيع الوكيل B: لماذا اختيار إعادة المحاولة أو التصعيد أو تعديل الطلب.

السؤال 4 (Claude Code السيناريو: توليد الكود باستخدام)

الموقف: تريد تغييراً معمارياً يمس عشرات الملفات. ما النهج الأنسب؟

ما النهج الأنسب؟

- A) تنفيذ مباشر فوراً.
- B) وضع التخطيط أولاً ثم التنفيذ بعد الموافقة. [صحيح]
- C) فقط /compact استخدام.
- D) حذف CLAUDE.md.

التغييرات الكبيرة أو المعمارية تحتاج استكشافًا آمنًا وخطة معتمدة قبل التنفيذ: B لماذا

5 Claude Code السيناريو: توليد الكود باستخدام السؤال

الموقف: أين تضع معايير المشروع التي يجب أن يستفيد منها كل الفريق؟

أين تضعها؟

- A) `~/claude/CLAUDE.md`
- B) `.claude/CLAUDE.md` أو `CLAUDE.md` [صحيح] في جذر المشروع.
- C) في متغير بيئة محلي فقط.
- D) VCS في ملف لا يدخل.

تعليمات المشروع يجب أن تكون ضمن المستودع حتى تصل لكل من يستنسخ المشروع: B لماذا

6 Claude Code السيناريو: توليد الكود باستخدام السؤال

المنتشرة في مجلدات كثيرة. ما أفضل مكان لها؟ `*.test.ts` الموقف: قاعدة اختبار تنطبق على كل ملفات

ما أفضل مكان؟

- A) داخل مجلد واحد `CLAUDE.md`.
- B) `.claude/rules/` مع `paths: ["**/*.test.ts"]` [صحيح]
- C) مطالبة يدوية في كل مرة.
- D) `~/claude.json`

قواعد المسارات مناسبة عندما تنطبق الاصطلاحات على نوع ملف منتشر عبر قاعدة الكود: B لماذا

السؤال 7 (السيناريو: نظام بحث متعدد الوكلاء)

الموقف: وكيل فرعي لا يرى نتائج البحث السابقة التي جمعها المنسق. ما السبب المرجح؟

ما السبب المرجح؟

- A) الوكلاء الفرعيون لا يرثون سياق المنسق تلقائيًا. [صحيح]
- B) يجب زيادة درجة الحرارة.
- C) Batch API يجب استخدام.
- D) `Task` يجب حذف.

`Task` سياق الوكيل الفرعي معزول. يجب تمرير كل السياق المطلوب صراحة في مطالبة: A لماذا

السؤال 8 (السيناريو: نظام بحث متعدد الوكلاء)

الموقف: تريد أن يعمل ثلاثة وكلاء فرعيين بالتوازي. كيف تفعل ذلك؟

كيف تفعل ذلك؟

- A) استدعاء عدة `Task` [صحيح]
- B) استخدام `/compact`.
- C) فقط `tool_choice: auto` فرض.
- D) إرسال كل شيء لوكيل واحد.

يمكن للمنسق إنشاء عدة مهام في الدور نفسه، فتعمل الوكلاء الفرعيون بالتوازي A: لماذا

السؤال 9 (السيناريو: نظام بحث متعدد الوكلاء)

الموقف: فشل وكيل البحث جزئيًا لكنه وجد بعض النتائج. ما أفضل إخراج؟

ما أفضل إخراج؟

- A) فقط "failed" إرجاع.
- B) إرجاع نتائج جزئية، نوع الفشل، الاستعلام، وتأثير التغطية. [صحيح]
- C) إسقاط النتائج الجزئية.
- D) إعادة المحاولة إلى ما لا نهاية.

الإخراج المنظم يمكن المنسق من استخدام النتائج الجزئية أو إعادة التفويض أو وضع تعليق تغطية B: لماذا

السؤال 10 (السيناريو: Claude Code for CI)

دون تفاعل؟ CI في خط Claude Code الموقف: ما العلم الصحيح لتشغيل

ما العلم الصحيح؟

- A) `--resume`
- B) `--print` أو `-p` [صحيح]
- C) `/memory`
- D) `fork_session`

في وضع غير تفاعلي، يطبع النتيجة ثم يخرج Claude يشغل `-p` B: لماذا

السؤال 11 (السيناريو: Claude Code for CI)

جديدة؟ commits بعد PR الموقف: كيف تقلل التعليقات المكررة عند إعادة مراجعة

كيف تقلل التعليقات المكررة؟

- A) احذف كل نتائج المراجعة السابقة.

- ضمّن نتائج المراجعة السابقة واطلب فقط الجديد أو غير المحلول. [صحيح] B)
- استخدم الجلسة نفسها دائمًا C)
- JSON output عطل D)

على عدم تكرار الملاحظات القديمة والتركيز على المشكلات Claude إدراج نتائج المراجعة السابقة يساعد B: لماذا الجديدة أو غير المعالجة.

السؤال 12 (السيناريو: مراجعة كود متعددة الملفات)

طلب سحب يحتوي على 14 ملفًا دفعة واحدة، وتلاحظ أنه يقدم تحليلًا عميقًا لأول بضعة Claude الموقف: تراجع ملفات ثم تعليقات سطحية للبقية. ما أفضل إعادة هيكلة للمراجعة؟

ما أفضل إعادة هيكلة؟

- فقط max_tokens زيادة A)
- تحليل كل ملف على حدة أولًا، ثم تشغيل تمرير تكاملي للعلاقات بين الملفات. [صحيح] B)
- استخدام نموذج أصغر للمراجعة C)
- PR. حذف الملفات الأقل أهمية من D)

المراجعة متعددة المرور تقلل تشتت الانتباه. التحليل المحلي لكل ملف يضمن تغطية أفضل، والتمرير B: لماذا التكامل يلتقط مشكلات تدفق البيانات والاعتمادات بين الملفات.

اختبار تدريبي

ملاحظة: هذه الأسئلة مصممة لتدريبك على نمط التفكير المطلوب في الاختبار. ركز على سبب صحة الإجابة، لا على حفظ الحروف.

السيناريو: نظام بحث متعدد الوكلاء

السؤال 1 (السيناريو: نظام بحث متعدد الوكلاء)

الموقف: لديك منسق يرسل نفس سؤال البحث إلى وكيل بحث ويب ووكيل تحليل مستندات. كلا الوكيلين يرجعان نتائج متداخلة، والتقرير النهائي يكرر النقاط نفسها. ما أفضل تعديل؟

ما أفضل تعديل؟

- اجعل كل الوكلاء يملكون كل الأدوات حتى يقرروا بأنفسهم A)
- اجعل المنسق يحدد نطاقًا واضحًا لكل وكيل، مثل "بحث ويب حديث" مقابل "تحليل المستندات المرفقة". [صحيح] B)
- احذف المنسق ودع الوكلاء يتواصلون مباشرة C)
- max_tokens أضف المزيد من الرموز إلى D)

المشكلة هي تداخل النطاق. المنسق يجب أن يوزع العمل بحدود واضحة حتى يقل التكرار وتزيد التغطية: B لماذا

السؤال 2 (السيناريو: نظام بحث متعدد الوكلاء)

يحتاج إلى مصادر دقيقة، لكن الوكلاء الفرعيين يرجعون ملخصات بلا روابط ولا تواريخ. ما synthesis الموقف: وكيل أفضل حل؟

ما أفضل حل؟

- تخمين المصادر synthesis اطلب من A)
- ودرجة ثقة. [صحيح] publication_date و source_url منظمة مع claims اجعل كل وكيل فرعي يرجع B)
- زد عدد الوكلاء C)
- من التقرير النهائي citations أزل D)

الحفاظ على المصدرية يبدأ من الوكلاء الفرعيين. يجب تمرير الادعاءات مع مصادرها وتواريخها حتى يستطيع B: لماذا synthesis إنتاج تقرير قابل للتحقق

السؤال 3 (السيناريو: نظام بحث متعدد الوكلاء)

لكن الوكيل الفرعي يعطي نتيجة عامة وغير، "Analyze the document": الموقف: المنسق يطلب من وكيل فرعي مفيدة. ما المشكلة الأساسية؟

ما المشكلة الأساسية؟

- المطالبة لا تمرر المستند ولا متطلبات الإخراج صراحة. [صحيح] A)
- Batch API يجب استخدام B)
- يجب تقليل عدد الأدوات إلى صفر C)
- Markdown المشكلة في D)

الوكلاء الفرعيون لا يرثون سياق المنسق تلقائيًا. يجب تمرير المستند، الهدف، والمعايير المطلوبة داخل A: لماذا Task .

السؤال 4 (السيناريو: نظام بحث متعدد الوكلاء)

لماذا هذا سيئ؟ "search unavailable" أرجع الوكيل، timeout: الموقف: بعد فشل وكيل بحث بسبب

لماذا هذا سيئ؟

- لأنه لا يحتوي على تفاصيل تساعد المنسق على التعافي. [صحيح] A)
- لأنه طويل جدًا B)
- لأنه ليس باللغة الإنجليزية C)
- لأنه لا يستخدم max_tokens D)

يجب أن تتضمن الأخطاء نوع الفشل، هل هو قابل لإعادة المحاولة، الاستعلام الذي فشل، النتائج الجزئية، A: لماذا والبدايل.

السؤال 5 (السيناريو: نظام بحث متعدد الوكلاء)

الموقف: تريد تقليل زمن التنفيذ في بحث يتضمن 4 محاور مستقلة. ما أفضل تصميم؟

ما أفضل تصميم؟

- منفصلة في الدور نفسه لتعمل بالتوازي. [صحیح] Task اجعل المنسق يرسل كل محور ك A)
- نفذ كل المحاور تسلسليًا دائمًا B)
- يبحث بنفسه في كل شيء synthesis اجعل C)
- استخدم /memory D)

عندما تكون المحاور مستقلة، تشغيل الوكلاء الفرعيين بالتوازي يقلل الزمن ويحافظ على فصل السياقات A: لماذا

السؤال 6 (السيناريو: نظام بحث متعدد الوكلاء)

يفقد معلومات مهمة لأنها تقع في منتصف مدخلات طويلة جدًا. ما أفضل معالجة؟ synthesis :الموقف

ما أفضل معالجة؟

- في البداية، ونظم التفاصيل بعناوين واضحة. [صحیح] key findings ضع ملخص A)
- ضع كل شيء في الوسط B)
- احذف المصادر C)
- استخدم ترتيبًا عشوائيًا لكل مهمة D)

وضع النتائج الأساسية في البداية يستفيد من موثوقية معالجة بداية السياق، والعناوين تساعد النموذج على A: لماذا التنقل داخل المحتوى الطويل.

السؤال 7 (السيناريو: نظام بحث متعدد الوكلاء)

يعمل جيدًا تحت synthesis من محتوى خام وسلاسل تفكير، لكن K tokens الموقف: الوكلاء الفرعيون يرجعون 155 ما الحل الأكثر فعالية؟ 50K tokens.

ما الحل الأكثر فعالية؟

- يرجعون بيانات منظمة ومختصرة: حقائق، اقتباسات، درجات صلة، ومصادر. [صحیح] upstream اجعل الوكلاء A)
- زد حجم السياق فقط B)
- يقرأ كل شيء كما هو synthesis اجعل C)
- ألقِ الوكلاء الفرعيين D)

تقليل الحجم من المصدر أفضل من تلخيص لاحق قد يفقد التفاصيل. الوكلاء يجب أن يرجعوا البيانات المهمة لا A: لماذا تفريغات ضخمة.

ما أفضل نمط؟

- يقرأه المنسق عند الاستئناف. [صحيح] manifest اجعل كل وكيل يصدر حالته المنظمة إلى ملفات مع A)
- اعتمد على ذاكرة النموذج فقط B)
- أعد تنفيذ كل شيء من الصفر دائمًا C)
- احذف النتائج الجزئية D)

استمرار الحالة المنظمة يسمح باستئناف موثوق، ومعرفة ما اكتمل وما لا يزال قيد التنفيذ: A لماذا

السؤال 12 (السيناريو: نظام بحث متعدد الوكلاء)

الموقف: المنسق يختار وكلاء فرعيين كثيرين جدًا لمهمة بسيطة، مما يزيد التعقيد والوقت. ما السبب المرجح؟

ما السبب المرجح؟

- التفكيك المفرط الضيق أو غير مناسب لحجم المهمة. [صحيح] A)
- يجب زيادة عدد الأدوات B)
- يجب حذف Task . C)
- يجب إلغاء كل القيود D)

التفكيك يجب أن يناسب المهمة. استخدام وكلاء أكثر من اللازم يزيد التنسيق والتكلفة دون فائدة: A لماذا

السؤال 13 (السيناريو: نظام بحث متعدد الوكلاء)

يستشهد K tokens، الموقف: تُظهر مراقبة الإنتاج أن جودة التركيب غير متسقة. عندما تكون النتائج المجمعة نحو 75 K tokens وآخر 10 (عناوين/مقتطفات بحث الويب) K tokens بالمعلومات الموجودة في أول 15 reliably وكيل التركيب حتى عندما تجيب مباشرة عن K tokens، لكنه غالبًا يفوّت نتائج حرجية في وسط الـ 50، (استنتاجات تحليل المستندات) سؤال البحث. كيف ينبغي إعادة هيكلة المدخلات المجمعة؟

كيف ينبغي إعادة هيكلة المدخلات المجمعة؟

- قبل التجميع حتى يبقى المحتوى ضمن نطاق K tokens تلخيص كل مخرجات الوكلاء الفرعيين إلى أقل من 20 A)
- معالجة الموثوقة للنموذج.
- بـ نتائج الوكلاء الفرعيين إلى وكيل التركيب تدريجيًا، مع معالجة نتائج بحث الويب أولاً حتى الاكتمال، ثم إضافة B)
- نتائج تحليل المستندات.
- وضع ملخص للنتائج الأساسية في بداية المدخلات المجمعة، وتنظيم النتائج التفصيلية بعناوين أقسام واضحة C)
- لتسهيل التنقل [CORRECT].
- تنفيذ تدوير يبدّل أي نتائج وكيل فرعي تظهر أولاً عبر مهام البحث، لضمان حصول كلا المصدرين على تموضع D)
- علوي متساوٍ بمرور الوقت.

وضع ملخص للنتائج الأساسية في البداية يستفيد من أثر الأولوية، بحيث تكون المعلومات الحرجية في الموضع: C لماذا الأكثر موثوقية للمعالجة. كما أن إضافة عناوين أقسام واضحة تساعد النموذج على التنقل والانتباه إلى محتوى منتصف "المدخلات، مما يخفف مباشرة من ظاهرة "الضياع في الوسط".

السؤال 14 (السيناريو: نظام بحث متعدد الوكلاء)

ووكيل تحليل (تشمل محتوى الصفحات 85K tokens) **الموقف:** في الاختبار، يبلغ مجموع مخرجات وكيل بحث الويب لكن وكيل التركيب يعمل بأفضل أداء عندما تكون 155 tokens نحو 70K tokens (تشمل سلاسل التفكير المستندات أي حل هو الأكثر فعالية؟ 50 tokens المدخلات أقل من

أي حل هو الأكثر فعالية؟

- لإرجاع بيانات منظمة (حقائق أساسية، اقتباسات، درجات صلة) بدل upstream agents تعديل الوكلاء السابقين A) [CORRECT]. المحتوى المطوّل والتفكير المطوّل
- إضافة وكيل تلخيص وسيط يختصر النتائج قبل تمريرها إلى التركيب B)
- جعل وكيل التركيب يعالج النتائج على دفعات متتابة مع الحفاظ على الحالة بين الاستدعاءات C)
- وإعطاء وكيل التركيب أدوات بحث للاستعلام أثناء عمله vector database تخزين النتائج في قاعدة بيانات D)

تعديل الوكلاء السابقين لإرجاع بيانات منظمة يعالج السبب الجذري بتقليل حجم الرموز من المصدر مع **A: لماذا** الحفاظ على المعلومات الأساسية. هذا يتجنب تمرير محتوى صفحات ضخمة وسلاسل تفكير تزيد الرموز دون تحسين خطوة التركيب.

السؤال 15 (السيناريو: نظام بحث متعدد الوكلاء)

الموقف: في الاختبار، تلاحظ أن وكيل التركيب يحتاج غالبًا إلى التحقق من ادعاءات محددة أثناء دمج النتائج. حاليًا، عند الحاجة إلى التحقق، يعيد وكيل التركيب التحكم إلى المنسق، فيستدعي المنسق وكيل بحث الويب ثم يعيد استدعاء التركيب مع النتائج. هذا يضيف 2-3 حلقات إضافية لكل مهمة ويزيد زمن الاستجابة بنسبة 40%. يُظهر تقييمك أن 85% من هذه التحقيقات هي فحوصات حقائق بسيطة (تواريخ، أسماء، إحصاءات)، وأن 15% تتطلب بحثًا أعمق. أي نهج يقلل العبء بأكبر فعالية مع الحفاظ على موثوقية النظام؟

أي نهج هو الأكثر فعالية؟

- إعطاء وكيل التركيب وصولاً إلى كل أدوات بحث الويب حتى يتعامل مباشرة مع أي حاجة للتحقق دون حلقات A) المنسق.
- جعل وكيل التركيب يجمع كل احتياجات التحقق ويعيدها كدفعة إلى المنسق في النهاية، ثم يرسلها المنسق كلها B) إلى وكيل بحث الويب مرة واحدة.
- جعل وكيل بحث الويب يخزن سياقًا إضافيًا حول كل مصدر أثناء البحث الأولي تحسبًا لاحتياج التركيب إلى C) التحقق.
- للفحوصات البسيطة، مع توجيه التحقيقات المعقدة `verify_fact` إعطاء وكيل التركيب أداة محدودة النطاق D) [CORRECT]. عبر المنسق إلى وكيل بحث الويب

محدودة النطاق تقلل الحلقات لمعظم التحقيقات البسيطة، مع الحفاظ على تحكم `verify_fact` أداة **D: لماذا** المنسق في الحالات المعقدة أو المفتوحة. هذا يوازن بين تقليل زمن الاستجابة والحفاظ على بنية موثوقة قابلة للملاحظة.

للتكامل المستمر Claude Code: السيناريو السؤال 16

لتوفير سياق CLAUDE.md باستخدام `--print` في وضع Claude Code CLI لديك واجهة CI **الموقف:** يشغل خط PR المشروع لمراجعة الكود، ويجد المطورون عمومًا أن المراجعات ذات قيمة. لكنهم يذكرون أن دمج النتائج في

صعب لأن المخرجات نص حر: أحيانًا تكون المشكلات في فقرات، وأحيانًا في قوائم، وأحيانًا بلا أرقام أسطر. تريد نشر ما التغيير الأكثر فعالية؟ GitHub تلقائيًا في inline تعليقات

ما التغيير الأكثر فعالية؟

- A) تنسيق النتائج باستمرار " داخل المطالبة، مع أمثلة لتنسيق جيد" Claude اطلب من
- B) message و severity و line و file يفرض حقولًا مثل `--json-schema` مع `--output-format json` استخدم [CORRECT]
- C) لاستخراج أسماء الملفات وأرقام الأسطر من النص الحر regex تستخدم post-processing شغل خطوة
- D) inline كاملاً بدل التعليقات Claude اطلب من المطورين قراءة تقرير

الاعتماد. inline يجعل النتائج قابلة للمعالجة آليًا وموثوقة للنشر كتعليقات JSON Schema الإخراج المنظم مع B: لماذا مستقر CI هش وغير كافٍ لتكامل regex على نص حر أو

للتكامل المستمر Claude Code: السيناريو السؤال 17

لتوليد اقتراحات كود، لكنك تلاحظ نمطًا: المشكلات غير الواضحة — مثل Claude Code الموقف: يستخدم فريقك نفسه Claude تغيير السلوك دون قصد — لا تُكتشف إلا عندما يراجعها cleanups تحسينات أداء تكسر حالات حدية أو في الجلسة نفسها التي اقترحت التغيير. تريد تحسين اكتشاف هذه المشكلات. ما أفضل تصميم للمراجعة؟

ما أفضل تصميم للمراجعة؟

- A) يراجع الكود في الجلسة نفسها التي ولدت الاقتراح حتى يعرف النية الأصلية Claude اجعل
- B) [CORRECT]. لمراجعة الكود المولد Claude استخدم مثيلًا مستقلًا أو جلسة منفصلة من
- C) إضافة تعليقات أكثر في الكود تشرح قراراته Claude اطلب من
- D) شغل الاختبارات فقط، لأن المراجعة المستقلة مكررة

الجلسة نفسها قد تكون منحازة إلى قراراتها وسياقها السابق. مثيل مستقل يراجع الكود بزاوية جديدة ويكون B: لماذا أكثر قدرة على تحدي الافتراضات واكتشاف الأخطاء غير الواضحة

للتكامل المستمر Claude Code: السيناريو السؤال 18

(imports، الملف المتغير، ثم قد يطلب ملفات ذات صلة Claude الموقف: مكوّن مراجعة الكود لديك تكراري: يحل عبر استدعاءات أدوات لفهم السياق قبل تقديم الملاحظات النهائية. تطبيقك يحاول نقل هذه (base classes، tests) لتقليل التكلفة. ما المشكلة الأساسية؟ Message Batches API العملية إلى

ما المشكلة الأساسية؟

- A) [CORRECT]. لا تدعم استدعاء الأدوات متعدد الأدوار داخل الطلب الواحد Message Batches API
- B) Message Batches API لا تدعم JSON output.
- C) لا يمكنها معالجة ملفات الكود Message Batches API
- D) Message Batches API لا تسمح باستخدام `custom_id`.

مناسب لطلبات مستقلة ذات استجابة واحدة، لكنه لا يدعم حلقات تفاعلية متعددة الأدوار حيث Batch API A: لماذا متزامنة أو حلقة وكيل خارجية API يطلب النموذج أدوات ثم يتلقى نتائجها ثم يتابع التفكير. هذا النمط يحتاج

(للتكامل المستمر Claude Code: السيناريو) السؤال 19

تمنع PR فحوصات أسلوب سريعة على كل (1) Claude: لديك ثلاث تحليلات مبنية على CI/CD الموقف: يشغل نظام الدمج حتى تكتمل، (2) تدقيقات أمنية أسبوعية شاملة لقاعدة الكود كلها، و(3) توليد حالات اختبار ليلية للملفات التي هي الأنسب؟ API تغيرت خلال اليوم. تريد تقليل التكلفة دون التأثير على سير المطورين. أي استراتيجية

هي الأنسب؟ API أي استراتيجية

- لكل التحليلات الثلاثة لتقليل التكلفة API Message Batches استخدم A)
- للتدقيقات API Message Batches استخدم الاستدعاءات المتزامنة للفحوصات التي تمنع الدمج، واستخدم B) [CORRECT]
- استخدم الاستدعاءات المتزامنة لكل شيء لتجنب اختلافات زمن المعالجة C)
- للفحوصات التي تمنع الدمج، والاستدعاءات المتزامنة للوظائف الليلية Batch API استخدم D)

الفحوصات التي تمنع الدمج تحتاج زمن استجابة متوقعًا وسريعًا. أما التدقيقات الأسبوعية والتوليد الليلي B: لماذا رغم زمن معالجة قد يصل إلى 24 ساعة Batch API فليست تفاعلية ويمكنها الاستفادة من توفير

(للتكامل المستمر Claude Code: السيناريو) السؤال 20

الموقف: تجد مراجعاتك الآلية مشكلات حقيقية، لكن المطورين يقولون إن الملاحظات غير قابلة للتنفيذ. تتضمن النتائج دون تحديد الملف/السطر/السبب/الإصلاح. ما أفضل "null pointer عبارات مثل" منطق توجيه التذاكر معقد" أو "احتمال تحسين؟

ما أفضل تحسين؟

- إنتاج نتائج منظمة تشمل الموقع، الوصف، سبب المشكلة، الأثر، واقتراح إصلاح محدد Claude اطلب من A) [CORRECT]
- تقليل عدد النتائج إلى أعلى 3 فقط Claude اطلب من B)
- لتعليقات المطورين sentiment analysis أضف C)
- يستخدم لغة ألطف حتى تكون الملاحظات مقبولة أكثر Claude اجعل D)

المشكلة ليست النبذة أو العدد فقط، بل نقص القابلية للتنفيذ. يجب أن تتضمن كل نتيجة موقعًا واضحًا، وسببًا، لماذا A: لماذا وأثرًا، وإصلاحًا مقترحًا حتى يستطيع المطور التعامل معها بسرعة

(للتكامل المستمر Claude Code: السيناريو) السؤال 21

حتى يكتمل، PR قبل الدمج يمنع دمج hook Claude: لديك وضعين لمراجعة الكود باستخدام CI الموقف: يتضمن خط و"تحليل عميق" يعمل ليلاً، يستعلم عن اكتمال الدفعة، وينشر اقتراحات مفصلة في الصباح. أي وصف صحيح؟

أي وصف صحيح؟

- لتقليل التكلفة API Message Batches يجب تشغيل الوضعين عبر A)
- CI يجب تشغيل الوضعين باستدعاءات متزامنة لأن كلاهما جزء من B)
- للتحليل الليلي العميق API Message Batches و pre-merge hook استخدم الاستدعاءات المتزامنة لـ C) [CORRECT]

- لأنه أقصر، والاستدعاءات المتزامنة للتحليل العميق لأنه pre-merge hook لل Message Batches API استخدم D) أطول.

أما التحليل الليلي فيناسب المعالجة Batch API أي شيء يمنع الدمج يحتاج استجابة فورية تقريبًا ولا يناسب C: لماذا
غير المتزامنة لأن النتيجة مطلوبة لاحقًا.

22 للتكامل المستمر Claude Code: السيناريو السؤال

التحقق من أن التعليقات Claude تطلب المطالبة الحالية من docstrings. الموقف: تحليل المراجعة الآلية التعليقات و
أو تعليقات وصفية بسيطة، أو تعليقات لا تغطي TODO markers دقيقة ومحدثة. غالبًا ترفع النتائج أنماطًا مقبولة مثل
كل التفاصيل. كيف تحسن الدقة؟

كيف تحسن الدقة؟

- [CORRECT]. أضف معايير صريحة لما يجب اعتباره تعليقًا غير صحيح، وما لا يجب رفعه A)
- "أن يكون" أكثر تحفظًا Claude اطلب من B)
- false positives عطّل كل مراجعة التعليقات لأنها تسبب C)
- استخدم نموذجًا أكبر فقط D).

التعليمات المبهمة تؤدي إلى تفسيرات واسعة. المعايير الصريحة، مثل رفع التعليق فقط عندما يناقض سلوك A: لماذا
الكود أو يشير إلى دالة غير موجودة، تقلل الإيجابيات الكاذبة مباشرة.

23 للتكامل المستمر Claude Code: السيناريو السؤال

تُقيّم null pointer الموقف: يُظهر نظام المراجعة الآلي تقييمات شدة غير متسقة؛ مشكلات متشابهة مثل مخاطر
في أخرى. تُظهر استطلاعات المطورين انخفاض الثقة لأنهم لا يعرفون كيف "medium" و PRs في بعض "critical"
يفسرون الشدة. ما أفضل تحسين؟

ما أفضل تحسين؟

- [CORRECT]. عرّف معايير شدة صريحة مع أمثلة لكل مستوى A)
- أزل كل تقييمات الشدة B).
- استخدم رقم ثقة فقط بدل الشدة C).
- "استخدم الشدة باستمرار" Claude اطلب من D).

معايير الشدة الواضحة مع أمثلة تقلل التذبذب وتوفر أساسًا مشتركًا لتفسير النتائج. التعليمات العامة لا تكفي A: لماذا
لتوحيد التصنيف.

24 للتكامل المستمر Claude Code: السيناريو السؤال

يضيف تتبع إكمال الدورات، يقترح PR عند مراجعة PR. الموقف: تولد المراجعة الآلية اقتراحات حالات اختبار لكل
عشر حالات اختبار، لكن ملاحظات المطورين تُظهر أن 6 سيناريوهات مكررة مغطاة بالفعل في ملفات اختبار Claude
موجودة. ما أفضل تحسين؟

ما أفضل تحسين؟

- واطلب اقتراح الاختبارات الجديدة غير المغطاة Claude ضمّن ملفات الاختبار الحالية ذات الصلة في سياق A) فقط. [CORRECT]
- توليد عدد أقل من الاختبارات Claude اطلب من B)
- احذف كل الاختبارات القديمة قبل التوليد C)
- فقط end-to-end يقترح اختبارات Claude اجعل D)

لا يرى التغطية الموجودة. توفير ملفات الاختبار الحالية يسمح له بتجنب التكرار Claude المشكلة هي أن A: لماذا واقتراح فجوات حقيقية.

25 للتكامل المستمر Claude Code: السيناريو السؤال

جديدة لمعالجة المشكلات. إعادة تشغيل commits الموقف: بعد مراجعة آلية أولية تحدد 12 نتيجة، يدفع المطور المراجعة تنتج 8 نتائج، لكن المطورين يقولون إن 5 منها تعليقات مكررة سابقة على كود لم يعد موجودًا أو تم إصلاحه جزئيًا. ما أفضل طريقة لتقليل التكرار؟

ما أفضل طريقة؟

- الإبلاغ فقط عن المشكلات Claude ضمّن نتائج المراجعة السابقة وسياق التغييرات الجديدة، واطلب من A) [CORRECT]. الجديدة أو التي لا تزال غير محلولة
- من رؤية أي مراجعات سابقة حتى يكون مستقرًا Claude امنع B)
- جديدة commits لا تعيد تشغيل المراجعة بعد C)
- احذف كل التعليقات القديمة قبل تشغيل المراجعة D)

على تمييز ما تم الإبلاغ عنه سابقًا وما تم إصلاحه وما لا يزال قائمًا. هذا Claude إدراج النتائج السابقة يساعد A: لماذا يقلل التعليقات المكررة ويركز على المشكلات الجديدة أو غير المعالجة.

26 للتكامل المستمر Claude Code: السيناريو السؤال

، "claude \"Analyze this pull request for security issues\" الأمر pipeline الموقف: يشغل سكربت ينتظر إدخالًا تفاعليًا. ما النهج الصحيح Claude Code يظل معلقًا إلى أجل غير مسمى. تُظهر السجلات أن job لكن CI؟ في Claude Code لتشغيل

ما النهج الصحيح؟

- أو claude -p \"Analyze this pull request for security issues\" استخدم A) --print. [CORRECT]
- قبل الأمر /compact استخدم B)
- CI دائمًا داخل --resume استخدم C)
- قصير فقط واتركه يفشل timeout أضف D)

في وضع غير تفاعلي، فيعالج المطالبة ويطبع النتيجة ويخرج Claude Code يشغل --print أو -p العلم A: لماذا CI. هذا هو النمط المناسب لـ

للتكامل المستمر Claude Code: السيناريو السؤال 27

أربعة عشر ملفًا في وحدة تتبع المخزون. مراجعة واحدة تمر على كل الملفات معًا تنتج نتائج غير PR الموقف: يغيّر متسقة: ملاحظات مفصلة لبعض الملفات، وتعليقات سطحية لأخرى، وأخطاء واضحة فائتة، وتغذية راجعة متناقضة. كيف ينبغي إعادة هيكلة المراجعة؟

كيف ينبغي إعادة هيكلة المراجعة؟

- تقسيمها إلى تمريرات مركزية: تحليل كل ملف منفردًا للمشكلات المحلية، ثم تشغيل تمريرة تكامل منفصلة A) [CORRECT]. لتدفقات البيانات بين الملفات
- الكبيرة إلى 3-4 ملفات PRs إجبار المطورين على تقسيم B).
- التبديل إلى نموذج سياق أكبر ومراجعة كل الملفات في تمريرة واحدة C).
- تشغيل ثلاث مراجعات كاملة مستقلة والإبلاغ فقط عن المشكلات التي تظهر في اثنتين منها على الأقل D).

التمريرات المركزية تعالج السبب الجذري، وهو تشتت الانتباه عند معالجة ملفات كثيرة دفعة واحدة. التحليل A: لماذا لكل ملف يضمن عمقًا متسقًا، وتمرير التكامل يلتقط المشكلات بين الملفات.

للتكامل المستمر Claude Code: السيناريو السؤال 28

قدره 40%. عنق false positive وبلغ المطورون عن معدل PR، الموقف: متوسط المراجعة الآلية لديك 15 نتيجة لكل لمعرفة هل هي حقيقية. يحتوي Claude الزجاجة هو وقت التحقيق: يجب على المطورين فتح كل نتيجة وقراءة منطق بالفعل على قواعد شاملة للأنماط المقبولة، ورفض أصحاب المصلحة أي نهج يرشح النتائج قبل أن يراها CLAUDE.md المطورون. ما التغيير الذي يعالج وقت التحقيق بأفضل شكل؟

ما التغيير الذي يعالج وقت التحقيق بأفضل شكل؟

- تضمين المبرر وتقدير الثقة مباشرة داخل كل نتيجة Claude اطلب من A) [CORRECT]
- تاريخية false positives يحلل أنماط النتائج ويخفي تلقائيًا النتائج التي تطابق توقعيات post-processor أضف B).
- مع متطلبات مراجعة مختلفة حسب المستوى "suggestions" مقابل "blocking issues" صنف النتائج إلى C).
- يعرض النتائج عالية الثقة فقط، مع ترشيح العلامات غير المؤكدة قبل أن يراها المطورون Claude اجعل D).

تضمين المبرر والثقة داخل كل نتيجة يقلل وقت التحقيق لأنه يسمح للمطورين بالفرز السريع دون فتح كل A: لماذا نتيجة. كما يلتزم بقيد "عدم الترشيح" لأن كل النتائج تبقى مرئية مع تسريع قرار المطور.

للتكامل المستمر Claude Code: السيناريو السؤال 29

حسب الفئة: نتائج الأمن/الصحة false positives الموقف: يُظهر تحليل مراجعة الكود الآلية اختلافات كبيرة في معدلات الأداء 18%، الأسلوب/التسمية 52%، والتوثيق 48%. تُظهر استطلاعات، false positives 8% فيها correctness false positives المطورين تراجعًا في الثقة؛ كثيرون يبدأون بتجاهل النتائج دون قراءتها لأن "نصفها خطأ". الفئات ذات العالية تضعف الثقة حتى في الفئات الدقيقة. أي نهج يستعيد ثقة المطورين بأفضل شكل مع تحسين النظام؟

أي نهج يستعيد ثقة المطورين بأفضل شكل؟

- العالية مؤقتًا (الأسلوب، التسمية، التوثيق)، والإبقاء فقط على الفئات عالية false positives تعطيل الفئات ذات A) [CORRECT]. الدقة أثناء تحسين المطالبات
- إبقاء كل الفئات مفعلة مع عرض درجات ثقة لكل نتيجة حتى يقرر المطورون ما يحققون فيه B).

- لتحسين الدقة لكل فئة خلال الأسابيع القادمة few-shot إبقاء كل الفئات مفعلة وإضافة أمثلة C)
- الإجمالي false positives تطبيق تقليل موحد للصرامة عبر كل الفئات لخفض معدل D)

تعطيل الفئات المزعجة مؤقتًا يوقف تآكل الثقة فورًا بإزالة النتائج الصاخبة التي تجعل المطورين يتجاهلون كل A: لماذا شيء، مع الحفاظ على قيمة الفئات عالية الدقة مثل الأمن والصحة. كما يتيح تحسين مطالبات الفئات المشككة قبل إعادة تفعيلها.

30 (للتكامل المستمر Claude Code: السيناريو) السؤال

ين: (1) workflow المتزامنة Claude للتحليل الآلي. حاليًا تدعم استدعاءات API الموقف: يريد فريقك تقليل تكاليف مانع يجب أن يكتمل قبل أن يستطيع المطورون الدمج، و(2) تقرير ديون تقنية يُؤلّد ليلاً للمراجعة في pre-merge فحص لتوفير 50%. كيف ينبغي تقييم هذا الاقتراح؟ Message Batches API صباح اليوم التالي. يقترح مديرك نقل كليهما إلى

كيف ينبغي تقييم هذا الاقتراح؟

- إلى الاستدعاءات المتزامنة إذا استغرقت الدفعات وقتًا طويلاً fallback انقل كليهما إلى المعالجة بالدفعات مع A)
- للتحقق من الاكتمال polling إلى المعالجة بالدفعات مع workflows انقل كلا ال B)
- pre-merge استخدم المعالجة بالدفعات فقط لتقارير الديون التقنية، واحتفظ بالاستدعاءات المتزامنة لفحوصات C) merge. [CORRECT]
- احتفظ بالاستدعاءات المتزامنة لكليهما لتجنب مشكلات ترتيب نتائج الدفعات D)

للكمون، وهذا مقبول لتقارير الديون التقنية SLA حتى 24 ساعة دون Message Batches API قد تستغرق C: لماذا API مع workflow المانعة حيث ينتظر المطورون. هذا يطابق كل pre-merge الليلية، لكنه غير مقبول لفحوصات المناسب حسب متطلبات زمن الاستجابة.

Claude Code السيناريو: توليد الكود باستخدام

31 (Claude Code السيناريو: توليد الكود باستخدام) السؤال

إلى صيغة داخلية موحدة. بعد تكرارين، لا يزال API تنفيذ دالة تحول استجابات Claude Code الموقف: طلبت من هيكل الإخراج لا يطابق التوقعات: بعض الحقول متداخلة بشكل مختلف، والتواريخ تُنسق بطريقة خاطئة. وصفت يفسرها بشكل مختلف في كل مرة Claude المتطلبات نشرًا، لكن

ما النهج الأكثر فعالية للتكرار التالي؟

- مقابله بعد كل تكرار Claude يصف بنية الإخراج المتوقعة وتحقق من مخرجات JSON Schema اكتب A)
- [CORRECT] تمثيلية API قَدِّم 2-3 أمثلة إدخال/إخراج ملموسة توضح التحويل المتوقع لاستجابات B)
- أعد كتابة المتطلبات بدقة تقنية أكبر، مع تحديد خرائط الحقول، وقواعد التداخل، وصيغ التواريخ بالضبط C)
- شرح فهمه الحالي للمتطلبات لتحديد مواضع اختلاف التفسير Claude اطلب من D)

الأمثلة الملموسة للإدخال والإخراج تزيل الغموض الموجود في الوصف الشرطي لأنها تعرض النتيجة المتوقعة B: لماذا بالضبط. هذا يعالج السبب الجذري مباشرة، وهو سوء تفسير المتطلبات النصية، عبر تقديم أنماط واضحة لبنية الحقول وتنسيق التواريخ.

Claude Code السيناريو: توليد الكود باستخدام) السؤال 32

كقناة إشعارات جديدة. قاعدة الكود الحالية لديها أنماط واضحة لقنوات البريد Slack الموقف: تحتاج إلى إضافة بسيطة وأحادية) incoming webhooks: تقدم طرق تكامل مختلفة جذريًا Slack API لكن push و SMS الإلكتروني و أحداث ثنائية الاتجاه وتتطلب موافقة) Slack Apps أو ،(تدعم تأكيد التسليم والتحكم البرمجي) bot tokens أو ،(الاتجاه دون تحديد طريقة التكامل أو طلب مزايا متقدمة مثل تتبع التسليم "Slack المهمة تقول" أضف دعم (workspace).

كيف ينبغي التعامل مع هذه المهمة؟

- لتطابق نمط الإشعارات أحادي الاتجاه الحالي incoming webhooks ابدأ بوضع التنفيذ المباشر باستخدام A)
- انتقل إلى وضع التخطيط لاستكشاف خيارات التكامل وتبعاتها المعمارية، ثم قدم توصية قبل التنفيذ B)
- باستخدام الأنماط الحالية، وأجل قرار طريقة التكامل Slack channel class ابدأ بوضع التنفيذ المباشر بإنشاء C)
- حتى يكون تأكيد التسليم ممكنًا bot-token ابدأ بوضع التنفيذ المباشر باستخدام D)

[CORRECT]

يملك عدة طرق صحيحة، ولكل طريقة تبعات معمارية مختلفة، والمتطلبات غامضة. وضع Slack تكامل B: لماذا والاتفاق على النهج قبل التنفيذ Slack Apps و bot tokens و webhooks التخطيط يسمح بتقييم المفاضلات بين

Claude Code السيناريو: توليد الكود باستخدام) السؤال 33

لديك أكثر من 400 سطر، ويحتوي على معايير كتابة الكود، واصطلاحات الاختبار، CLAUDE.md الموقف: أصبح ملف دائمًا معايير الكود Claude مفصلة، وتعليمات النشر، وإجراءات ترحيل قاعدة البيانات. تريد أن يتبع PR وقائمة مراجعة فقط عند تنفيذ تلك المهام migrations و deploy و PR review واصطلاحات الاختبار، لكن يطبق إرشادات

ما نهج إعادة الهيكلة الأكثر فعالية؟

- واترك وصفاً مختصراً للمشروع، workflow منفصلة منظمة حسب نوع Skills انقل كل الإرشادات إلى ملفات A)
- لتنظيمه في ملفات منفصلة حسب الفئة @import لكن استخدم صيغة CLAUDE.md أبق كل شيء في B)
- مرتبطة بالمسارات بحيث لا تُحمّل glob patterns مع .claude/rules/ إلى ملفات تحت CLAUDE.md قسّم C)
- كل قاعدة إلا لأنواع الملفات ذات الصلة
- PR review مثل workflows للإرشادات الخاصة بالـ Skills وأنشئ CLAUDE.md أبق المعايير العامة في D)
- trigger keywords مع migrations و deploy [CORRECT]

Skills يُحمّل في كل جلسة، لذلك يناسب المعايير العامة التي يجب أن تطبق دائمًا. أما CLAUDE.md محتوى D: لماذا مناسبة، وهذا مثالي للإرشادات الخاصة بسير عمل معين trigger كلمات Claude فتُستدعى عند الحاجة عندما يكتشف migrations أو النشر أو PR مثل مراجعة

Claude Code السيناريو: توليد الكود باستخدام) السؤال 34

يؤثر هذا على عشرات الملفات ويتطلب قرارات. microservices إلى monolithic الموقف: كُلفت بإعادة هيكلة تطبيق حول حدود الخدمات واعتماديات الوحدات.

أي نهج ينبغي اختياره؟

- انتقل إلى وضع التخطيط لاستكشاف قاعدة الكود وفهم الاعتماديات وتصميم نهج التنفيذ قبل إجراء التغييرات A) [CORRECT]
- ابدأ بوضع التنفيذ المباشر، وانتقل إلى التخطيط فقط بعد مواجهة تعقيد غير متوقع أثناء التنفيذ B)
- ابدأ بوضع التنفيذ المباشر ونفذ تغييرات تدريجية، ودع التنفيذ يكشف حدود الخدمات الطبيعية C)
- استخدم التنفيذ المباشر مع تعليمات مسبقة مفصلة تحدد بنية كل خدمة D)

؛ فهو يسمح monolith وضع التخطيط هو الاستراتيجية المناسبة لإعادة هيكلة معمارية معقدة مثل تقسيم A: لماذا باستكشاف آمن واتخاذ قرارات مستنيرة حول الحدود قبل الالتزام بتغييرات مكلفة عبر ملفات كثيرة.

35 Claude Code السيناريو: توليد الكود باستخدام) السؤال

تنفذ تحليلًا عميقًا للكود: فحص الاعتماديات، وإحصاءات تغطية /analyze-codebase الموقف: أنشأ فريقك مهارة أصبح أقل استجابة في الجلسة Claude الاختبارات، ومقاييس جودة الكود. بعد تشغيل الأمر، يذكر أعضاء الفريق أن ويفقد سياق المهمة الأصلية.

كيف تصلح ذلك بأفضل طريقة مع الحفاظ على قدرات التحليل الكاملة؟

- الخاص بالمهارة لتشغيل التحليل في سياق وكيل فرعي معزول frontmatter في context: fork أضف A) [CORRECT]
- لاستخدام نموذج أسرع وأرخص للتحليل frontmatter في model: haiku أضف B)
- قسّم المهارة إلى ثلاث مهارات أصغر، بحيث تنتج كل واحدة مخرجات أقل C)
- أضف تعليمات إلى المهارة لضغط كل النتائج في ملخص قصير قبل عرضها D)

يشغل التحليل في سياق وكيل فرعي معزول، بحيث لا تلوث المخرجات الكبيرة نافذة context: fork لماذا A: المهمة الأصلية. هذا يحافظ على قدرات التحليل الكاملة مع إبقاء الجلسة Claude سياق الجلسة الرئيسية ولا يفقد الرئيسية مستجيبة.

36 Claude Code السيناريو: توليد الكود باستخدام) السؤال

يريد أحد المطورين .claude/skills/commit/SKILL.md في /commit الموقف: يستخدم فريقك مهارة دون التأثير على زملائه (وفحوصات إضافية، commit صيغة مختلفة لرسالة) تخصيصها لسير عمله الشخصي

بماذا توصي؟

- باسم مختلف، مثل ~/claude/skills/ إنشاء نسخة شخصية تحت A) /my-commit
- الخاص بمهارة المشروع frontmatter إضافة منطق شرطي حسب اسم المستخدم في B)
- بالاسم نفسه ~/claude/skills/commit/SKILL.md إنشاء نسخة شخصية في C) [CORRECT]
- الخاص بالمهارة الشخصية لجعلها أعلى أولوية من نسخة المشروع frontmatter في override: true تعيين D)

المهارات الشخصية لها أولوية على مهارات المشروع عند استخدام الاسم نفسه. لذلك فإن مهارة شخصية في C: لماذا ستتجاوز مهارة الفريق، مما يسمح للمطور بتخصيص سير عمله مع ~/claude/skills/commit/SKILL.md لاستخدامه الشخصي، دون التأثير على زملائه /commit الحفاظ على اسم الأمر المعتاد

37 Claude Code السيناريو: توليد الكود باستخدام) السؤال

“always” يتبع إرشاد Claude منذ أشهر. مؤخرًا، يقول ثلاثة مطورين إن Claude Code الموقف: يستخدم فريقك لا يتبعه. الأربعة يعملون في Claude لكن مطورًا رابعًا انضم حديثًا يقول إن “include comprehensive error handling”، الرينو نفسه ولديهم أحدث نسخة من الكود.

ما السبب الأكثر احتمالاً وما الإصلاح؟

- A) الخاصة بالمطورين الأصليين على مستوى المستخدم، `~/claude/CLAUDE.md` الإرشاد موجود في ملفات `.claude/CLAUDE.md` وليس في ملف المشروع `.claude/CLAUDE.md`. أعضاء الفريق [CORRECT]
- B) لدى المطور الجديد يحتوي تعليمات متعارضة تتجاوز إعدادات المشروع؛ يجب `~/claude/CLAUDE.md` ملف `.claude/CLAUDE.md` حذف القسم المتعارض.
- C) يتعلم تفضيلات كل مستخدم بمرور الوقت؛ يجب على المطور الجديد تكرار المتطلب حتى Claude Code Claude.
- D) مؤقتًا بعد أول قراءة؛ المطورون الأصليون يستخدمون نسخًا مخزنة. يجب `CLAUDE.md` يزن Claude Code cache الخاص بـ `cache` على الجميع مسح.

إذا أضيف الإرشاد فقط إلى إعدادات المستخدم الخاصة بالمطورين الأصليين ولم يُضف إلى A: لماذا على مستوى المشروع، فلن يحصل عليه أعضاء الفريق الجدد. نقله إلى إعداد المشروع `.claude/CLAUDE.md` يضمن أن كل الأعضاء الحاليين والمستقبليين يحصلون عليه تلقائيًا.

38 Claude Code السيناريو: توليد الكود باستخدام) السؤال

API endpoints كسياق يحسن الاتساق كثيرًا عند توليد endpoints الموقف: تجد أن تضمين 2–3 أمثلة كاملة لتنفيذ جديدة، وليس عند التصحيح أو مراجعة الكود أو الأعمال endpoints جديدة. لكن هذا السياق مفيد فقط عند إنشاء API الأخرى في مجلد.

ما نهج الإعداد الأكثر فعالية؟

- A) الخاص بالمشروع حتى تكون متاحة دائمًا `CLAUDE.md` وتوثيق الأنماط إلى endpoints أضف أمثلة A).
- B) في كل طلب توليد عبر نسخ الكود إلى المطالبة endpoints أشر يدويًا إلى أمثلة B).
- C) ونُفَّع عند العمل في مجلد endpoints تتضمن أمثلة `.claude/rules/api/` أعد قواعد خاصة بالمسارات في API.
- D) slash وتحتوي تعليمات اتباع النمط، وتُستدعى عند الحاجة عبر endpoints أنشئ مهارة تشير إلى أمثلة D) command. [CORRECT]

جديدة، وليس أثناء مهام غير endpoints المهارة التي تُستدعى عند الطلب تحمل سياق الأمثلة فقط عند توليد D: لماذا ذات صلة مثل التصحيح أو المراجعة. هذا يحافظ على نظافة السياق الرئيسي مع الحفاظ على جودة عالية عند التوليد.

39 Claude Code السيناريو: توليد الكود باستخدام) السؤال

عبر migration تأخذ اسم database migration. تولد ملفات `/migration` الموقف: أنشأ فريقك مهارة `$ARGUMENTS` مما ينتج arguments، في الإنتاج لاحظت ثلاث مشكلات: (1) كثيرًا ما يشغل المطورون المهارة دون.

من محادثات سابقة غير ذات صلة، و(3) شغل مطور schema ملفات بأسماء سيئة، (2) تستخدم المهارة أحيانًا تفاصيل تنظيميًا اختباريًا مدمرًا بالخطأ لأن المهارة كان لديها وصول واسع للأدوات.

أي نهج إعداد يصلح المشكلات الثلاث كلها؟

- افترض مدخلات محددة، وأضف مراجع \$ARGUMENTS مثل 1\$ و 2\$ بدل positional parameters استخدم A) .يحذر من العمليات المدمرة frontmatter صريحة عبر صيغة @ للتحكم في السياق، وأضف وصف schema لعزل context: fork لطلب المعاملات المطلوبة، واستخدم frontmatter في argument-hint أضف B) [CORRECT]. إلى عمليات كتابة الملفات allowed-tools التنفيذ، وقيد وأضف تعليمات تحقق لطلب اسم ، /migration-apply و /migration-create قسمها إلى مهارتين C) allowed-tools . عند غيابه، واستخدم نطاقات مختلفة ل migration اسم صالح، وأضف مطالبات لتجاهل سياق \$ARGUMENTS لضمان أن SKILL.md أضف تعليمات تحقق في D) .المحادثات السابقة، واسرد العمليات المحظورة.

يحسن إدخال المعاملات ويقلل `argument-hint`: هذا يستخدم ثلاث مزايا إعداد مختلفة لمعالجة كل مشكلة **B: لماذا** `allowed-tools` يمنع تسرب سياق المحادثات السابقة، و `context: fork` غيابها، و `context: fork` آمنة للملفات، مما يمنع الإجراءات المدمرة.

Claude Code السيناريو: توليد الكود باستخدام السؤال 40

مع functional style تستخدم React **الموقف**: تحتوي قاعدة الكود لديك على مناطق ذات اصطلاحات مختلفة: مكونات repository مع معالجة أخطاء محددة، ونماذج قاعدة البيانات تتبع async/await تستخدم API handlers و hooks، بجوار Button.test.tsx (مثل ملفات الاختبار موزعة عبر قاعدة الكود بجوار الكود المختبر pattern. وتريد أن تتبع كل الاختبارات الاصطلاحات نفسها بغض النظر عن الموقع، (Button.tsx

يطبق الاصطلاحات الصحيحة تلقائيًا عند توليد الكود؟ Claude ما الطريقة الأكثر دعمًا لضمان أن

- ليستنتج القسم Claude الجذري تحت عناوين لكل منطقة واعتمد على CLAUDE.md ضع كل الاصطلاحات في A) المناسب.
 - B) SKILL.md لكل نوع كود، وضع الاصطلاحات داخل كل `.claude/skills/` في Skills أنشئ
 - C) منفصلاً في كل مجلد فرعي يحتوي اصطلاحات تلك المنطقة CLAUDE.md ضع ملف
 - D) تطبيق الاصطلاحات glob patterns يحدد YAML frontmatter مع `.claude/rules/` أنشئ ملفات قواعد تحت
- [CORRECT] شرطياً بناءً على مسارات الملفات

تتيح تطبيقًا حتميًا للاصطلاحات بناءً على المسار، بغض النظر عن بنية المجلدات. هذا هو النهج `src/api/**/*.ts` و `**./*.test.tsx` مثل glob patterns وYAML frontmatter مع `.claude/rules/`. ملفات **D: لماذا** الأكثر دعمًا للأنماط العابرة للمجلدات مثل ملفات الاختبار الموزعة.

Claude Code السيناريو: توليد الكود باستخدام السؤال 41

يشغل قائمة مراجعة الكود القياسية لفريقك. يجب أن `/review` مخصص slash command **الموقف**: تريد إنشاء
يكون متاحًا لكل مطور عند استنساخ الريبو أو تحديثه.

أين ينبغي إنشاء ملف الأمر؟

- A) في `~/ .claude/commands/` لكل مطور home داخل مجلد.

- B) [CORRECT] .claude/commands/ في مستودع المشروع تحت
- C) في .claude/config.json كملصوفة commands.
- D) الجذري للمشروع CLAUDE.md في

داخل مستودع المشروع يجعلها مدارة عبر .claude/commands/ المخصصة تحت slash وضع أوامر B: لماذا ومتاحة تلقائيًا لكل مطور يستنسخ الريبو أو يحدثه. هذا هو الموقع المقصود لأوامر المشروع في version control Claude Code.

Claude Code السيناريو: توليد الكود باستخدام السؤال 42

وإرشادات الاختبار، TypeScript الخاص بفريقك أكثر من 500 سطر، ويمزج اصطلاحات CLAUDE.md الموقف: أصبح وإجراءات النشر. يجد المطورون صعوبة في العثور على الأقسام الصحيحة وتحديثها، API وأنماط

لتنظيم تعليمات المشروع إلى وحدات موضوعية مركزة؟ Claude Code ما النهج الذي يدعمه

- A) CLAUDE.md يربط أنماط الملفات بأقسام محددة داخل .claude/config.yaml عرّف ملف
- B) بحيث يغطي كل ملف موضوعًا واحدًا مثل ، .claude/rules/ منفصلة في Markdown أنشئ ملفات [CORRECT] testing.md و api-conventions.md
- C) تلقائيًا كتعليمات Claude في المجلدات ذات الصلة بحيث يحملها README.md قسّم التعليمات إلى ملفات
- D) في مستويات مختلفة من شجرة المجلدات، بحيث يتجاوز كل ملف CLAUDE.md أنشئ عدة ملفات باسم تعليمات الأب

منفصلة للتوجيهات Markdown حيث يمكنك إنشاء ملفات .claude/rules/ مجلد Claude Code يدعم B: لماذا مما يسمح للفريق بتنظيم مجموعات التعليمات الكبيرة ، api-conventions.md و testing.md الموضوعية مثل في وحدات مركزة وأسهل صيانة

Claude Code السيناريو: توليد الكود باستخدام السؤال 43

يستخدمها الفريق للعصف الذهني وتقييم مقاربات /explore-alternatives الموقف: تنشئ مهارة مخصصة اللاحقة بنقاش البدائل، وأحيانًا Claude التنفيذ قبل اختيار واحدة. يذكر المطورون أنه بعد تشغيل المهارة، تتأثر ردود تشير إلى مقاربات مرفوضة أو تحتفظ بسياق الاستكشاف مما يتداخل مع التنفيذ الفعلي

كيف ينبغي إعداد هذه المهارة بأكثر طريقة فعالية؟

- A) bash subprocess استخدم بادئة ! في المهارة لتشغيل منطق الاستكشاف كعملية
- B) [CORRECT] الخاص بالمهارة frontmatter في context: fork أضف
- C) لتحديد حدود سياق الاستكشاف الذي يجب تجاهله /explore-end و /explore-start: قسّمها إلى مهارتين
- D) .claude/skills/ بدل ~/.claude/skills/ أنشئ المهارة في

يشغل المهارة في سياق وكيل فرعي معزول، بحيث لا تلوث نقاشات الاستكشاف سجل context: fork لماذا B: المحادثة الرئيسي. هذا يمنع المقاربات المرفوضة وسياق العصف الذهني من التأثير على أعمال التنفيذ اللاحقة

Claude Code السيناريو: توليد الكود باستخدام) السؤال 44

لكل Claude Code عبر CI والتحقق من حالة PRs للبحث في GitHub MCP server الموقف: يريد فريقك إضافة شخصي. تريد أدوات متسقة عبر الفريق دون حفظ بيانات الاعتماد في GitHub واحد من ستة مطورين رمز وصول version control.

ما نهج الإعداد الأكثر فعالية؟

- A) `claude mcp add --scope user` . اجعل كل مطور يضيف الخادم في نطاق المستخدم عبر
 - B) ثم أضف، GitHub API ويعمل كوسيط لاستدعاءات `.env` . يقرأ الرموز من ملف MCP server wrapper أنشئ wrapper الخاص بالمشروع `.mcp.json` . إلى
 - C) `{GITHUB_TOKEN}` الخاص بالمشروع باستخدام استبدال متغير البيئة `.mcp.json` . أضف الخادم إلى [CORRECT] . الخاص بالمشروع README للمصادقة، ووثق متغير البيئة المطلوب في
 - D) . ثم أخبر المطورين بتجاوزه في إعداداتهم المحلية، token placeholder أعد الخادم في نطاق المشروع مع
- على مستوى المشروع مع استبدال متغيرات البيئة هو النهج الطبيعي: يوفر مصدرًا `.mcp.json` . استخدام C: لماذا مع السماح لكل مطور بتوفير بيانات اعتماده عبر متغيرات البيئة. توثيق MCP، واحدًا مضيوطًا بالإصدارات لإعداد أسهل دون حفظ الأسرار في المستودع onboarding المتغير يجعل

Claude Code السيناريو: توليد الكود باستخدام) السؤال 45

خارجية عبر قاعدة كود تحتوي على 120 ملقًا. API لمعالجة الأخطاء حول استدعاءات wrappers الموقف: تضيف العمل له ثلاث مراحل: (1) اكتشاف كل مواضع الاستدعاء والأنماط، (2) تصميم نهج معالجة الأخطاء بشكل تعاوني، و(3) مخرجات كبيرة تسرد مئات مواضع الاستدعاء مع السياق، Claude باتساق. في المرحلة الأولى، يولد wrappers تنفيذ مما يملأ نافذة السياق بسرعة قبل انتهاء الاكتشاف

ما النهج الأكثر فعالية لإكمال المهمة مع الحفاظ على اتساق التنفيذ؟

- A) للمرحلة الأولى لعزل مخرجات الاكتشاف المطولة وإرجاع ملخص، ثم تابع Explore subagent استخدم [CORRECT] .المرحلتين 2 و3 في المحادثة الرئيسية
- B) دوريًا لتقليل استخدام السياق أثناء التنقل بين `/compact` نفذ كل المراحل في المحادثة الرئيسية، مع استخدام الملفات.
- C) batch مع تمرير ملخصات سياق صريحة بين استدعاءات `--continue` باستخدام headless mode انتقل إلى للحفاظ على الاستمرارية.
- D) ثم عالج الملفات على دفعات عبر جلسات متعددة مع الاعتماد `CLAUDE.md` عرّف نمط معالجة الأخطاء في .على ملف الذاكرة المشترك للاتساق

مخرجات الاكتشاف المطولة في سياق منفصل، ولا يعيد إلى المحادثة الرئيسية إلا Explore subagent يعزل A: لماذا ملخصًا موجزًا. هذا يحافظ على نافذة السياق الرئيسية لمراحل التصميم التعاوني والتنفيذ المتسق، وهي المراحل التي يكون فيها الاحتفاظ بالسياق أكثر قيمة

السيناريو: وكيل خدمة العملاء

السؤال 46 (السيناريو: وكيل خدمة العملاء)

عندما يسأل المستخدمون عن حالة الطلب، `get_customer` الموقف: أثناء الاختبار، تلاحظ أن الوكيل غالبًا يستدعي `lookup_order` ستكون أنسب. ما أول شيء ينبغي فحصه لمعالجة المشكلة؟ رغم أن

ما أول شيء ينبغي فحصه؟

- `lookup_order` لاكتشاف طلبات الطلبات وتوجيهها مباشرة إلى preprocessing تنفيذ مصنف A).
- تقليل عدد الأدوات المتاحة للوكيل لتبسيط الاختيار B).
- تغطي كل أنماط طلبات الطلبات الممكنة لتحسين اختيار الأداة system prompt إلى few-shot إضافة أمثلة C).
- فحص أوصاف الأدوات للتأكد من أنها تميز بوضوح وظيفة كل أداة D). [CORRECT]

أوصاف الأدوات هي المدخل الأساسي الذي يستخدمه النموذج لتحديد الأداة التي يستدعيها. عندما يختار D: لماذا الوكيل أداة خاطئة باستمرار، فأول خطوة تشخيصية هي التأكد من أن أوصاف الأدوات تفصل بوضوح بين وظيفة كل أداة وحدود استخدامها.

السؤال 47 (السيناريو: وكيل خدمة العملاء)

للطلب #1234. لكن عندما refund الموقف: يتعامل وكيلك مع الطلبات ذات المشكلة الواحدة بدقة 94%، مثل: "أحتاج للطلب #1234 وأريد أيضًا تحديث عنوان الشحن refund يضمّن العملاء عدة مشكلات في رسالة واحدة، مثل: "أحتاج للطلب #5678"، تنخفض دقة اختيار الأدوات إلى 58%. عادةً يحل الوكيل مشكلة واحدة فقط أو يخلط المعاملات بين الطلبات. ما النهج الأكثر فعالية لتحسين موثوقية الطلبات متعددة المشكلات؟

ما النهج الأكثر فعالية؟

- تستخدم استدعاء نموذج منفصل لتفكيك الرسائل متعددة المشكلات إلى طلبات preprocessing تنفيذ طبقة A).
- منفصلة، ومعالجة كل طلب باستقلال، ثم دمج النتائج.
- دمج الأدوات ذات الصلة في أدوات عامة أقل عددًا B).
- إلى المطالبة توضح الاستدلال الصحيح وتسلسل الأدوات للطلبات متعددة المشكلات few-shot إضافة أمثلة C).
- تنفيذ تحقق من الاستجابة يكتشف الإجابات غير المكتملة ويعيد مطالبة الوكيل تلقائيًا لحل المشكلات الفائتة D). [CORRECT]

التي توضح الاستدلال الصحيح وتسلسل الأدوات للطلبات متعددة المشكلات هي الأكثر فعالية few-shot أمثلة C: لماذا لأن الوكيل يعمل جيدًا أصلًا مع المشكلات المنفردة. ما يحتاجه هو إرشاد حول نمط تفكيك عدة مشكلات وتوجيه كل واحدة والحفاظ على فصل المعاملات.

السؤال 48 (السيناريو: وكيل خدمة العملاء)

للطلب #1234، يحل الوكيل المشكلة خلال 3-4 refund الموقف: تُظهر سجلات الإنتاج أنه في الطلبات البسيطة مثل استدعاءات أدوات بنسبة نجاح 91%. لكن في الطلبات المعقدة مثل "تمت فوترتي مرتين، والخصم لم يُطبق، وأريد الإلغاء"، يتجاوز المتوسط 12 استدعاء أداة مع نجاح 54% فقط. غالبًا يحقق في المشكلات بشكل تسلسلي ويجلب بيانات العميل نفسها مرارًا لكل مشكلة. ما التغيير الأكثر فعالية لتحسين التعامل مع الطلبات المعقدة؟

ما التغيير الأكثر فعالية؟

- إضافة نقاط تحقق صريحة بين المراحل، بحيث يُطلب من الوكيل تسجيل التقدم بعد حل كل مشكلة قبل الانتقال A) إلى التالية.
- وأدوات الفوترة في أداة واحدة lookup_order و get_customer تقليل عدد الأدوات بدمج B) investigate_issue .
- تفكيك الطلب إلى مشكلات منفصلة، ثم التحقيق في كل واحدة بالتوازي باستخدام سياق العميل المشترك قبل C) [CORRECT] تركيب حل نهائي
- المثالية لسيناريوهات فوترة متعددة tool-call توضح تسلسلات system prompt إلى few-shot إضافة أمثلة D) الجوانب.

تفكيك الطلب إلى مشكلات منفصلة والتحقيق فيها بالتوازي مع سياق عميل مشترك يعالج المشكلتين C: لماذا الأساسيتين: يقلل جلب البيانات المكرر بإعادة استخدام السياق المشترك، ويقلل حلقات استدعاء الأدوات عبر تنفيذ التحقيقات بالتوازي قبل تركيب حل واحد.

السؤال 49 (السيناريو: وكيل خدمة العملاء)

الموقف: يحقق وكيلك 55% فقط من الحل من أول تواصل، وهو أقل بكثير من هدف 80%. تُظهر مراجعة السجلات أنه القياسية، لكنه يحاول حل استثناءات سياسية معقدة بنفسه. ما أفضل تدخل refund يصعد حالات بسيطة مثل طلبات أولي لتحسين المعايير؟

ما أفضل تدخل أولي؟

- اطلب من الوكيل تقييم ثقته من 1 إلى 10 قبل كل رد، والتوجيه تلقائيًا للبشر عندما تنخفض الثقة عن حد معين A)
- تاريخية للتنبؤ بالطلبات التي تحتاج تصعيدًا قبل بدء الوكيل الرئيسي tickets انشر مصنعًا منفصلًا مدرّبًا على B)
- توضح متى يصعد ومتى يحل ذاتيًا few-shot مع أمثلة system prompt أضف معايير تصعيد صريحة إلى C) [CORRECT]
- لتحديد مستوى إحباط العميل والتصعيد تلقائيًا عند تجاوز عتبة سلبية sentiment analysis نفذ D)

تعالج السبب الجذري مباشرة: حدود قرار غير واضحة بين الحالات few-shot معايير التصعيد الصريحة مع أمثلة C: لماذا البسيطة والمعقدة. هذا هو التدخل الأكثر تناسبًا وفعالية أولاً لأنه يعلم الوكيل متى يصعد ومتى يحل ذاتيًا دون إضافة بنية تحتية جديدة.

السؤال 50 (السيناريو: وكيل خدمة العملاء)

أصبح لدى الوكيل كل بيانات النظام المتاحة لكنه ما زال lookup_order و get_customer الموقف: بعد استدعاء escalate_to_human ؟ يواجه عدم يقين. أي موقف هو الأكثر تبريرًا لاستدعاء

أي موقف يبرر التصعيد أكثر؟

- عميل يريد إلغاء طلب شحن أمس وسيصل غدًا. ينبغي للوكيل التصعيد لأن العميل قد يغيّر رأيه بعد استلام A) الشحنة.
- عميل يدعي أنه لم يستلم طلبًا، لكن التتبع يظهر أنه تم التسليم والتوقيع في عنوانه قبل ثلاثة أيام. ينبغي للوكيل B) التصعيد لأن عرض دليل يناقض كلام العميل قد يضر العلاقة.
- عميل يطلب مطابقة سعر منافس. سياساتك تسمح بتعديل السعر عند انخفاض السعر في موقعك خلال 14 C) [CORRECT] يومًا، لكنها لا تذكر أسعار المنافسين. ينبغي للوكيل التصعيد لتفسير السياسة

- رسالة العميل تحتوي سؤالاً عن الفوترة وطلب إرجاع منتج. ينبغي للوكيل التصعيد حتى ينسق الإنسان (D) المشكلات في تفاعل واحد.

هذه فجوة سياسة حقيقية: قواعد الشركة تغطي انخفاض الأسعار في موقعك لكنها لا تغطي مطابقة أسعار C لماذا المنافسين. لا ينبغي للوكيل اختراع سياسة، بل يصعد للحكم البشري حول تفسير أو توسيع القواعد الحالية.

السؤال 51 (السيناريو: وكيل خدمة العملاء)

lookup_order ويستدعي get_customer الموقف: تُظهر سجلات الإنتاج أنه في 12% من الحالات، يتخطى الوكيل غير صحيحة. ما refunds مباشرة باستخدام الاسم الذي قدمه العميل فقط، مما يؤدي أحياناً إلى تحديد حسابات خاطئة و التغيير الأكثر فعالية لإصلاح مشكلة الموثوقية هذه؟

ما التغيير الأكثر فعالية؟

- أولاً، حتى عندما يقدم العملاء تفاصيل get_customer توضح أن الوكيل يستدعي دائماً few-shot إضافة أمثلة A) الطلب طوعاً.
- يحلل كل طلب ويفعل فقط مجموعة فرعية من الأدوات المناسبة لذلك النوع من الطلبات routing تنفيذ مصنف B)
- معرف get_customer حتى يرجع process_refund و lookup_order إضافة شرط برمجي مسبق يمنع C) عميل محققاً [CORRECT]
- إلزامي قبل أي عمليات طلبات get_customer بعبارة أن التحقق من العميل عبر system prompt تقوية D)

الشرط البرمجي المسبق يوفر ضماناً حتمياً بأن التسلسل المطلوب سيُتبع. هذا هو النهج الأكثر فعالية لأنه C لماذا LLM. يزيل احتمال تخطي التحقق بغض النظر عن سلوك

السؤال 52 (السيناريو: وكيل خدمة العملاء)

الموقف: تُظهر مقاييس الإنتاج أن رضا العملاء في نزاعات الفوترة المعقدة أو الإرجاعات متعددة الطلبات أقل بـ 15% من الحالات البسيطة، حتى عندما يكون الحل صحيحاً تقنياً. يوضح تحليل السبب الجذري أن الوكيل يقدم حلولاً دقيقة لكنه يشرح المبرر بشكل غير متسق: أحياناً يحذف تفاصيل سياسة ذات صلة، وأحياناً يفوت معلومات الجدول الزمني أو الخطوات التالية. فجوات السياق المحددة تختلف من حالة لأخرى. تريد تحسين جودة الحل دون إضافة رقابة بشرية.

ما النهج الأكثر فعالية؟

- إضافة قالب استجابة منظم يتطلب من الوكيل تضمين المبرر، والسياسة ذات الصلة، والجدول الزمني، A) [CORRECT]. والخطوات التالية قبل إنهاء الحالات المعقدة
- تصعيد كل الحالات المعقدة إلى إنسان حتى لا يترك الوكيل تفاصيل تفسيرية ناقصة B)
- مطالبة الوكيل بأن يكون "أكثر تفصيلاً" في الحالات المعقدة C)
- إضافة حد أدنى لطول الاستجابة في الحالات المعقدة لضمان شرح كافٍ D)

المشكلة ليست دقة الحل بل اكتمال الشرح واتساقه. قالب استجابة منظم يجعل العناصر المهمة مطلوبة A لماذا صراحة، مثل السياسة والجدول الزمني والخطوات التالية، دون الحاجة إلى تصعيد بشري أو الاعتماد على تعليمات عامة.

السؤال 53 (السيناريو: وكيل خدمة العملاء)

ثم ، `lookup_order` ثم ، `get_customer` الموقف: في نزاعات الفوترة، يتبع الوكيل نمطًا تسلسليًا: يستدعي مرة أخرى بعد كل تفصيل جديد. ينتج عن `check_billing` مرة أخرى، ثم `lookup_order` ثم ، `check_billing` ذلك زمن استجابة مرتفع وتكرار في الأدوات، مع أن معظم البيانات الأساسية يمكن جلبها مرة واحدة وإعادة استخدامها.

ما التغيير الأكثر فعالية؟

- توجيه الوكيل إلى جمع سياق العميل والطلب والفوترة مرة واحدة في بداية الحالة، ثم استخدام هذا السياق A) [CORRECT]
- المشترك أثناء حل المشكلات
- حتى يستطيع الوكيل الاحتفاظ بمزيد من مخرجات الأدوات `max_tokens` زيادة B)
- دمج كل أدوات العميل والطلب والفوترة في أداة واحدة كبيرة C)
- إضافة تعليمات تطلب من الوكيل تقليل عدد استدعاءات الأدوات D)

المشكلة هي نمط استدعاءات متكرر وتسلسلي. جمع السياق المشترك مرة واحدة ثم إعادة استخدامه يعالج A: لماذا التكرار مباشرة، ويقلل زمن الاستجابة دون توسيع الأدوات أو الاعتماد على تعليمات عامة

السؤال 54 (السيناريو: وكيل خدمة العملاء)

الموقف: تُظهر سجلات الإنتاج نمطًا: يشير العملاء إلى مبالغ محددة، مثل "الخصم 15% الذي ذكرته"، لكن الوكيل يرد بقيم خاطئة. يوضح التحقيق أن هذه التفاصيل ذُكرت قبل أكثر من 20 دورًا ثم صُغِطت في ملخصات مبهمه مثل "تمت مناقشة تسعير ترويجي".

ما الإصلاح الأكثر فعالية؟

- زيادة عتبة التلخيص من 70% إلى 85% حتى يكون للمحادثات مساحة أكبر قبل تشغيل التلخيص A)
- عندما يكتشف الوكيل إشارات مثل "كما retrieval تخزين سجل المحادثة الكامل في تخزين خارجي وتنفيذ B)
- مستمرة تُدرج في كل "case facts" استخراج الحقائق التبادلية، مثل المبالغ والتواريخ وأرقام الطلبات، إلى كتلة C) [CORRECT]
- تعديل مطالبة التلخيص لتطلب صراحة الحفاظ على كل الأرقام والنسب والتواريخ وتوقعات العميل حرفيًا D)

التلخيص يفقد التفاصيل الدقيقة بطبيعته. استخراج الحقائق التبادلية إلى كتلة منظمة خارج السجل الملخص C: لماذا يحفظ المعلومات الحرجة بحيث تكون متاحة بثبات في كل مطالبة مهما طال عدد الأدوار التي تم تلخيصها

السؤال 55 (السيناريو: وكيل خدمة العملاء)

Claude لديك ترجع كل النتائج عند البحث بالاسم. حاليًا، عند وجود عدة نتائج، يختار `get_customer` الموقف: أداة العميل صاحب أحدث طلب، لكن بيانات الإنتاج تُظهر أن هذا يختار الحساب الخاطئ في 15% من الحالات الغامضة. كيف تعالج ذلك؟

كيف تعالج ذلك؟

- تنفيذ نظام درجات ثقة يتصرف ذاتيًا فوق 85% ثقة ويطلب توضيحًا تحت العتبة A)

- إلى طلب معرف إضافي، مثل البريد الإلكتروني أو الهاتف أو رقم الطلب، عندما يرجع Claude توجيهه B) `get_customer` [CORRECT]. عدة نتائج قبل اتخاذ أي إجراء خاص بالعميل
- لترجع تطابقًا واحدًا "الأكثر احتمالًا" بناءً على خوارزمية ترتيب، مما يزيل الغموض `get_customer` تعديل C)
- إلى المطالبة توضح الاستدلال الصحيح وتسلسل الأدوات للتطابقات الغامضة few-shot إضافة أمثلة D)

طلب معرف إضافي من المستخدم هو الطريقة الأكثر موثوقية لحل الغموض لأن المستخدم يملك معرفة B: لماذا قطعية بهويته. دور محادثة إضافي واحد ثمن بسيط لإزالة معدل خطأ 15% بسبب اختيار الحساب الخطأ

السؤال 56 (السيناريو: وكيل خدمة العملاء)

أولاً في 78% من الحالات. وعندما `get_customer` يستدعي الوكيل، "I want to check my account for an order I made yesterday"، يظهر سجلات الإنتاج نمطًا ثابتًا: عندما يضمن العملاء كلمة يستدعي، "I want to check an order I made yesterday" مثل، "account" يصيغ العملاء طلبات مشابهة دون أولاً في 93% من الحالات. أوصاف الأدوات واضحة وغير غامضة. ما السبب الجذري الأكثر احتمالًا `lookup_order` لهذا الفرق؟

ما السبب الجذري الأكثر احتمالًا؟

- على تعليمات حساسة للكلمات المفتاحية توجه السلوك بناءً على مصطلحات مثل system prompt يحتوي A) [CORRECT]. مما ينشئ أنماط اختيار أدوات غير مقصودة، "account"
- وعمليات العملاء تتجاوز أوصاف الأدوات "account" التدريب الأساسي للنموذج ينشئ ارتباطات بين مصطلحات B)
- على أمثلة تحتوي fine-tuning يحتاج النموذج إلى بيانات تدريب أكثر عن الرسائل متعددة المفاهيم وينبغي C) مصطلحات الحساب والطلب معًا.
- تحتاج أوصاف الأدوات إلى أمثلة سلبية إضافية تحدد متى لا تستخدم كل أداة لمنع هذا الالتباس الناتج عن الكلمة D) المفتاحية.

النمط المنهجي المدفوع بالكلمة المفتاحية (78% مقابل 93%) يشير بقوة إلى وجود منطق توجيه صريح في A: لماذا ويدفع الوكيل نحو أدوات العميل. وبما أن أوصاف الأدوات واضحة أصلاً، "account" يتفاعل مع كلمة system prompt. فالاختلاف يشير إلى تعليمات على مستوى المطالبة تخلق توجيهًا سلوكيًا غير مقصود

السؤال 57 (السيناريو: وكيل خدمة العملاء)

عندما يسأل المستخدمون عن الطلبات، مثل `get_customer` الموقف: تُظهر سجلات الإنتاج أن الوكيل غالبًا يستدعي "Gets customer information" (كلتا الأداتين لهما أوصاف مختصرة جدًا. `lookup_order` بدلاً من، "check my order #12345"، وتقبلان صيغ معرفات تبدو متشابهة. ما أول خطوة أكثر فعالية لتحسين موثوقية "Gets order details" / "lookup_order" اختيار الأداة؟

ما أول خطوة أكثر فعالية؟

- تحليل إدخال المستخدم قبل كل دور وتختار مسبقًا الأداة الصحيحة بناءً على الكلمات routing تنفيذ طبقة A) المفتاحية وأنماط المعارف.
- تستعلم عنه backend تقبل أي معرف وتقرر داخليًا أي `lookup_entity` دمج الأداتين في أداة واحدة B)
- توضح أنماط اختيار الأدوات الصحيحة، مع 5-8 أمثلة توجه system prompt إلى few-shot إضافة أمثلة C) استعلامات الطلبات إلى `lookup_order`.

- توسيع وصف كل أداة ليشمل صيغ الإدخال، وأمثلة الاستعلامات، والحالات الحدية، والحدود التي تشرح متى D) [CORRECT]. تستخدمها مقابل الأدوات المشابهة

توسيع أوصاف الأدوات بإضافة صيغ الإدخال، وأمثلة الاستعلامات، والحالات الحدية، والحدود الواضحة يعالج D: لماذا معلومات كافية للتمييز بين أدوات متشابهة. هذه خطوة أولى LLM السبب الجذري مباشرة: الأوصاف المختصرة لا تمنح منخفضة الجهد وعالية الأثر لأنها تحسن آلية الاختيار الأساسية التي يعتمد عليها النموذج.

السؤال 58 (السيناريو: وكيل خدمة العملاء)

تشغيل) يجب أن تقرر هل تواصل الحلقة، Claude API الموقف: تنفذ حلقة الوكيل لوكيل الدعم. بعد كل استدعاء أم تتوقف (عرض الإجابة النهائية للعميل). ما الذي يحدد هذا القرار؟ (مجددًا Claude الأدوات المطلوبة ثم استدعاء

ما الذي يحدد هذا القرار؟

- A) end_turn وتوقف إذا كان tool_use ؛ واصل إذا كان Claude في استجابة stop_reason افحص حقل [CORRECT]
- B) ؛ فالإشارات اللغوية "Can I help with anything else?" أو "I'm done" للبحث عن عبارات مثل Claude حلل نص B) الطبيعية تدل على اكتمال المهمة.
- C) يشير Claude عيّن حدًا أقصى للتكرارات، مثل 10 استدعاءات، وتوقف عند الوصول إليه بغض النظر عما إذا كان C) إلى الحاجة لمزيد من العمل.
- D) نصًا تفسيريًا فينبغي إنهاء الحلقة Claude ؛ إذا أنشأ assistant افحص هل تحتوي الاستجابة على نص D)

Claude يعني أن tool_use :المنظمة والصريحة للتحكم في الحلقة Claude هو إشارة stop_reason لماذا A: أكمل استجابته وينبغي إنهاء الحلقة Claude يعني أن end_turn يريد تشغيل أداة واستلام نتائجها، بينما

السؤال 59 (السيناريو: وكيل خدمة العملاء)

الزمنية من Unix لديك: طوابع MCP الموقف: تُظهر سجلات الإنتاج أن الوكيل يسيء تفسير مخرجات أدوات بعض (1=pending, 2=shipped) ورموز حالة رقمية ، lookup_order من ISO 8601 وتواريخ ، get_customer خارجية لا يمكنك تعديلها. أي نهج لتطبيع صيغ البيانات هو الأكثر قابلية للصيانة؟ MCP الأدوات من خوادم

أي نهج هو الأكثر قابلية للصيانة؟

- A) لا اعتراض مخرجات الأدوات وتطبيق تحويلات التنسيق قبل أن يعالجها الوكيل PostToolUse hook استخدم A) [CORRECT]
- B) لأدوات الطرف الثالث wrappers عدّل الأدوات التي تتحكم بها لترجع صيغًا مفهومة للبشر، وأنشئ B)
- C) يستدعيها الوكيل بعد كل جلب بيانات لتحويل القيم normalize_data أنشئ أداة C)
- D) يشرح اصطلاحات بيانات كل أداة system prompt أضف توثيقًا تفصيليًا للصيغ إلى D)

نقطة مركزية وحتمية لا اعتراض كل مخرجات الأدوات وتطبيعها، بما في ذلك بيانات PostToolUse hook يوفر A: لماذا الخارجية، قبل أن يعالجها الوكيل. هذا أكثر قابلية للصيانة لأن التحويلات تعيش في الكود وتطبق بشكل MCP خوادم LLM. موحد، بدل الاعتماد على تفسير

السؤال 60 (السيناريو: وكيل خدمة العملاء)

أنسب، خصوصًا `lookup_order` عندما تكون `get_customer` الموقف: تُظهر سجلات الإنتاج أن الوكيل يختار أحيانًا system إلى few-shot قررت إضافة أمثلة. "I need help with my recent purchase" في الاستعلامات الغامضة مثل لتحسين اختيار الأدوات. أي نهج يعالج المشكلة بأكبر فعالية؟

أي نهج هو الأكثر فعالية؟

- A) في وصف كل أداة لتغطية الحالات الغامضة "don't use when" و "use when" إضافة إرشادات صريحة.
- B) `lookup_order` معًا، ثم كل سيناريوهات `get_customer` إضافة أمثلة مجمعة حسب الأداة: كل سيناريوهات معًا.
- C) إضافة 4-6 أمثلة موجهة للحالات الغامضة، مع مبرر لكل مثال يوضح لماذا اختيرت أداة معينة بدل البدائل [CORRECT]. الممكنة
- D) إضافة 10-15 مثالًا لطلبات واضحة وغير غامضة توضح اختيار الأداة الصحيح للسيناريوهات النموذجية لكل أداة.

للحالات الغامضة المحددة التي تحدث فيها الأخطاء، مع مبرر صريح لسبب تفضيل few-shot استهداف أمثلة C: لماذا أداة على بدائلها، يعلّم النموذج عملية القرار المقارن المطلوبة للحالات الحديثة. هذا أكثر فعالية من الأمثلة العامة أو القواعد التصريحية.

السؤال 61 (السيناريو: أنماط معمارية الذكاء الاصطناعي الحواري)

لمعينة التأثيرات قبل التنفيذ. يُظهر `dry_run: boolean` معامل `remove_team_member` الموقف: تستخدم أداة تحتاج إلى ضمان أن كل `dry_run=false` مراقبة الإنتاج أن الوكيل يتجاوز خطوة المعالجة باستدعاء الأداة مباشرة مع عملية إزالة يسبقها عرض معالجة يؤكد أنها المستخدمة صراحة.

ما النهج الأكثر موثوقية؟

- A) بالمعاملات `dry_run=true` فقط إذا حدث استدعاء `dry_run=false` إضافة تحقق من جهة الخادم يسمح بـ A) نفسها خلال آخر 60 ثانية.
- B) لتطلب موافقة المستخدم قبل تمرير أي استدعاءات orchestration وسم الأداة بأنها تتطلب تأكيدًا، وإعداد طبقة B) للأدوات الموسومة.
- C) أولاً وانتظار `dry_run=true` إلى وصف الأداة تلزم الوكيل باستدعاء few-shot إضافة تعليمات مفصلة وأمثلة C) تأكيد المستخدم قبل الاستدعاء مرة أخرى.
- D) صالحًا للاستخدام confirmation token ترجع تفاصيل التأثير و `preview_remove_member`: استبدالها بأداتين D) [CORRECT]. تتطلب ذلك الرمز، بحيث يرتبط التنفيذ بالمعالجة `execute_remove_member` مرة واحدة، و

نهج الأدوات مع ربط التنفيذ بالرمز يجعل التنفيذ دون معالجة سابقة مستحيلًا معماريًا؛ أداة التنفيذ تتطلب حرفيًا D: لماذا للتعليمات، أو على LLM رمزًا لا تولده إلا أداة المعالجة. هذا يفرض القيد على مستوى الكود بدل الاعتماد على أمثال heuristics خارجية orchestration زمنية، أو على بنية heuristics.

السؤال 62 (السيناريو: أنماط معمارية الذكاء الاصطناعي الحواري)

نتج network timeouts تفشل بنسبة 12%: منها 8% أخطاء `search_catalog` الموقف: تُظهر مراقبة الإنتاج أن أداة لا تنجح مهما أُعيدت المحاولة. حاليًا يُرجع النوعان بالطريقة نفسها، مما query syntax عند إعادة المحاولة، و4% أخطاء. يسبب إعادة محاولات مهذرة.

كيف ينبغي تعديل معالجة الأخطاء في الأداة؟

- توضيح كيفية التمييز بين أخطاء الشبكة وأخطاء الصياغة system prompt إلى few-shot إضافة أمثلة A)
- على كل الأخطاء بشكل موحد exponential backoff retry logic تطبيق B)
- داخل الأداة، وإرجاع أخطاء الصياغة فورًا مع network timeout لأخطاء backoff تنفيذ إعادة محاولة تلقائية مع C) [CORRECT]
- تفاصيل التحقق من المعاملات
- وتفاصيل نوع الخطأ retryable إرجاع كل الأخطاء مع راية D)

التعامل مع إعادة المحاولة داخل الأداة للأخطاء العابرة هو الحد التجريدي الصحيح؛ الأداة تعرف نوع الخطأ: C لماذا prompt-level. أما اتباع تعليمات flag حتمي دون الاعتماد على الوكيل لتفسير retry logic بشكل حاسم ويمكنها تنفيذ التي لن تنجح syntax الموحد فيهدر الوقت على أخطاء backoff.

السؤال 63 (السيناريو: أنماط معمارية الذكاء الاصطناعي الحواري)

ثم قال لاحقًا ، "I have a very low risk tolerance" :الموقف: خلال عدة أدوار عن استراتيجية استثمار، قال المستخدم "What should I invest in?" :والآن يسأل "I want to maximize my returns."

أي نهج يضمن بأفضل شكل أن التوصية تتوافق مع أولوية المستخدم الفعلية؟

- [CORRECT]. إبراز التناقض وسؤال المستخدم توضيح أي الأمرين أهم A)
- تقديم توصيات منفصلة لكلا السيناريوهين B)
- المتابعة بناءً على آخر تفضيل ذكره المستخدم C)
- توصية بمحفظة متوازنة دون معالجة التعارض D)

عندما تتعارض تفضيلات المستخدم مباشرة، فإن إبراز التعارض وطلب التوضيح هو الطريقة الوحيدة لضمان A: لماذا أن التوصية تتوافق مع نيته الحقيقية. أي نهج آخر يتضمن افتراضًا قد يكون خاطئًا؛ تعظيم العوائد وتحمل المخاطر المنخفض هدفان متعارضان جوهريًا ويتطلبان قرارًا من المستخدم.

السؤال 64 (السيناريو: أنماط معمارية الذكاء الاصطناعي الحواري)

يسأل ، "I love jazz" عبر عدة أدوار. بعد رسالتين من قول المستخدم playlist الموقف: يضبط المستخدمون تفضيلات Claude: "What genres do you enjoy?"

ما السبب الأكثر احتمالاً؟

- للحفاظ على ذاكرة المحادثة vector database إلى اتصال Claude يحتاج A)
- تم تجاوز نافذة السياق الخاصة بالنموذج B)
- session_id معامل Claude API يتطلب C)
- messages تطبيقك لا يضمن الرسائل السابقة في مصفوفة D) [CORRECT]

إذا لم تُضمّن سجل المحادثة الكامل في API stateless ذاكرة من جهة الخادم؛ كل استدعاء Claude لا يملك D: لماذا ليس session_id و Vector databases. الأدوار السابقة Claude لكل طلب، فلن يعرف messages مصفوفة. وتجاوز نافذة السياق مستحيل في تبادل من رسالتين، Claude جزءًا من معمارية.

السؤال 65 (السيناريو: أنماط معمارية الذكاء الاصطناعي الحواري)

يتضمن السجل حساسية غذائية، tokens الموقف: بعد جلسة طبخ مدتها 40 دقيقة، وصلت المحادثة إلى 78,000 وتعديل كميات الوصفة، وتوضيح مصطلحات طبخ، ونقاشًا عامًا. يجب تقليل الرموز مع الحفاظ على المعلومات المهمة

أي نهج يوازن بأفضل شكل بين الحفظ وتقليل الرموز؟

- A) تلخيص سجل المحادثة كله.
- B) فقط tokens الاحتفاظ بآخر 20,000.
- C) استخراج البيانات الحرجة المنظمة (الحساسية، الكميات، التفضيلات)، وتلخيص النقاش العام، والاحتفاظ [CORRECT] بالتبادلات الأخيرة حرفيًا.
- D) semantic search تخزين المحادثة كاملة خارجيًا واسترجاع الأجزاء ذات الصلة عبر

النهج الهجين يحفظ أعلى المعلومات قيمة بأقل تكلفة. الحقائق الحرجة مثل الحساسية والكميات تُستخرج في C: لماذا كتلة منظمة مضغوطة، مما يمنع فقدان الدقة الذي يحدث أثناء التلخيص. أما النقاش العام فيُلخص، والتبادلات الأخيرة تعقيد معماري D يخاطران بفقدان معلومات غذائية حرجة، و A و B تُحفظ حرفيًا للحفاظ على اتساق الحوار. الخياران زائد لجلسة طبخ واحدة

السؤال 66 (السيناريو: أنماط معمارية الذكاء الاصطناعي الحواري)

الموقف: يذكر المستخدمون أنه أثناء المحادثات الطويلة يفقد المساعد تتبع المواضيع والتفضيلات السابقة. تنفيذك الحالي يحتفظ فقط بآخر 25 زوجًا من الرسائل

ما الحل الأكثر فعالية؟

- A) [CORRECT] نهج هجين: تلخيص الرسائل القديمة مع الاحتفاظ بالرسائل الحديثة حرفيًا
- B) على سجل المحادثة الكامل vector similarity search استخدام
- C) زيادة النافذة إلى 50 زوجًا من الرسائل
- D) في البداية running summary تلخيص الرسائل التي تُحذف في كل دور وإضافة

النهج الهجين يعالج بُعدين للمشكلة: الاحتفاظ بالسياق الحديث بشكل دقيق، وهو مهم لاتساق الحوار، مع A: لماذا الحفاظ على تمثيل مضغوط للتفضيلات القديمة حتى لا تضيق كليًا عند حذف الأزواج القديمة. زيادة النافذة تؤجل سياقًا مهمًا غير مشابه دلاليًا للاستعلام الحالي. أما التلخيص الكامل في كل vector search المشكلة فقط. وقد يفوّت دور فيضيف تكلفة وبراكم أخطاء التلخيص

السؤال 67 (السيناريو: أنماط معمارية الذكاء الاصطناعي الحواري)

الموقف: يذكر المستخدمون أن زمن الاستجابة والتكلفة يرتفعان عندما تتجاوز المحادثات 50 دورًا

ما السبب الأساسي؟

- A) [CORRECT] API. يتم تضمين سجل المحادثة الكامل مع كل طلب
- B) يولد النموذج ردودًا أطول تدريجيًا
- C) تبطؤ عمليات قاعدة البيانات كلما نما السجل
- D) يبني النموذج ملفًا داخليًا للمستخدم يتطلب معالجة أكثر

مع كل messages بالكامل، لذلك يجب تضمين سجل المحادثة الكامل في مصفوفة Claude API stateless **A: لماذا** طلب. ومع نمو المحادثات، يحمل كل طلب رموزًا أكثر، مما يزيد زمن المعالجة والتكلفة مباشرة. لا يحتفظ النموذج بأي حالة داخلية بين الاستدعاءات، وطول الرد ليس مرتبطًا بالضرورة بطول المحادثة.

السؤال 68 (السيناريو: أنماط معمارية الذكاء الاصطناعي الحواري)

What: عندما يسأل المستخدم tokens الموقف: بعد ثلاثة أشهر من جلسات أسبوعية، نما سجل المحادثة إلى 85,000. يعطي المساعد إجابات عامة بدل الرجوع إلى نقاشات سابقة "What did we conclude about the theme of isolation?"

ما النهج الأكثر فعالية؟

- A) rolling window truncation.
- B) progressive summarization الأساسية.
- C) semantic embeddings مع استرجاع التبادلات ذات الصلة [CORRECT].
- D) لتمييز استنتاجات النقاش structured XML tags إضافة.

C: لماذا على سجل المحادثة هو النهج الوحيد الذي يتوسع لثلاثة أشهر من النقاشات ويستطيع إظهار semantic search. يضغط progressive summarization معظم السجل، و rolling window. تبادلات محددة ذات صلة عند الطلب تتطلب إعادة هيكلة كل XML tags النقاشات إلى تجريدات تفقد الاستنتاجات المحددة التي يسأل عنها المستخدم، و المحتوى السابق ولا تحل مشكلة الاسترجاع بهذا الحجم.

السؤال 69 (السيناريو: أنماط معمارية الذكاء الاصطناعي الحواري)

في أول 10–15 دورًا، لكن الردود اللاحقة تحرف. ما system prompt إرشادات Claude يتبع، QA الموقف: أثناء اختبار زالت المحادثة ضمن حدود الرموز.

ما أفضل حل؟

- A) user. نقل الإرشادات السلوكية إلى أول رسالة.
- B) بدء محادثة جديدة بعد 20 دورًا.
- C) [CORRECT]. تعزز الإرشادات عند نقاط فاصلة في المحادثة user-role إدراج رسائل.
- D) لإعادة توليد الردود غير المتوافقة post-response validation استخدام.

C: لماذا مباشرة بإعادة تثبيت القيود في نقاط منتظمة مع تراكم instruction drift إدراج تذكيرات سلوكية دورية يعالج. post-response validation يقلل سلطتها، وبدء محادثة جديدة يدمر السياق، أما user سجل المحادثة. نقل الإرشادات إلى أول رسالة. فهو تصحيحي لا وقائي ويضيف زمن استجابة كبيرًا response validation.

السؤال 70 (السيناريو: أنماط معمارية الذكاء الاصطناعي الحواري)

يحدد منهجية التعليم وقواعد التكييف. بعد 12 tokens بطول 2,800 system prompt الخاص بك AI tutor الموقف: لدى دورًا، يبدأ المساعد بتجاهل مستويات الإثقان.

ما الإصلاح الأكثر فعالية؟

- A) إدراج تذكيرات كل 4–5 أدوار.

- B) [CORRECT]. توضح التكيّف مع مستوى الإتقان few-shot استبدال القواعد المطولة بأمثلة
- C) system prompt. وضع القواعد الحرجة في نهاية
- D). تقييم الردود وإعادة توليدها إذا لم يطابق مستوى الصعوبة

طويل مليء بقواعد تصريحية يكون عرضة للانحراف؛ لأن القواعد المجردة تتطلب من system prompt: لماذا B: مملوسة توضح التكيّف الصحيح مع few-shot النموذج التفكير فيها في كل دور. استبدال القواعد المطولة بأمثلة مستوى الإتقان يعطي النموذج أنماطاً سلوكية واضحة يطابقها، وهذا أكثر موثوقية عبر الأدوار من التعليمات المجردة. التذكيرات تساعد لكنها تعالج العرض، ووضع القواعد في النهاية يساعد في البداية فقط، وإعادة التوليد مكلفة وتصحيحية.

السؤال 71 (السيناريو: أنماط معمارية الذكاء الاصطناعي الحواري)

الموقف: يجب أن يحافظ المساعد على نبرة حماسية، ويشرح منطقته، ويسأل أسئلة توضيحية. أين ينبغي تعريف هذه الإرشادات السلوكية؟

أين ينبغي تعريف هذه الإرشادات السلوكية؟

- A) user. إضافتها قبل كل رسالة
- B) [CORRECT] system prompt. في
- C) assistant. في أول رسالة
- D) environment variables. في

لماذا B: system prompt: مصمم خصيصاً للقيود والإرشادات السلوكية المستمرة التي تطبق طوال المحادثة. إضافتها user قبل كل رسالة غير موثوقة لأن النموذج قد ينحرف عن تصريحاته السابقة، assistant تكرار زائد، وأول رسالة user قبل كل رسالة environment variables لا تؤثر في سلوك النموذج

السؤال 72 (السيناريو: أنماط معمارية الذكاء الاصطناعي الحواري)

“I’d be happy to help!” و “Certainly!” الموقف: يذكر المستخدمون بدايات ردود متكررة مثل

ما النهج الأكثر فعالية؟

- A) [CORRECT]. بداية رد مباشرة partial assistant message إلحاق
- B) temperature. خفض إعداد
- C) post-process. للردود لإزالة التحيات
- D) system prompt. لتجنب هذه العبارات إضافة تعليمات في

لماذا A: prefill بدل توليد prefill مباشرة يمنع أنماط التحية على مستوى التوليد؛ النموذج يكمل من assistant لبداية رد prefill: لماذا A: post-processing. قد تساعد لكنها أقل موثوقية لأن النموذج قد ينتج تنويعات system prompt عبارات افتتاحية جديدة. تعليمات يتحكم بالعشوائية لا بعبارات محددة temperature محل هش، و

السؤال 73 (السيناريو: أنماط معمارية الذكاء الاصطناعي الحواري)

نظامك بأن طرد المستخدم قد سُجن بينما المستخدم ما يزال في محادثة نشطة. تريد أن webhook الموقف: يخبر يدمج المساعد هذه المعلومة طبيعيًا في الرد التالي.

ما أفضل نهج؟

- A) system prompt إضافة حالة الشحن إلى
- B) synthetic user message إرسال
- C) status tool إجبار المساعد على استدعاء
- D) [CORRECT] إلحاق تحديث الحالة كبداية برسالة المستخدم التالية

إضافة تحديث الحالة كبداية لرسالة المستخدم التالية يحقن سياقًا آنيًا عند حد طبيعي في المحادثة دون تعطيل D: لماذا قد يكسر synthetic user message. يتطلب إعادة بناء الجلسة أو يكون مرهقًا معماريًا system prompt التدفق. تعديل attribution التدفق الطبيعي ولبس tool call وإجبار.

السؤال 74 (السيناريو: أنماط معمارية الذكاء الاصطناعي الحواري)

يطرح المساعد أكثر من 4 أسئلة "Book a venue for the party." الموقف: كثيرًا ما يرسل المستخدمون طلبات مثل abandonment% توضيحية، مما يسبب 35.

أي نهج يحسن المفاضلة بأفضل شكل؟

- A) مخفية defaults المتابعة باستخدام
- B) طرح كل الأسئلة التوضيحية في رسالة مركبة واحدة
- C) [CORRECT] ذكر الافتراضات صراحة والمتابعة مع دعوة المستخدم لتصحيحها
- D) structured intake form استخدام

ذكر الافتراضات صراحة والمتابعة يعطي المستخدم ردًا مفيدًا فورًا مع الحفاظ على قدرته على تصحيح C: لماذا المخفية لا تخبر المستخدم بما تم افتراضه، وقائمة الأسئلة المركبة ما تزال تطلب جهدًا defaults. الافتراضات الخاطئة abandonment يزيد الاحتكاك بدل تقليل structured form والنموذج upfront.

السؤال 75 (السيناريو: أنماط معمارية الذكاء الاصطناعي الحواري)

تتبع الأدوار الأولى القواعد، لكن عند الدور 7 يقدم contractor. بشخصية system prompt الموقف: يستخدم المساعد tokens المساعد نصائح عامة. طول المحادثة فقط 2,500.

ما السبب الأكثر احتمالًا؟

- A) لا تؤسس إلا السلوك الأولي system prompts
- B) يضعف انتباه النموذج مع تراكم الأدوار
- C) [CORRECT] system prompt يخفف تأثير assistant تراكم ردود
- D) مرة واحدة فقط system prompt يتم إرسال

system prompt داخل سجل المحادثة، تنخفض نسبة النص الذي يعكس قيود assistant مع تراكم ردود C: لماذا يبدأ النموذج أكثر فأكثر بمطابقة أنماط ردوده assistant. السلوكية مقارنة بجسم متزايد من المحتوى الذي ولده

يُضمّن في كل استدعاء system prompt. مما يراكم الانحراف حتى عند عدد رموز صغير، system prompt السابقة بدل tokens. ليس هو العامل في 2,500 attention degradation ليست تفسيرًا قائمًا بذاته، و D لذلك، API،

السؤال 76 (السيناريو: أنماط معمارية الذكاء الاصطناعي الحواري)

فيرد المساعد بطرح عدة أسئلة: "Can you help with the report?" الموقف: يرسل المستخدمون طلبات غامضة مثل abandonment. % أي تقرير؟ أي نوع من المساعدة؟ ما الموعد النهائي؟ وهذا يسبب 40

ما أفضل حل؟

- [CORRECT]. وضع افتراضات معقولة، وذكرها صراحة، وعرض تعديلها A)
- تصنيف الغموض باستخدام نموذج أصغر قبل الرد B)
- استخدام تفسيرات مسبقة دون ذكر الافتراضات C)
- قصر المساعد على سؤال توضيحي واحد في كل دور D)

المتابعة بافتراضات معقولة ومعلنة تلغي تبادل الأسئلة الطويل مع إبقاء المستخدم مطلعًا ومتحكمًا. A: لماذا التفسيرات الصامتة ترك المستخدم إذا لم تطابق نيته. حد السؤال الواحد ما يزال يتطلب أدواتًا من الأخذ والرد. ونموذج التصنيف الأصغر يضيف زمنيًا وتعقيد بنية دون حل مشكلة تجربة المستخدم الأساسية

تمارين عملية

التمرين 1: وكيل متعدد الأدوات مع منطق تصعيد

الهدف: تصميم حلقة وكيل مع تكامل الأدوات، ومعالجة أخطاء منظمة، وتصعيد

الخطوات:

1. بأوصاف مفصلة، وتضمنين أداتين متشابهتين لاختبار اختيار الأداة MCP عرّف 3-4 أدوات.
 2. نفذ حلقة وكيل تفحص (stop_reason ("tool_use" / "end_turn")).
 3. أضف استجابات أخطاء منظمة: errorCategory , isRetryable , description.
 4. يمنع العمليات فوق حد معين ويوجهها إلى التصعيد interceptor hook نفذ.
 5. اختبر الطلبات متعددة الجوانب.
- (السياق والموثوقية) 5، (MCP الأدوات و) المجالات: 1 (بنية الوكيل)، 2

لتطوير الفريق Claude Code التمرين 2: إعداد

MCP والأوامر المخصصة، والقواعد الخاصة بالمسارات، وخواص، CLAUDE.md الهدف: إعداد

الخطوات:

1. على مستوى المشروع يحتوي على المعايير العامة CLAUDE.md أنشئ.
2. أنشئ ملفات (paths: ["src/api/**/*"], paths: ["**/*.test.*"]).

3. على مستوى المشروع تحت Skill أنشئ `.claude/skills/` مع `context: fork` و `allowed-tools`.
 4. شخصي في `override` + باستخدام متغيرات البيئة `.mcp.json` في MCP أعد خادم `~/claude.json`.
 5. اختبر وضع التخطيط مقابل التنفيذ المباشر على مهام ذات تعقيد مختلف.
- (MCP الأدوات و) 2، (Claude Code إعدادات) المجالات: 3

التمرين 3: خط استخراج بيانات منظمة

للمخرجات المنظمة، وحلقات التحقق/إعادة المحاولة، ومعالجة الدفعات `tool_use`، و `JSON` الهدف: مخططات

الخطوات:

1. (nullable وحقول، "other" مع enums، required/optional حقول) JSON schema عرّف أداة استخراج مع.
2. ابن حلقة تحقق: عند الخطأ، أعد المحاولة مع المستند، والاستخراج غير الصحيح، وخطأ التحقق المحدد.
3. لمستندات بنى مختلفة few-shot أضف أمثلة.
4. `custom_id` مستند، وتعامل مع الإخفاقات عبر Message Batches API: 100 عبر batch processing استخدم.
5. وجّه الحالات إلى البشر: درجات ثقة على مستوى الحقول، وتحليل حسب نوع المستند.

المجالات: 4 (هندسة المطالبات)، 5 (السياق والموثوقية)

التمرين 4: تصميم وتصحيح خط بحث متعدد الوكلاء

الهدف: تنسيق الوكلاء الفرعيين، وتمرير السياق، وانتشار الأخطاء، والتركيب مع تتبع المصادر

الخطوات:

1. (ويتم تمرير السياق صراحة داخل المطالبات، "Task" يتضمن `allowedTools`) منسق مع أكثر من وكيل فرعي.
2. في استجابة واحدة `Task` شغل الوكلاء الفرعيين بالتوازي عبر عدة استدعاءات.
3. `claim`، `quote`، `source URL`، `publication date`: اطلب إخراجًا منظمًا من الوكيل الفرعي.
4. لوكيل فرعي: أعد سياق خطأ منظم إلى المنسق وتابع بالنتائج الجزئية `timeout` حاك.
5. اختبر البيانات المتعارضة: احتفظ بالقيمتين مع الإسناد، وافصل النتائج المؤكدة عن المتنازع عليها.

(السياق والموثوقية) 5، (MCP الأدوات و) المجالات: 1 (بنية الوكيل)، 2

الملحق: التقنيات والمفاهيم

التقنية	الجوانب الأساسية
Claude Agent SDK	إنشاء الوكلاء، (PostToolUse) الخطافات، <code>stop_reason</code> ، حلقات الوكيل، <code>AgentDefinition</code> ، <code>allowedTools</code> ، و <code>Task</code> الفرعيين عبر
Model Context Protocol (MCP)	متغيرات البيئة، <code>.mcp.json</code> ، أوصاف الأدوات، <code>isError</code> ، الأدوات، الموارد، MCP خوادم

Claude Code	<code>.claude/commands/</code> ، و <code>glob patterns</code> مع <code>.claude/rules/</code> ، و <code>CLAUDE.md</code> هرمية، و <code>compact</code> ووضع التخطيط، و <code>SKILL.md</code> مع <code>.claude/skills/</code> ، و <code>--resume</code> ، و <code>fork_session</code>
Claude Code CLI	<code>--json-schema</code> ، و <code>--output-format json</code> للوضع غير التفاعلي، و <code>--print</code> / <code>-p</code>
Claude API	<code>tool_choice</code> (<code>"auto"</code> / <code>"any"</code> / <code>forced</code>)، و <code>JSON schemas</code> مع <code>tool_use</code> ، و <code>system prompts</code> ، و <code>max_tokens</code> ، و <code>stop_reason</code>
Message Batches API	متعدد الأدوار tool calling لا يدعم، <code>custom_id</code> ، توفير 50%، نافذة تصل إلى 24 ساعة
JSON Schema	الوضع الصارم، <code>"other"</code> + detail، <code>enum</code> أنواع، <code>nullable</code> الحقول، <code>optional</code> مقابل <code>required</code>
Pydantic	التحقق من المخطط، الأخطاء الدلالية، حلقات التحقق/إعادة المحاولة
الأدوات المدمجة	الغرض ومعايير الاختيار — Read, Write, Edit, Bash, Grep, Glob
Few-shot prompting	أمثلة موجهة للحالات الغامضة، والتعميم إلى أنماط جديدة
Prompt chaining	تفكيك متتابع إلى تمريرات مركزة
نافذة السياق	scratchpad ملفات، "lost in the middle"، ميزانيات الرموز، التلخيص التدريجي
إدارة الجلسات	والجلسات المسماة، وعزل السياق، <code>fork_session</code> ، و <code>Resume</code>
معايرة الثقة	درجات على مستوى الحقول، والمعايرة على بيانات موسومة، والعينات التطبيقية

مواضيع خارج النطاق

المواضيع المجاورة التالية لن تكون ضمن الاختبار

- أو تدريب نماذج مخصصة Claude نماذج Fine-tuning
- أو الفوترة أو إدارة الحساب Claude API مصادقة
- تفاصيل التنفيذ في لغات برمجة أو أطر عمل محددة، باستثناء ما يلزم لإعداد الأدوات/المخططات
- container orchestrationمثل البنية التحتية، والشبكات، و MCP نشر أو استضافة خوادم
- model weights أو عملية التدريب، أو Claude البنية الداخلية لـ
- أو منهجيات تدريب السلامة RLHF أو Constitutional AI
- vector database أو تفاصيل تنفيذ embeddings نماذج
- مثل أتمتة المتصفح أو التفاعل مع سطح المكتب، Computer use
- (Vision) قدرات تحليل الصور
- Streaming API أو server-sent events
- التفصيلية API أو حسابات تكلفة quotas أو Rate limiting
- أو تفاصيل بروتوكولات المصادقة API keys أو تدوير OAuth
- Azure أو GCP أو AWS إعدادات خاصة بمزودي السحابة مثل

- مقاييس الأداء أو مقارنة النماذج
- باستثناء معرفة أنه موجود، prompt caching تفاصيل تنفيذ
- tokenization خوارزميات عدّ الرموز أو تفاصيل

توصيات الاستعداد

1. ومعالجة الأخطاء، وإدارة، tool calling نفذ حلقة وكيل كاملة مع — Claude Agent SDK ابن وكيلاً باستخدام. الجلسات. تدرب على الوكلاء الفرعيين وتميرير السياق الصريح.
2. والقواعد الخاصة بالمسارات في، CLAUDE.md لمشروع حقيقي — استخدم هرمية Claude Code أعد MCP. وتكامل خوادم، allowed-tools و context: fork والمهارات مع، .claude/rules/.
3. retry flags، أوصافاً تميز الأدوات المتشابهة، وأرجع أخطاء منظمة مع فئات و — MCP صمم واختبر أدوات. واختبرها ضد طلبات مستخدم غامضة.
4. وحقول، validation/retry وحلقات، JSON schemas مع tool_use ابن خط استخراج بيانات — استخدم optional/nullable، Message Batches API ومعالجة الدفعات عبر.
5. للحالات الغامضة، ومعايير مراجعة صريحة، وبنى متعددة few-shot تدرب على هندسة المطالبات — أضف أمثلة. المرور لمراجعات الكود الكبيرة.
6. وفوض، scratchpad ادرس أنماط إدارة السياق — استخراج الحقائق من المخرجات المطولة، واستخدم ملفات الاكتشاف إلى الوكلاء الفرعيين للتعامل مع حدود السياق.
7. افهم التصعيد وإدخال الإنسان في الحلقة — متى تصعد: فجوات السياسة، طلب المستخدم الصريح، عدم القدرة. التوجيه حسب الثقة workflows على إحراز تقدم، و.
8. خذ اختباراً تدريبياً قبل الاختبار الحقيقي. فهو يستخدم السيناريوهات والصيغة نفسها.